



MANUAL BOOK

**PENERAPAN *FEW-SHOT ASPECT-BASED SENTIMENT ANALYSIS* UNTUK KOMENTAR AKREDITASI BERBAHASA INDONESIA:
PERBANDINGAN MODEL *BASELINE* DAN MODEL
TERAUGMENTASI TESAUROS**

PEMEGANG HAK CIPTA

- 1. FARIS FARHAN**
- 2. DR. ARIANA YUNITA**
- 3. ADE IRAWAN, PH.D**

**PROGRAM STUDI ILMU KOMPUTER
UNIVERSITAS PERTAMINA
JULI 2026**



A. Gambaran Umum

Manual book ini disusun sebagai panduan teknis untuk penggunaan dan implementasi model *few-shot Aspect Based Sentiment Analysis* (ABSA) berbasis IndoBERT, yang diperkaya dengan tesaurus domain hasil ekstraksi *large language model* (LLM). Panduan dirancang sebagai langkah untuk menganalisis inkonsistensi penilaian akreditasi melalui deteksi relevansi aspek (*Aspect Category Detection* / ACD) dan klasifikasi sentimen (*Aspect Sentiment Classification* / ASC). Panduan ini dibuat untuk memandu pengguna dalam melakukan instalasi lingkungan kerja yang dibutuhkan, menjelaskan alur kerja teknis dari pra-pemrosesan data, konstruksi tesaurus, hingga pelatihan dan evaluasi model, serta memberikan instruksi untuk menjalankan setiap *script* kode secara berurutan.

B. Tujuan Sistem

Sistem ini bertujuan untuk mengolah data komentar mentah dari asesor akreditasi, membangun tesaurus dan leksikon sentimen domain menggunakan LLM, menerapkan augmentasi data berbasis tesaurus tersebut pada model IndoBERT yang dilatih dalam kondisi *few-shot*, hingga menghasilkan data terstruktur dan metrik evaluasi yang digunakan untuk menganalisis potensi inkonsistensi penilaian antara tahap *Asesmen Kecukupan* (AK) dan *Asesmen Lapangan* (AL).

C. Spesifikasi Perangkat

Perangkat Keras:

- Prosesor : Intel Core i7 atau setara (*multi-core*, untuk menjalankan Ollama secara lokal)
- RAM : minimal 16 GB — dibutuhkan untuk memuat model LLM (**gemma4:e4b** untuk ekstraksi, serta **qwen3.5:8b** dan **llama3.1:8b** untuk validasi) melalui Ollama secara bersamaan dengan proses Python/pandas
- GPU (disarankan) : NVIDIA GPU dengan dukungan CUDA dan VRAM minimal 8–12 GB (mis. Google Colab GPU runtime) untuk mempercepat inferensi LLM maupun pelatihan IndoBERT
- Penyimpanan : ruang kosong beberapa GB per model untuk mengunduh *weight* model melalui `ollama pull`

Perangkat Lunak:

- Aplikasi : Google Colaboratory atau Jupyter Notebook
- *Runtime* LLM : Ollama (untuk menjalankan model **gemma4:e4b**, **qwen3.5:8b**, dan **llama3.1:8b**)
- Pustaka Utama : pandas, numpy, nltk, transformers, torch, scikit-learn, iterative-stratification, ollama, openpyxl, matplotlib, seaborn, tqdm

D. Menu dan Cara Penggunaan

1. Persiapan Lingkungan

- Siapkan aplikasi yang dapat menjalankan file dengan ekstensi `.ipynb`, misalnya Google Colaboratory.

- Untuk tahap konstruksi tesaurus (ACD/ASC), pastikan Ollama telah terpasang dan model yang dibutuhkan telah diunduh (`ollama pull`).

2. Persiapan Data

- Pastikan file dataset penelitian sudah diunggah ke Google Drive Anda.
- Jika menggunakan Google Colaboratory, unggah file dataset tersebut ke sesi Colab atau ke Google Drive Anda.
- Sesuaikan baris kode `BASE_PATH` pada setiap *cell* konfigurasi agar sesuai dengan lokasi file Anda.

3. Eksekusi Program

- Sistem ini terdiri dari beberapa file `.ipynb` yang harus dijalankan secara berurutan sesuai tahapan di bawah, dan untuk menjalankan keseluruhan program pada setiap file, klik menu navigasi *Runtime* > *Run all*.
- Beberapa tahap (konstruksi tesaurus ACD/ASC dan kombinasi AK+AL) memproses data AK dan AL secara terpisah; sesuaikan konfigurasi sesuai catatan pada tiap file.

Catatan: setiap tahap pada bagian D di bawah disarankan dijalankan sebagai satu *file .ipynb* yang terpisah, agar alur kerja tetap modular dan mudah ditelusuri apabila terjadi kesalahan pada salah satu tahap.

Tahapan yang dicakup dalam manual book ini:

1. Tahap Preprocessing
2. Tahap Deteksi dan Penyamaran Identitas Institusi
3. Tahap Konstruksi Tesaurus - ACD (AK/AL)
4. Tahap Konstruksi Leksikon Sentimen - ASC (AK/AL)
5. Tahap Analisis & Kombinasi Tesaurus - ACD (AK + AL)
6. Tahap Analisis & Kombinasi Sentimen - ASC (AK + AL)
7. Tahap *Few-Shot Fine-Tuning* - Baseline (IndoBERT)
8. Tahap *Few-Shot Fine-Tuning* - Tesaurus (IndoBERT)
9. Tahap Analisis Metrik Terpadu - Baseline vs Tesaurus

1 Tahap Preprocessing

Langkah ini dilakukan sebagai tahapan persiapan data mentah sebelum digunakan pada proses ekstraksi maupun pemodelan. Data dipisahkan menjadi dua subset berdasarkan tahap asesmen, yaitu Asesmen Kecukupan (AK) dan Asesmen Lapangan (AL), agar karakteristik duplikasi dan nilai kosong pada masing-masing tahap dapat ditangani dengan aturan yang sesuai.

```
[ ]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import re
import string
```

```
[ ]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[ ]: df = pd.read_excel('/content/drive/MyDrive/file_name.xlsx') # Modifikasi ↵
      ↪file_name
```

```
[ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20850 entries, 0 to 20849
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            20850 non-null  int64
1   nomor_usulan          20850 non-null  object
2   nomor_aspek           20850 non-null  object
3   keterangan_aspek      20850 non-null  object
4   komen_AK              20846 non-null  object
5   nilai_AK              20850 non-null  float64
6   no_asesor_AK          20850 non-null  int64
7   tipe_asesor           20850 non-null  int64
8   komen_AL              20850 non-null  object
9   nilai_AL              20850 non-null  object
10  kode_asesor_AL         20850 non-null  object
11  tipe_asesor_AL         20850 non-null  object
dtypes: float64(1), int64(3), object(8)
memory usage: 1.9+ MB
```

1.1 Filter hanya yang mengandung 'C9.2.A'

```
[ ]: df = df[df['nomor_aspek'].str.startswith('C9.2.A')]
      output_path = '/content/drive/MyDrive/c92a.xlsx'
      df.to_excel(output_path, index=False)
```

```
[ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 578 entries, 66 to 20799
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Unnamed: 0            578 non-null    int64
 1   nomor_usulan          578 non-null    object
 2   nomor_aspek           578 non-null    object
 3   keterangan_aspek      578 non-null    object
 4   komen_AK              578 non-null    object
 5   nilai_AK              578 non-null    float64
 6   no_asesor_AK          578 non-null    int64
 7   tipe_asesor           578 non-null    int64
 8   komen_AL              578 non-null    object
 9   nilai_AL              578 non-null    object
10   kode_asesor_AL        578 non-null    object
11   tipe_asesor_AL        578 non-null    object
dtypes: float64(1), int64(3), object(8)
memory usage: 58.7+ KB
```

```
[ ]: selected_columns = ['nomor_usulan',
                        'komen_AK', 'no_asesor_AK',
                        'komen_AL', 'kode_asesor_AL', 'tipe_asesor_AL']
```

```
[ ]: df[selected_columns] = df[selected_columns].replace('(NULL)', np.nan).
      infer_objects(copy=False)
      print(df[selected_columns].isna().sum())
```

```
nomor_usulan      0
komen_AK          0
no_asesor_AK      0
komen_AL          16
kode_asesor_AL    296
tipe_asesor_AL    296
dtype: int64
```

```
/tmp/ipykernel_4883/1467794762.py:1: FutureWarning: Downcasting behavior in
`replace` is deprecated and will be removed in a future version. To retain the
old behavior, explicitly call `result.infer_objects(copy=False)`. To opt-in to
the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
df[selected_columns] = df[selected_columns].replace('(NULL)',
```

```
np.nan).infer_objects(copy=False)
```

```
[ ]: null_komen_al_rows = df[df['komen_AL'].isna()]
      display(null_komen_al_rows)
```

1.2 Pisah subset (AK dan AL)

```
[ ]: ak_columns = [
      'nomor_usulan',
      'komen_AK',
      'no_asesor_AK'
    ]

    al_columns = [
      'nomor_usulan',
      'komen_AL',
      'kode_asesor_AL',
      'tipe_asesor_AL'
    ]
```

```
[ ]: df_ak = df[ak_columns].copy()
      df_al = df[al_columns].copy()

      df_ak.to_excel('/content/drive/MyDrive/komentar_AK.xlsx', index=False)
      df_al.to_excel('/content/drive/MyDrive/komentar_AL.xlsx', index=False)
```

```
[ ]: print(f"Komentar AK rows: {len(df_ak)}")
      print(f"Komentar AL rows: {len(df_al)}")
```

Komentar AK rows: 578

Komentar AL rows: 578

1.3 File AK

```
[ ]: df_ak.replace('(NULL)', np.nan).infer_objects(copy=False)
      print(df_ak.isna().sum())
```

```
nomor_usulan    0
komen_AK        0
no_asesor_AK    0
dtype: int64
```

```
[ ]: duplicate_count = df_ak.duplicated(
      subset=['nomor_usulan', 'komen_AK']
    ).sum()

      print(f"Number of duplicates: {duplicate_count}")
```

Number of duplicates: 2

```
[ ]: # Remove duplicates based on nomor_usulan and komen_AK
before_rows = len(df_ak)

df_ak = df_ak.drop_duplicates(
    subset=['nomor_usulan', 'komen_AK'],
    keep='first'
).reset_index(drop=True)

after_rows = len(df_ak)

print(f"Removed {before_rows - after_rows} duplicate rows")
print(f"Remaining rows: {after_rows}")

# Save cleaned file
df_ak.to_excel('/content/drive/MyDrive/komentar_AK.xlsx', index=False)
```

Removed 2 duplicate rows
Remaining rows: 576

```
[ ]: print(df_ak.shape)

(576, 3)
```

1.4 File AL

1.5 Missing Values

```
[ ]: df_al.replace('(NULL)', np.nan).infer_objects(copy=False)
print(df_al.isna().sum())
```

```
nomor_usulan      0
komen_AL          16
kode_asesor_AL    296
tipe_asesor_AL    296
dtype: int64
```

```
[ ]: komen_al_rows = df_al[df_al['komen_AL'].isna()]
display(komen_al_rows)
```

```
[ ]: df_al = df_al.dropna(subset=['komen_AL']).reset_index(drop=True)

df_al.to_excel('/content/drive/MyDrive/komentar_AL.xlsx', index=False)
```

```
[ ]: print("\nNull values after deletion:")
print(df_al.isna().sum())

print(f"\nRemaining rows: {len(df_al)}")
```

Null values after deletion:

```
nomor_usulan      0
komen_AL          0
kode_asesor_AL    281
tipe_asesor_AL    281
dtype: int64
```

Remaining rows: 562

```
[ ]: # Check whether all missing kode_asesor_AL rows are exactly
# the same rows as the missing tipe_asesor_AL rows
kode_null = df_al['kode_asesor_AL'].isna()
tipe_null = df_al['tipe_asesor_AL'].isna()

print("\nSame rows?")
print((kode_null == tipe_null).all())

mismatch_rows = df_al[kode_null != tipe_null]
print("\nRows with mismatched missing values:", len(mismatch_rows))
```

Same rows?

True

Rows with mismatched missing values: 0

1.6 Check Duplicates

```
[ ]: print(
    df_al.duplicated(
        subset=['nomor_usulan', 'komen_AL']
    ).sum()
)
```

281

```
[ ]: # Select rows where BOTH assessor-related columns are missing
null_rows = df_al[
    df_al['kode_asesor_AL'].isna() &
    df_al['tipe_asesor_AL'].isna()
]

# Select all rows that belong to a duplicate group based on proposal number,
↳ aspect, and comment
duplicate_rows = df_al[
    df_al.duplicated(
        subset=['nomor_usulan', 'komen_AL'],
        keep=False
    )
]
```

```

]

# Print the number of rows with missing assessor information
print(len(null_rows))

# Print the total number of rows involved in duplicate groups
print(len(duplicate_rows))

# Keep only the columns used to define duplicates, then remove repeated
↳ occurrences of those combinations
duplicate_keys = duplicate_rows[
    ['nomor_usulan', 'komen_AL']
].drop_duplicates()

# Check whether each null row has a matching duplicate key by joining the null
↳ rows with the duplicate combinations
check = null_rows.merge(
    duplicate_keys,
    on=['nomor_usulan', 'komen_AL'],
    how='inner'
)

# Print how many null rows are also duplicate rows
print(f"Matching rows: {len(check)}")

```

```

281
562
Matching rows: 281

```

```

[ ]: display(
    duplicate_rows.sort_values(
        ['nomor_usulan', 'komen_AL']
    ).head(20)
)

```

```

[ ]: # Remove rows where BOTH assessor columns are missing
df_al = df_al.drop(
    df_al[
        df_al['kode_asesor_AL'].isna() &
        df_al['tipe_asesor_AL'].isna()
    ].index
).reset_index(drop=True)

```

```

[ ]: # Verify remaining null values
print(df_al.isna().sum())

```

```

nomor_usulan    0
komen_AL        0

```

```
kode_asesor_AL    0
tipe_asesor_AL    0
dtype: int64
```

```
[ ]: # Save updated dataframe
df_al.to_excel('/content/drive/MyDrive/komentar_AL.xlsx', index=False)

print(f"Remaining rows: {len(df_al)}")
print("Updated file saved to /content/drive/MyDrive/komentar_AL.xlsx")
```

Remaining rows: 281

Updated file saved to /content/drive/MyDrive/TA/komentar_AL.xlsx

2 Tahap Deteksi dan Penyamaran Identitas Institusi

Langkah ini dilakukan untuk menjaga kerahasiaan institusi yang dinilai, karena komentar asesor berbentuk narasi bebas yang kadang menyebut identitas institusi secara langsung maupun tidak langsung. Deteksi dilakukan melalui dua tahap, yaitu pencarian pola struktural menggunakan sebelas pola regular expression dan peninjauan token berbasis frekuensi untuk menangkap akronim maupun singkatan. Kedua tahap ini hanya menandai kandidat teks; seluruh hasil penandaan kemudian ditinjau dan disamakan secara manual dengan placeholder generik seperti universitas x atau program studi x.

```
[ ]: import pandas as pd
import numpy as np
import re
import string
from collections import Counter
```

```
[ ]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[ ]: df_ak = pd.read_excel('/content/drive/MyDrive/komentar_AK.xlsx')
df_al = pd.read_excel('/content/drive/MyDrive/komentar_AL.xlsx')

for df in [df_ak, df_al]:
    drop_cols = [c for c in df.columns if c in ('level_0', 'index', 'Unnamed: 0')]
    df.drop(columns=drop_cols, inplace=True)
    df.reset_index(drop=True, inplace=True)

print(f'komentar_AK rows : {len(df_ak)}')
print(f'komentar_AL rows : {len(df_al)}')
print(f'\ndf_ak columns : {list(df_ak.columns)}')
print(f'df_al columns : {list(df_al.columns)}')
```

komentar_AK rows : 576

komentar_AL rows : 281

df_ak columns : ['nomor_usulan', 'komen_AK', 'no_asesor_AK']

df_al columns : ['nomor_usulan', 'komen_AL', 'kode_asesor_AL', 'tipe_asesor_AL']

```
[ ]: def preprocess_text(text):
    if pd.isna(text):
        return np.nan
    text = str(text)
    text = text.lower()
    text = re.sub(r'https?://\S+|www\.\S+', '', text) # 1. lowercase
    text = re.sub(r'\s+', ' ', text).strip() # 2. remove URLs
    text = re.sub(r'\s+', ' ', text).strip() # 3. normalise spaces
    return text
```

```
[ ]: df_ak['komentar_AK_lower'] = df_ak['komen_AK'].apply(preprocess_text)
df_al['komentar_AL_lower'] = df_al['komen_AL'].apply(preprocess_text)

print('Preprocessing done.')
display(df_ak[['komen_AK', 'komentar_AK_lower']].head(3))
display(df_al[['komen_AL', 'komentar_AL_lower']].head(3))
```

Preprocessing done.

```
                                komen_AK  \
0  - Ketersediaan dokumen terkait: * CP: tersedi...
1  Kelulusan Tepat Waktu: berdasarkan table 9.2 d...
2  1) Pemenuhan CPL untuk mata kuliah aspek sikap...
```

```
                                komentar_AK_lower
0  - ketersediaan dokumen terkait: * cp: tersedia...
1  kelulusan tepat waktu: berdasarkan table 9.2 d...
2  1) pemenuhan cpl untuk mata kuliah aspek sikap...
```

```
                                komen_AL  \
0  Keterlaksanaan kebijakan, standar, IKU, dan IK...
1  Pemenuhan luaran pendidikan mencakup: 1) Pemen...
2  Pemenuhan Capaian Pembelajaran Lulusan yang te...
```

```
                                komentar_AL_lower
0  keterlaksanaan kebijakan, standar, iku, dan ik...
1  pemenuhan luaran pendidikan mencakup: 1) pemen...
2  pemenuhan capaian pembelajaran lulusan yang te...
```

```
[ ]: df_ak.to_excel('/content/drive/MyDrive/komentar_AK_lower.xlsx', index=False)
df_al.to_excel('/content/drive/MyDrive/komentar_AL_lower.xlsx', index=False)
```

2.1 Structural Pattern Matches

```
[ ]: STRUCTURAL_PATTERNS = [
    ('universitas / universiti + next word',
     r'\b(?:universitas|universiti)\s+\w+'),
    ('program studi + next word',
     r'\bprogram\s+studi\s+\w+'),
    ('prodi + next word',
     r'\bprodi\s+\w+'),
    ('jurusan + next word',
     r'\bjurusan\s+\w+'),
    ('fakultas + next word',
     r'\bfakultas\s+\w+'),
    ('perguruan tinggi + next word',
     r'\bperguruan\s+tinggi\s+\w+'),
    ('institut / politeknik / akademi + next word',
     r'\b(?:institut|politeknik|poltek|akademi)\s+\w+'),
```

```

('sekolah tinggi + next word',
 r'\bsekolah\s+tinggi\s+\w+'),
('word + university',
 r'\b\w+\s+university\b'),
('himpunan + next word',
 r'\bhimpunan\s+\w+'),
('bem + next word',
 r'\bbem\s+\w+'),
]

def context_preview(text, match, window=6):
    words = text.split()
    pos = text.find(match)
    if pos == -1:
        return match
    word_idx = len(text[:pos].split())
    match_span = len(match.split())
    end = min(len(words), word_idx + match_span + window)
    snippet = ' '.join(words[word_idx:end])
    suffix = ' ...' if end < len(words) else ''
    return f'{snippet}{suffix}'

def scan_structural(df, text_col):
    rows = []
    for pos, (_, row) in enumerate(df.iterrows()):
        text = row[text_col]
        if pd.isna(text):
            continue
        for label, pattern in STRUCTURAL_PATTERNS:
            hits = re.findall(pattern, text)
            for hit in hits:
                rows.append({
                    'excel_row': pos + 2,
                    'pattern': label,
                    'matched': hit,
                    'context': context_preview(text, hit),
                })
    return pd.DataFrame(rows) if rows else pd.DataFrame(
        columns=['excel_row', 'pattern', 'matched', 'context'])

```

```

[ ]: hits_ak = scan_structural(df_ak, 'komentar_AK_lower')
print(f'AK - {len(hits_ak)} structural hit(s) across '
      f'{hits_ak["excel_row"].nunique() if len(hits_ak) else 0} row(s)')
display(hits_ak)

```

```
[ ]: hits_al = scan_structural(df_al, 'komentar_AL_lower')
print(f'AL - {len(hits_al)} structural hit(s) across '
      f'{hits_al["excel_row"].nunique() if len(hits_al) else 0} row(s)')
display(hits_al)
```

2.2 Words Starting with u: Frequency Table

```
[ ]: def u_word_freq(df, text_col):
    counter = Counter()
    row_map = {} # word -> list of excel_rows
    for idx, text in df[text_col].dropna().items():
        seen_in_row = set()
        for raw_word in text.split():
            word = raw_word.strip(string.punctuation)
            if len(word) >= 2 and word.startswith('u'):
                counter[word] += 1
                if word not in seen_in_row:
                    row_map.setdefault(word, []).append(idx + 2) # Excel row
                    seen_in_row.add(word)
    result = pd.DataFrame(counter.most_common(), columns=['word', 'count'])
    result['excel_rows'] = result.apply(
        lambda r: row_map.get(r['word'], []) if r['count'] < 20 else '',
        axis=1
    )
    return result
```

```
[ ]: freq_ak = u_word_freq(df_ak, 'komentar_AK_lower')
print(f'AK - {len(freq_ak)} unique u-words')
display(freq_ak)
```

AK - 88 unique u-words

	word	count	excel_rows
0	untuk	560	
1	umum	58	
2	upps	55	
3	universitas	50	
4	uraian	33	
..	...	...	...
83	universiti	1	[485]
84	unusia	1	[505]
85	upp	1	[507]
86	upi-yptkpadang	1	[564]
87	umumnya	1	[567]

[88 rows x 3 columns]

```
[ ]: freq_al = u_word_freq(df_al, 'komentar_AL_lower')
print(f'AL - {len(freq_al)} unique u-words')
display(freq_al)
```

AL - 74 unique u-words

	word	count	excel_rows
0	untuk	269	
1	umum	39	
2	universitas	38	
3	upps	20	
4	usaha	11	[62, 83, 124, 142, 209]
..	...	...	...
69	ub	1	[237]
70	universiti	1	[241]
71	unhan	1	[256]
72	unisyss	1	[263]
73	unm	1	[273]

[74 rows x 3 columns]

3 Tahap Konstruksi Tesaurus - ACD (AK)

Tahap ini menjalankan ekstraksi sinonim, singkatan, dan padanan istilah untuk setiap aspek (*Aspect Category Detection* / ACD), menggunakan model LLM (gemma4:latest) yang dijalankan melalui Ollama dengan pendekatan *Subject-Predicate-Object* (SPO). Seluruh delapan aspek diproses dalam satu panggilan `ollama.chat()` per baris komentar. Hasil ekstraksi disimpan sebagai kandidat tesaurus domain yang akan melalui tahap pembersihan dan validasi pada tahap berikutnya.

Versi ini menggunakan data AK (Asesmen Kecukupan). Untuk memproses data AL (Asesmen Lapangan), ubah `FILE_NAME`, `TEXT_COL`, `TIPE_KOMENTAR`, dan `DATASET_NAME` pada cell konfigurasi di bawah; baris yang perlu diubah ditandai dengan komentar `# ← ganti ke AL di sini`.

```
[ ]: from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call `drive.mount("/content/drive", force_remount=True)`.

3.1 Install Ollama + Start Server + Pull Model

```
[ ]: import subprocess, threading, time

# 1. System deps + Ollama install
!sudo apt-get install -y zstd pciutils -q
!curl -fsSL https://ollama.com/install.sh | sh

# 2. Start server in background thread
def run_ollama_serve():
    subprocess.Popen(["ollama", "serve"])

thread = threading.Thread(target=run_ollama_serve)
thread.start()
time.sleep(5)
print("Ollama server started.")

# 3. Pull model - change MODEL_NAME below to switch models
MODEL_NAME = "gemma4:latest" # ← change here if needed
!ollama pull {MODEL_NAME}
print(f" {MODEL_NAME} ready.")
```

Reading package lists...

Building dependency tree...

Reading state information...

pciutils is already the newest version (1:3.7.0-6).

zstd is already the newest version (1.4.8+dfsg-3build1).

0 upgraded, 0 newly installed, 0 to remove and 53 not upgraded.

>>> Cleaning up old version at /usr/local/lib/ollama

>>> Installing ollama to /usr/local

```
>>> Downloading ollama-linux-amd64.tar.zst
##### 100.0%
>>> Adding ollama user to video group...
>>> Adding current user to ollama group...
>>> Creating ollama systemd service...
WARNING: systemd is not running
>>> NVIDIA GPU installed.
>>> The Ollama API is now available at 127.0.0.1:11434.
>>> Install complete. Run "ollama" from the command line.
Ollama server started.
  gemma4:latest ready.
```

```
[ ]: import sys, subprocess

PACKAGES = {
    "ollama": "ollama",
    "pandas": "pandas",
    "openpyxl": "openpyxl",
    "tqdm": "tqdm",
}

for imp, pkg in PACKAGES.items():
    try:
        __import__(imp)
    except ImportError:
        subprocess.check_call([sys.executable, "-m", "pip", "install", pkg,
                                ↪"-q"])

print("All packages OK.")
```

All packages OK.

```
[ ]: import os, re
import pandas as pd
import ollama
from datetime import datetime
from tqdm import tqdm

print("All imports OK.")
```

All imports OK.

```
[ ]: BASE_PATH      = "/content/drive/MyDrive"

FILE_NAME          = "komentar_AK_lower.xlsx"    # ← ganti ke AL di sini:
    ↪ "komentar_AL_lower.xlsx"
TEXT_COL           = "komentar_AK_lower"         # ← ganti ke AL di sini:
    ↪ "komentar_AL_lower"
```

```

TIPE_KOMENTAR = "AK"                                # ← ganti ke AL di sini: "AL"
MODEL_NAME     = "gemma4:latest"
DATASET_NAME   = "ACD_AK"                            # ← ganti ke AL di sini: "ACD_AL"

MODEL_FOLDER = MODEL_NAME.replace(":", "-").replace("/", "-")
OUTPUT_DIR   = os.path.join(BASE_PATH, "Thesaurus", "ACD_AK", MODEL_FOLDER) #
    ← ganti "ACD_AK" ke "ACD_AL" di sini juga
LOGS_DIR      = os.path.join(OUTPUT_DIR, "logs")
OUTPUT_EXCEL  = os.path.join(OUTPUT_DIR, "HASIL_ACD_AK_synonym.xlsx")      #
    ← ganti "AK" ke "AL" di sini juga

os.makedirs(LOGS_DIR, exist_ok=True)

ROW_START = 1 # ← e.g. 1 → start from row 1 of the Excel
ROW_END   = 576

print(f"Base path : {BASE_PATH}")
print(f"Source file: {FILE_NAME}")
print(f"Output dir : {OUTPUT_DIR}")
print(f"Excel out  : {OUTPUT_EXCEL}")
print(f"Logs dir   : {LOGS_DIR}")

```

```

[ ]: ASPEK_LIST = [
    "capaian pembelajaran lulusan",
    "indeks prestasi kumulatif",
    "kelulusan tepat waktu",
    "masa tunggu lulusan",
    "pelacakan dan perekaman data lulusan",
    "prestasi mahasiswa",
    "karya dengan hak kekayaan intelektual",
    "kesesuaian bidang kerja",
]

print(f"{len(ASPEK_LIST)} aspects configured.")

```

8 aspects configured.

```

[ ]: import sys

try:
    models = ollama.list()
    available = [m.model for m in models.models]
    print(f"Available models: {available}")
    if not any(MODEL_NAME in m for m in available):
        print(f"\n[ERROR] Model '{MODEL_NAME}' not found.")
        print(f"Run: !ollama pull {MODEL_NAME}")
        raise RuntimeError(f"Model not found: {MODEL_NAME}")

```

```

    print(f"Model '{MODEL_NAME}' found. OK.")
except Exception as e:
    print(f"[ERROR] Cannot connect to Ollama: {e}")
    print("  Make sure Cell 1 (Ollama install) was run successfully.")
    raise

```

Available models: ['gemma4:latest']
 Model 'gemma4:latest' found. OK.

```

[ ]: file_path = os.path.join(BASE_PATH, FILE_NAME)
    df         = pd.read_excel(file_path)

    if "nomor_usulan" not in df.columns:
        df["nomor_usulan"] = df.index.astype(str)
    df["nomor_usulan"] = df["nomor_usulan"].astype(str).str.strip()
    total_rows = len(df)

    if ROW_START is not None or ROW_END is not None:
        _s = (ROW_START - 1) if ROW_START is not None else 0
        _e = ROW_END         if ROW_END   is not None else total_rows
        df = df.iloc[_s:_e].copy()
        print(f"Row window applied : rows {ROW_START or 1} - {ROW_END or 1}
        ↳total_rows} "
              f"({len(df)} rows in window)")

    all_results      = []
    rows_to_process = df
    remaining        = len(rows_to_process)

    print(f"Dataset           : {FILE_NAME}")
    print(f"Column              : {TEXT_COL}")
    print(f"Total rows in file : {total_rows}")
    print(f"Rows to process    : {remaining}")

    if remaining == 0:
        print("\nNo rows to process.")

```

3.2 System Prompt

```

[ ]: SPO_SYSTEM = """## Task Role
Anda adalah seorang linguist yang berspesialisasi dalam leksikologi
dan identifikasi relasi ekuivalensi antar istilah, termasuk variasi
terminologi, singkatan, dan padanan yang muncul dalam laporan asesor
akreditasi pendidikan tinggi.

## Task Context

```

Anda membantu menyusun thesaurus domain-spesifik untuk mendukung Aspect-Based,
→Sentiment Analysis (ABSA) pada komentar penilaian akreditasi pendidikan,
→tinggi.

Dataset berisi komentar asesor yang tersimpan pada kolom komentar_AK.

Komentar dapat mengandung:

- variasi istilah
- singkatan
- padanan istilah aspek

Task Objective

Bacalah komentar berikut dan identifikasi semua sinonim, padanan istilah,
→singkatan, variasi penyebutan, serta rujukan eksplisit maupun implisit yang
→mengacu pada istilah aspek yang diberikan.

Instruction Rules

1. Gunakan struktur SPO (Subject\u2013Predicate\u2013Object) untuk
→mengidentifikasi relasi sinonim.

2. Setiap entri HARUS mengikuti format:

Istilah Aspek | Frasa Kunci Penanda Relasi | Sinonim/Padanan

HANYA 1 Hasil Sinonim per row.

3. Hanya lakukan ekstraksi berdasarkan komentar yang diberikan.

4. Hanya identifikasi sinonim/padanan yang:
- muncul secara eksplisit/implicit pada komentar

5. Frasa kunci penanda relasi HANYA boleh:

- disebut sebagai
- disingkat sebagai
- dikenal sebagai
- merujuk pada
- mengacu pada
- merupakan istilah lain dari

6. Jangan membuat istilah baru yang tidak muncul pada komentar.

8. Jika istilah aspek pada input sudah berupa nama lengkap, pertahankan nama,
→lengkap tersebut pada output. Jangan ubah menjadi singkatan kecuali,
→singkatan itu muncul eksplisit di komentar sebagai hubungan singkatan.

9. Sertakan komentar asli sebagai bukti ekstraksi.

```

10. Pertahankan metadata:
- tipe komentar
- nomor_usulan

untuk menjaga traceability.
"""

print("SPO system prompt loaded.")
print(f"Length: {len(SPO_SYSTEM)} chars")

```

SPO system prompt loaded.
Length: 1775 chars

```

[ ]: def build_messages_all(aspek_list, komentar, tipe_komentar, nomor_usulan):
    aspek_lines = "\n".join(f"- {a}" for a in aspek_list)

    user_content = f"""{komentar}

Tipe Komentar: {tipe_komentar}
Nomor Usulan: {nomor_usulan}

Aspek:
{aspek_lines}"""

    return [
        {"role": "system", "content": SPO_SYSTEM},
        {"role": "user", "content": user_content},
    ]

print("Prompt builder ready.")

```

Prompt builder ready.

```

[ ]: import threading

# Output cleaner
def clean_raw_output(raw: str) -> str:
    cleaned = re.sub(r"<think>.*?</think>", "", raw, flags=re.DOTALL)
    cleaned = re.sub(r"```[a-zA-Z]*", "", cleaned).replace("`", "")
    return cleaned.strip()

# Table parser
def parse_table_all(clean_text, aspek_list, tipe_komentar, nomor_usulan):
    rows = []
    seen_keys = set()

```

```

for line in clean_text.splitlines():
    if "|" not in line:
        continue
    parts = [p.strip() for p in line.split("|")]
    parts = [p for p in parts if p]
    if len(parts) < 3:
        continue
    if all(re.fullmatch(r"[-:= ]+", p) for p in parts):
        continue
    if any(h in parts[0].lower() for h in ("istilah", "frasa", "aspek")):
        continue
    key = (parts[0].lower(), parts[1].lower(), parts[2].lower())
    if key in seen_keys:
        continue
    seen_keys.add(key)
    rows.append({
        "Istilah Aspek": parts[0],
        "Frasa Kunci Penanda Relasi": parts[1],
        "Sinonim/Padanan": parts[2],
        "Tipe Komentar": tipe_komentar,
        "Nomor Usulan": nomor_usulan,
    })

# Fallback: ensure every aspect has at least one row
found = {r["Istilah Aspek"].lower() for r in rows}
for aspek in aspek_list:
    if aspek.lower() not in found:
        rows.append({
            "Istilah Aspek": aspek,
            "Frasa Kunci Penanda Relasi": "Tidak ditemukan",
            "Sinonim/Padanan": "Tidak ditemukan",
            "Tipe Komentar": tipe_komentar,
            "Nomor Usulan": nomor_usulan,
        })
return rows

# Inference runner
def run_inference(messages):
    _done = threading.Event()
    _result = {}

    def _worker():
        _result["response"] = ollama.chat(
            model=MODEL_NAME,
            messages=messages,
            options={

```

```

        "temperature": 0.0,
        "num_ctx": 8192,
        "num_predict": 4096,
        "num_keep": 0,
    }
)
_done.set()

thread = threading.Thread(target=_worker, daemon=True)
thread.start()

with tqdm(
    desc=" Inferencing",
    unit="s",
    ncols=72,
    bar_format="{desc}: {elapsed} elapsed [{postfix}]",
    dynamic_ncols=False,
    leave=False,
    position=1,
) as ibar:
    while not _done.is_set():
        time.sleep(1)
        ibar.update(1)
        ibar.set_postfix_str("waiting for Ollama...")
        ibar.set_postfix_str("done [OK]")

thread.join()
response = _result["response"]
raw = response.message.content
thinking_text = getattr(response.message, "thinking", "") or ""
return raw, thinking_text

# Log saver
def save_row_log(nomor_usulan, idx, komentar, raw, thinking_text, cleaned,
    ↪ parsed_rows):
    log_path = os.path.join(LOGS_DIR, f"{nomor_usulan}_row{idx}.txt")
    with open(log_path, "w", encoding="utf-8") as f:
        f.write(f"{'=' * 70}\n")
        f.write(f"NOMOR USULAN : {nomor_usulan}\n")
        f.write(f"DATASET : {DATASET_NAME} | COLUMN: {TEXT_COL}\n")
        f.write(f"TIPE KOMENTAR : {TIPE_KOMENTAR}\n")
        f.write(f"TIMESTAMP : {datetime.now().strftime('%Y-%m-%d %H:%M:
    ↪ %S')}\n")
        f.write(f"{'=' * 70}\n\n")
        f.write(f"[KOMENTAR ASLI]\n")
        f.write(komentar + "\n\n")

```

```

f.write("[THINKING - reasoning block]\n")
f.write((thinking_text or "(empty)") + "\n\n")
f.write("[RAW OUTPUT]\n")
f.write((raw or "(empty)") + "\n\n")
f.write("[CLEANED OUTPUT]\n")
f.write((cleaned or "(empty)") + "\n\n")
f.write("[REPR - exact bytes into parser]\n")
f.write(repr(cleaned) + "\n\n")
f.write(f"[PARSED ROWS: {len(parsed_rows)}]\n")
for r in parsed_rows:
    f.write(
        f"  Aspek   : {r['Istilah Aspek']}\n"
        f"  Frasa   : {r['Frasa Kunci Penanda Relasi']}\n"
        f"  Sinonim: {r['Sinonim/Padanan']}\n\n"
    )

# Excel saver
def save_excel(results):
    pd.DataFrame(results).to_excel(OUTPUT_EXCEL, index=False)

print("All helper functions ready.")

```

All helper functions ready.

3.3 Run LLM

```

[ ]: if remaining == 0:
    print("No rows to process. Check ROW_START / ROW_END in Cell 4.")
else:
    print(f"Starting - {remaining} row(s) to process")
    print(f"  Logs   → {LOGS_DIR}")
    print(f"  Excel  → {OUTPUT_EXCEL}")
    print()

    last_processed = None

    try:
        with tqdm(
            total=remaining,
            desc="  Rows",
            unit="row",
            ncols=80,
            position=0,
        ) as pbar:

            for idx, row in rows_to_process.iterrows():

```

```

        nomor_usulan = str(row.get("nomor_usulan", idx))
        komentar      = str(row.get(TEXT_COL, "")).strip()

        # Empty / NaN row
        if not komentar or komentar.lower() == "nan":
            skipped = [{
                "Istilah Aspek":          aspek,
                "Frasa Kunci Penanda Relasi": "SKIPPED",
                "Sinonim/Padanan":        "komentar kosong",
                "Tipe Komentar":          TIPE_KOMENTAR,
                "Nomor Usulan":           nomor_usulan,
                "Dataset":                DATASET_NAME,
            } for aspek in ASPEK_LIST]
            all_results.extend(skipped)
            save_excel(all_results)
            with open(os.path.join(LOGS_DIR, f"{nomor_usulan}_row{idx}.
↳txt"), "w", encoding="utf-8") as f:
                f.write(f"NOMOR USULAN: {nomor_usulan}\nSTATUS: SKIPPED -
↳komentar kosong\n")
                pbar.set_postfix_str(f"{nomor_usulan} SKIPPED")
                pbar.update(1)
                continue

        # Build prompt + run inference
        pbar.set_postfix_str(f"{nomor_usulan} inferencing...")
        messages = build_messages_all(ASPEK_LIST, komentar, TIPE_KOMENTAR,
↳nomor_usulan)

        try:
            raw, thinking_text = run_inference(messages)
        except Exception as e:
            tqdm.write(f" [ERROR] {nomor_usulan}: {e}")
            error_rows = [{
                "Istilah Aspek":          aspek,
                "Frasa Kunci Penanda Relasi": "ERROR",
                "Sinonim/Padanan":        str(e),
                "Tipe Komentar":          TIPE_KOMENTAR,
                "Nomor Usulan":           nomor_usulan,
                "Dataset":                DATASET_NAME,
            } for aspek in ASPEK_LIST]
            all_results.extend(error_rows)
            save_excel(all_results)
            with open(os.path.join(LOGS_DIR, f"{nomor_usulan}_row{idx}.
↳txt"), "w", encoding="utf-8") as f:
                f.write(f"NOMOR USULAN: {nomor_usulan}\nSTATUS:
↳ERROR\n{e}\n")
                pbar.update(1)

```

```

        continue

    # Guard: model returned nothing
    if not raw or not raw.strip():
        tqdm.write(f" [WARN] {nomor_usulan}: model returned empty_
↳output")

        if thinking_text:
            tqdm.write(
                f"         Thinking was {len(thinking_text)} chars - "
                f"raise num_predict if this keeps happening"
            )

    # Parse
    cleaned      = clean_raw_output(raw or "")
    parsed_rows = parse_table_all(cleaned, ASPEK_LIST, TIPE_KOMENTAR,
↳nomor_usulan)
    for r in parsed_rows:
        r["Dataset"] = DATASET_NAME

    all_results.extend(parsed_rows)

    # Save log + Excel immediately after each row
    save_row_log(nomor_usulan, idx, komentar, raw or "", thinking_text,
↳cleaned, parsed_rows)
    save_excel(all_results)

    last_processed = nomor_usulan
    pbar.set_postfix_str(f"{nomor_usulan} [OK] {len(parsed_rows)} rows")
    pbar.update(1)

except KeyboardInterrupt:
    print(f"\n[WARNING] Processing interrupted!")
    print(f"    Last row completed : {last_processed or 'none (no rows_
↳finished)'}")
    print(f"    Resume: re-run Cell 7 → Cell 11; already-saved rows are_
↳auto-skipped.")
    finally:
        if last_processed:
            print(f"\n [OK] Last successfully saved row : {last_processed}")

print()
print("=" * 80)
print("DONE!")
print(f"    Total result rows : {len(all_results)}")
print(f"    Excel              : {OUTPUT_EXCEL}")
print(f"    Logs folder         : {LOGS_DIR}")
print("=" * 80)

```

4 Tahap Konstruksi Leksikon Sentimen - ASC (AK)

Tahap ini menjalankan ekstraksi ekspresi sentimen untuk setiap aspek (*Aspect Sentiment Classification* / ASC), menggunakan model LLM (gemma4:latest) yang dijalankan melalui Ollama dengan pendekatan *Subject-Predicate-Object* (SPO). Untuk setiap aspek, model menentukan kelas sentimen, mengutip ekspresi indikator dari komentar sebagai bukti, dan menghasilkan bentuk generalisasi dari ekspresi tersebut agar dapat digunakan kembali pada komentar lain. Seluruh delapan aspek diproses dalam satu panggilan `ollama.chat()` per baris komentar. Hasil ekstraksi disimpan sebagai kandidat leksikon sentimen domain yang akan melalui tahap pembersihan dan validasi pada tahap berikutnya.

Versi ini menggunakan data AK (Asesmen Kecukupan). Untuk memproses data AL (Asesmen Lapangan), ubah `FILE_NAME`, `TEXT_COL`, `TIPE_KOMENTAR`, dan `DATASET_NAME` pada cell konfigurasi di bawah; baris yang perlu diubah ditandai dengan komentar `# ← ganti ke AL di sini`.

```
[ ]: from google.colab import drive
drive.mount('/content/drive')
```

4.1 Install Ollama + Start Server + Pull Model

```
[ ]: import subprocess, threading, time

# 1. System deps + Ollama install
!sudo apt-get install -y zstd pciutils -q
!curl -fsSL https://ollama.com/install.sh | sh

# 2. Start server in background thread
def run_ollama_serve():
    subprocess.Popen(["ollama", "serve"])

thread = threading.Thread(target=run_ollama_serve)
thread.start()
time.sleep(5)
print("Ollama server started.")

# 3. Pull model - change MODEL_NAME below to switch models
MODEL_NAME = "gemma4:latest" # ← change here if needed
!ollama pull {MODEL_NAME}
print(f" {MODEL_NAME} ready.")
```

```
[ ]: import sys, subprocess

PACKAGES = {
    "ollama": "ollama",
    "pandas": "pandas",
    "openpyxl": "openpyxl",
    "tqdm": "tqdm",
```

```

}

for imp, pkg in PACKAGES.items():
    try:
        __import__(imp)
    except ImportError:
        subprocess.check_call([sys.executable, "-m", "pip", "install", pkg,
↪ "-q"])

print("All packages OK.")

```

```

[ ]: import os, re
import pandas as pd
import ollama
from datetime import datetime
from tqdm import tqdm

print("All imports OK.")

```

```

[ ]: BASE_PATH      = "/content/drive/MyDrive"

FILE_NAME          = "komentar_AK_lower.xlsx"    # ← ganti ke AL di sini:
↪ "komentar_AL_lower.xlsx"
TEXT_COL           = "komentar_AK_lower"         # ← ganti ke AL di sini:
↪ "komentar_AL_lower"
TIPE_KOMENTAR      = "AK"                        # ← ganti ke AL di sini: "AL"
MODEL_NAME         = "gemma4:latest"             # must match what was pulled in Cell 1
DATASET_NAME       = "ASC_AK"                    # ← ganti ke AL di sini: "ASC_AL"

# Output organised by dataset and model name
MODEL_FOLDER = MODEL_NAME.replace(":", "-").replace("/", "-")
OUTPUT_DIR    = os.path.join(BASE_PATH, "Thesaurus", "ASC_AK", MODEL_FOLDER) #
↪ ← ganti "ASC_AK" ke "ASC_AL" di sini juga
LOGS_DIR      = os.path.join(OUTPUT_DIR, "logs")
OUTPUT_EXCEL  = os.path.join(OUTPUT_DIR, "HASIL_ASC_AK_sentiment.xlsx")    #
↪ ← ganti "AK" ke "AL" di sini juga

os.makedirs(LOGS_DIR, exist_ok=True)

ROW_START = 1 # ← e.g. 1 → start from row 1 of the Excel
ROW_END   = 576

print(f"Base path : {BASE_PATH}")
print(f"Source file: {FILE_NAME}")
print(f"Output dir : {OUTPUT_DIR}")
print(f"Excel out  : {OUTPUT_EXCEL}")
print(f"Logs dir   : {LOGS_DIR}")

```

```
[ ]: ASPEK_LIST = [
    "capaian pembelajaran lulusan",
    "indeks prestasi kumulatif",
    "kelulusan tepat waktu",
    "masa tunggu lulusan",
    "pelacakan dan perekaman data lulusan",
    "prestasi mahasiswa",
    "karya dengan hak kekayaan intelektual",
    "kesesuaian bidang kerja",
]

print(f"{len(ASPEK_LIST)} aspects configured.")
```

```
[ ]: import sys

try:
    models = ollama.list()
    available = [m.model for m in models.models]
    print(f"Available models: {available}")
    if not any(MODEL_NAME in m for m in available):
        print(f"\n[ERROR] Model '{MODEL_NAME}' not found.")
        print(f"Run: !ollama pull {MODEL_NAME}")
        raise RuntimeError(f"Model not found: {MODEL_NAME}")
    print(f"Model '{MODEL_NAME}' found. OK.")
except Exception as e:
    print(f"[ERROR] Cannot connect to Ollama: {e}")
    print("Make sure Cell 1 (Ollama install) was run successfully.")
    raise
```

```
[ ]: file_path = os.path.join(BASE_PATH, FILE_NAME)
df = pd.read_excel(file_path)

if "nomor_usulan" not in df.columns:
    df["nomor_usulan"] = df.index.astype(str)
df["nomor_usulan"] = df["nomor_usulan"].astype(str).str.strip()
total_rows = len(df)

if ROW_START is not None or ROW_END is not None:
    _s = (ROW_START - 1) if ROW_START is not None else 0
    _e = ROW_END if ROW_END is not None else total_rows
    df = df.iloc[_s:_e].copy()
    print(f"Row window applied : rows {ROW_START or 1} - {ROW_END or total_rows} "
          f"({len(df)} rows in window)")

# Resume: load existing Excel results if present
if os.path.exists(OUTPUT_EXCEL):
```

```

existing_df = pd.read_excel(OUTPUT_EXCEL)
all_results = existing_df.to_dict("records")
print(f"Resumed          : {len(all_results)} rows loaded from existing_
↳Excel")
else:
    all_results = []
    print("No existing Excel found - starting fresh.")

rows_to_process = df
remaining        = len(rows_to_process)

print(f"Dataset          : {FILE_NAME}")
print(f"Column           : {TEXT_COL}")
print(f"Total rows in file : {total_rows}")
print(f"Rows to process    : {remaining}")

if remaining == 0:
    print("\nNo rows to process.")

```

4.2 System Prompt

```
[ ]: SPO_SYSTEM = """## Task Role
Anda adalah seorang linguis yang berspesialisasi dalam analisis sentimen
dan identifikasi ekspresi evaluatif dalam laporan asesor akreditasi
pendidikan tinggi.

## Task Context
Anda membantu membangun leksikon sentimen domain-spesifik dari komentar
asesor akreditasi pendidikan tinggi untuk Aspect-Based Sentiment Analysis_
↳(ABSA).
Tugas Anda adalah ekstraksi penanda linguistik, bukan melakukan ABSA itu_
↳sendiri.

## Task Objective
Untuk setiap aspek yang diberikan, bacalah komentar dan tentukan:
1. Kelas sentimen aspek tersebut (positif / negatif / netral / tidak relevan).
2. Ekspresi pendek yang dikutip langsung dari komentar sebagai bukti sentimen.
3. Generalisasi linguistik dari ekspresi tersebut - tanpa angka, nama aspek,
    atau data spesifik - yang dapat berlaku pada komentar lain dengan konteks_
↳berbeda.

Anda HARUS menentukan sentimen sendiri berdasarkan isi komentar.
Tidak ada label yang diberikan - klasifikasi adalah bagian dari tugas Anda.

## Output Format

WAJIB: Setiap baris memiliki TEPAT 5 kolom dipisahkan TEPAT 4 tanda |.
```

Tidak ada header. Tidak ada teks lain. Tidak ada penjelasan. Hanya baris data.

Istilah Aspek | Kelas Sentimen | Frasa Kunci Penanda Sentimen | Ekspresi
↳ Indikator Sentimen | Ekspresi Generalisasi

Aturan Per Kolom

Kolom 1 - Istilah Aspek

Salin nama aspek persis seperti diberikan. Satu baris per ekspresi evaluatif
↳ yang ditemukan.

Kolom 2 - Kelas Sentimen

Pilih TEPAT SATU. Tidak boleh kosong. Tidak boleh nilai lain:

positif → aspek diekspresikan secara baik dalam komentar
negatif → aspek dikritik atau dinyatakan kurang dalam komentar
netral → aspek disebut tanpa muatan evaluatif yang jelas
tidak relevan → aspek tidak disebut sama sekali dalam komentar

Kolom 3 - Frasa Kunci Penanda Sentimen

Pilih TEPAT SATU dari empat nilai berikut. Tidak boleh diubah, disingkat, atau
↳ diparafrase:

dinilai positif melalui frasa
dinilai negatif melalui frasa
diidentifikasi sebagai penanda netral melalui frasa
Tidak Ditemukan

Panduan pemilihan:

→ Kelas Sentimen = positif : gunakan "dinilai positif melalui frasa"
→ Kelas Sentimen = negatif : gunakan "dinilai negatif melalui frasa"
→ Kelas Sentimen = netral : gunakan "diidentifikasi sebagai penanda
↳ netral melalui frasa"
→ Kelas Sentimen = tidak relevan : gunakan "Tidak Ditemukan"

Kolom 4 - Ekspresi Indikator Sentimen

Kutipan PENDEK (1-7 kata) yang diambil LANGSUNG dari komentar sebagai bukti
↳ sentimen.

→ Jika Kelas Sentimen = tidak relevan : isi "Tidak Ditemukan"
→ Jika tidak ada bukti yang dapat dikutip : isi "Tidak Ditemukan"
→ DILARANG membuat kutipan yang tidak muncul di komentar.

Kolom 5 - Ekspresi Generalisasi

Ikuti PERSIS tiga langkah ini secara berurutan. Jangan lakukan hal lain.

LANGKAH 1: Jika Kolom 4 = "Tidak Ditemukan" → tulis "Tidak Ditemukan".
↳ SELESAI.

LANGKAH 2: Periksa apakah Kolom 4 mengandung setidaknya satu dari:
 angka, digit, persentase (%), nama institusi, nama orang.
 → TIDAK mengandung satupun → salin Kolom 4 PERSIS kata per kata. □
 → SELESAI.

→ MENGANDUNG salah satu → lanjut ke Langkah 3.

LANGKAH 3: Hapus semua angka, digit, persentase, nama institusi, nama orang.
 Ungkapkan sisa makna evaluatifnya dalam 2-5 kata TANPA menyebut
 angka atau data spesifik apapun.

Contoh: "rata2 ipk > 3.35" → "nilai akademik tinggi"

Contoh: "prosentase lulus 100%" → "kelulusan sangat tinggi"

Contoh: "kurang dari 4 bulan" → "waktu tunggu singkat"

Pertahankan metadata (tipe komentar dan nomor_usulan) pada setiap baris.
 ""

```
print("SPO system prompt loaded.")
print(f"Length: {len(SPO_SYSTEM)} chars")
```

```
[ ]: def build_messages_all(aspek_list, komentar, tipe_komentar, nomor_usulan):
    aspek_lines = "\n".join(f"- {a}" for a in aspek_list)

    user_content = f"""{komentar}

Tipe Komentar: {tipe_komentar}
Nomor Usulan: {nomor_usulan}

Aspek:
{aspek_lines}"""

    return [
        {"role": "system", "content": SPO_SYSTEM},
        {"role": "user", "content": user_content},
    ]

print("Prompt builder ready.")
```

```
[ ]: import threading

def clean_raw_output(raw: str) -> str:
    cleaned = re.sub(r"<think>.*?</think>", "", raw, flags=re.DOTALL)
    cleaned = re.sub(r"```[a-zA-Z]*", "", cleaned).replace("`", "")
    return cleaned.strip()

def parse_table_all(clean_text, aspek_list, tipe_komentar, nomor_usulan):
    rows = []
```

```

seen = set()

for line in clean_text.splitlines():
    if "|" not in line:
        continue
    parts = [p.strip() for p in line.strip().strip("|").split("|")]
    # Need at least aspek + sentimen
    if len(parts) < 2:
        continue
    # Skip separator and header lines
    if all(re.fullmatch(r"[:- ]+", p) for p in parts):
        continue
    if parts[0].lower() in ("istilah aspek", "istilah", "aspek"):
        continue
    # Pad missing columns rather than dropping the row
    while len(parts) < 5:
        parts.append("Tidak Ditemukan")
    key = tuple(p.lower() for p in parts[:5])
    if key in seen:
        continue
    seen.add(key)
    rows.append({
        "Istilah Aspek": parts[0],
        "Kelas Sentimen": parts[1],
        "Frasa Kunci Penanda Sentimen": parts[2],
        "Ekspresi Indikator Sentimen": parts[3],
        "Ekspresi Generalisasi": parts[4],
        "Tipe Komentar": tipe_komentar,
        "Nomor Usulan": nomor_usulan,
    })

# Ensure every aspect has at least one row
found = {r["Istilah Aspek"].lower() for r in rows}
for aspek in aspek_list:
    if aspek.lower() not in found:
        rows.append({
            "Istilah Aspek": aspek,
            "Kelas Sentimen": "tidak relevan",
            "Frasa Kunci Penanda Sentimen": "Tidak Ditemukan",
            "Ekspresi Indikator Sentimen": "Tidak Ditemukan",
            "Ekspresi Generalisasi": "Tidak Ditemukan",
            "Tipe Komentar": tipe_komentar,
            "Nomor Usulan": nomor_usulan,
        })
return rows

```

```

def run_inference(messages):
    _done = threading.Event()
    _result = {}

    def _worker():
        _result["response"] = ollama.chat(
            model=MODEL_NAME,
            messages=messages,
            options={"temperature": 0.0, "num_ctx": 8192, "num_predict": 4096}
        )
        _done.set()

    thread = threading.Thread(target=_worker, daemon=True)
    thread.start()

    with tqdm(
        desc=" Inferencing", unit="s", ncols=72,
        bar_format="{desc}: {elapsed} elapsed [{postfix}]",
        dynamic_ncols=False, leave=False, position=1,
    ) as ibar:
        while not _done.is_set():
            time.sleep(1)
            ibar.update(1)
            ibar.set_postfix_str("waiting for Ollama...")
            ibar.set_postfix_str("done [OK]")

    thread.join()
    raw = _result["response"].message.content
    thinking_text = getattr(_result["response"].message, "thinking", "") or ""
    return raw, thinking_text

def save_row_log(nomor_usulan, idx, komentar, raw, thinking_text, cleaned,
↳ parsed_rows):
    log_path = os.path.join(LOGS_DIR, f"{nomor_usulan}_row{idx}.txt")
    with open(log_path, "w", encoding="utf-8") as f:
        f.write(f"{'=' * 70}\n")
        f.write(f"NOMOR USULAN : {nomor_usulan}\n")
        f.write(f"DATASET : {DATASET_NAME} | COLUMN: {TEXT_COL}\n")
        f.write(f"TIPE KOMENTAR : {TIPE_KOMENTAR}\n")
        f.write(f"TIMESTAMP : {datetime.now().strftime('%Y-%m-%d %H:%M:
↳ %S')}\n")
        f.write(f"{'=' * 70}\n\n")
        f.write(f"[KOMENTAR ASLI]\n")
        f.write(komentar + "\n\n")
        f.write(f"[THINKING - reasoning block]\n")
        f.write((thinking_text or "(empty)") + "\n\n")

```

```

f.write("[RAW OUTPUT]\n")
f.write((raw or "(empty)") + "\n\n")
f.write("[CLEANED OUTPUT]\n")
f.write((cleaned or "(empty)") + "\n\n")
f.write("[REPR - exact bytes into parser]\n")
f.write(repr(cleaned) + "\n\n")
f.write(f"[PARSED ROWS: {len(parsed_rows)}]\n")
for r in parsed_rows:
    f.write(
        f"    Aspek          : {r['Istilah Aspek']}\n"
        f"    Sentimen        : {r['Kelas Sentimen']}\n"
        f"    Frasa           : {r['Frasa Kunci Penanda Sentimen']}\n"
        f"    Ekspresi        : {r['Ekspresi Indikator Sentimen']}\n"
        f"    Generalisasi    : {r['Ekspresi Generalisasi']}\n\n"
    )

def save_excel(results):
    pd.DataFrame(results).to_excel(OUTPUT_EXCEL, index=False)

print("All helper functions ready.")

```

4.3 Run LLM

```

[ ]: if remaining == 0:
    print("No rows to process. Check ROW_START / ROW_END in Cell 4.")
else:
    print(f"Starting - {remaining} row(s) to process")
    print(f"  Logs   → {LOGS_DIR}")
    print(f"  Excel  → {OUTPUT_EXCEL}")
    print()

    last_processed = None    # tracks the most-recently completed nomor_usulan

    try:
        with tqdm(
            total=remaining,
            desc="  Rows",
            unit="row",
            ncols=80,
            position=0,
        ) as pbar:

            for idx, row in rows_to_process.iterrows():
                nomor_usulan = str(row.get("nomor_usulan", idx))
                komentar      = str(row.get(TEXT_COL, "")).strip()

```

```

# Empty / NaN row
if not komentar or komentar.lower() == "nan":
    skipped = [{
        "Istilah Aspek": aspek,
        "Kelas Sentimen": "tidak relevan",
        "Frasa Kunci Penanda Sentimen": "SKIPPED",
        "Ekspresi Indikator Sentimen": "komentar kosong",
        "Ekspresi Generalisasi": "komentar kosong",
        "Tipe Komentar": TIPE_KOMENTAR,
        "Nomor Usulan": nomor_usulan,
        "Dataset": DATASET_NAME,
    } for aspek in ASPEK_LIST]
    all_results.extend(skipped)
    save_excel(all_results)
    with open(os.path.join(LOGS_DIR, f"{nomor_usulan}_row{idx}.
↳txt"), "w", encoding="utf-8") as f:
        f.write(f"NOMOR USULAN: {nomor_usulan}\nSTATUS: SKIPPED -\n
↳komentar kosong\n")
        pbar.set_postfix_str(f"{nomor_usulan} SKIPPED")
        pbar.update(1)
        continue

# Build prompt + run inference
pbar.set_postfix_str(f"{nomor_usulan} inferencing...")
messages = build_messages_all(ASPEK_LIST, komentar, TIPE_KOMENTAR,
↳nomor_usulan)

try:
    raw, thinking_text = run_inference(messages)
except Exception as e:
    tqdm.write(f" [ERROR] {nomor_usulan}: {e}")
    error_rows = [{
        "Istilah Aspek": aspek,
        "Kelas Sentimen": "ERROR",
        "Frasa Kunci Penanda Sentimen": "ERROR",
        "Ekspresi Indikator Sentimen": str(e),
        "Ekspresi Generalisasi": "ERROR",
        "Tipe Komentar": TIPE_KOMENTAR,
        "Nomor Usulan": nomor_usulan,
        "Dataset": DATASET_NAME,
    } for aspek in ASPEK_LIST]
    all_results.extend(error_rows)
    save_excel(all_results)
    with open(os.path.join(LOGS_DIR, f"{nomor_usulan}_row{idx}.
↳txt"), "w", encoding="utf-8") as f:

```

```

        f.write(f"NOMOR USULAN: {nomor_usulan}\nSTATUS:␣
↳ERROR\n{e}\n")
        pbar.update(1)
        continue

    # Guard: model returned nothing
    if not raw or not raw.strip():
        tqdm.write(f"    [WARN] {nomor_usulan}: model returned empty␣
↳output")
        if thinking_text:
            tqdm.write(
                f"        Thinking was {len(thinking_text)} chars - "
                f"raise num_predict if this keeps happening"
            )

    # Parse
    cleaned      = clean_raw_output(raw or "")
    parsed_rows = parse_table_all(cleaned, ASPEK_LIST, TIPE_KOMENTAR,␣
↳nomor_usulan)
    for r in parsed_rows:
        r["Dataset"] = DATASET_NAME

    all_results.extend(parsed_rows)

    # Save log + Excel immediately after each row
    save_row_log(nomor_usulan, idx, komentar, raw or "", thinking_text,␣
↳cleaned, parsed_rows)
    save_excel(all_results)

    last_processed = nomor_usulan
    pbar.set_postfix_str(f"{nomor_usulan} [OK] {len(parsed_rows)} rows")
    pbar.update(1)

except KeyboardInterrupt:
    print(f"\n[WARNING] Processing interrupted!")
    print(f"    Last row completed : {last_processed or 'none (no rows␣
↳finished)'}")
    print(f"    Resume: re-run Cell 7 → Cell 11; already-saved rows are␣
↳auto-skipped.")
    finally:
        if last_processed:
            print(f"\n    [OK] Last successfully saved row : {last_processed}")

print()
print("=" * 80)
print("DONE!")

```

```
print(f" Total result rows : {len(all_results)}")
print(f" Excel              : {OUTPUT_EXCEL}")
print(f" Logs folder        : {LOGS_DIR}")
print("=" * 80)
```

5 Analisis & Kombinasi Tesaurus - ACD (AK + AL)

Tahap ini memproses tesaurus ACD_AK dan ACD_AL secara terpisah (lower-casing, quality control terpadu, penghapusan duplikat), menggabungkan keduanya menjadi satu tesaurus ACD, memeriksa duplikat lintas-sumber, lalu menyimpan hasil akhir ke Excel.

Quality control terpadu (`drop_invalid_rows`): menghapus baris dengan (1) Frasa Kunci Penanda Relasi tidak valid, dan/atau (2) Istilah Aspek di luar 8 aspek yang ditentukan, sehingga angka Total baris dimuat dan total tabel distribusi selalu konsisten.

```
[ ]: import pandas as pd
import os
from google.colab import drive
drive.mount('/content/drive')

# Konfigurasi
BASE_DIR = "/content/drive/MyDrive/Thesaurus"

FILE_PATH_AK = f"{BASE_DIR}/ACD/gemma4-latest/HASIL_ACD_AK_synonym.xlsx"
FILE_PATH_AL = f"{BASE_DIR}/ACD/gemma4-latest/HASIL_ACD_AL_synonym.xlsx"

OUTPUT_DIR_COMBINED = f"{BASE_DIR}/ACD/Distribution"
OUTPUT_DIR_COMBINED_CLEAN = f"{BASE_DIR}/ACD/Distribution_clean"

ASPEK_LIST = [
    "capaian pembelajaran lulusan",
    "indeks prestasi kumulatif",
    "kelulusan tepat waktu",
    "masa tunggu lulusan",
    "pelacakan dan perekaman data lulusan",
    "prestasi mahasiswa",
    "karya dengan hak kekayaan intelektual",
    "kesesuaian bidang kerja",
]

VALID_FRASA_RELASI = {
    "disebut sebagai",
    "disingkat sebagai",
    "dikenal sebagai",
    "merujuk pada",
    "mengacu pada",
    "merupakan istilah lain dari",
    "tidak ditemukan",
}

# Kolom teks yang di-lowercase di awal (sebelum analisis apa pun) agar
# perbandingan & deduplikasi antar-sumber (AK vs AL) konsisten.
```

```

TEXT_COLUMNS_TO_LOWER = ["Istilah Aspek", "Frasa Kunci Penanda Relasi",
    ↪ "Sinonim/Padanan"]
DUP_KEYS = ["Istilah Aspek", "Sinonim/Padanan"]

os.makedirs(OUTPUT_DIR_COMBINED, exist_ok=True)
os.makedirs(OUTPUT_DIR_COMBINED_CLEAN, exist_ok=True)
print(f"[OK] File AK          : {FILE_PATH_AK}")
print(f"[OK] File AL          : {FILE_PATH_AL}")
print(f"[OK] Combined raw dir   : {OUTPUT_DIR_COMBINED}")
print(f"[OK] Combined clean dir : {OUTPUT_DIR_COMBINED_CLEAN}")

```

Mounted at /content/drive

```

[OK] File AK          : /content/drive/MyDrive/Thesaurus/ACD/gemma4-latest/
    ↪ HASIL_ACD_AK_synonym.xlsx
[OK] File AL          : /content/drive/MyDrive/Thesaurus/ACD/gemma4-latest/
    ↪ HASIL_ACD_AL_synonym.xlsx
[OK] Combined raw dir   : /content/drive/MyDrive/Thesaurus/ACD/Distribution
[OK] Combined clean dir :
/content/drive/MyDrive/Thesaurus/ACD/Distribution_clean

```

5.1 Fungsi Bantu

```

[ ]: def lowercase_text_columns(df: pd.DataFrame, columns=TEXT_COLUMNS_TO_LOWER) ->
    ↪ pd.DataFrame:
    """Lower-case (+ strip) kolom teks agar perbandingan & deduplikasi
    ↪ konsisten."""
    df = df.copy()
    for col in columns:
        if col in df.columns:
            df[col] = df[col].astype(str).str.strip().str.lower()
    return df

def print_summary_table(summary_df: pd.DataFrame, title: str = "RINGKASAN"):
    """Cetak tabel ringkasan ASCII per aspek."""
    col_w = max(len(a) for a in summary_df["Aspek"]) + 4
    sep     = "=" * (col_w + 16 + 20 + 10)
    hdr_line = "-" * col_w + "+" + "-" * 12 + "+" + "-" * 18 + "+" + "-" * 10

    print(sep)
    print(f" {title}")
    print(sep)
    print(f" {'ASPEK':<{col_w}} | {'DITEMUKAN':>10} | {'TIDAK DITEMUKAN':>15} |
    ↪ {'TOTAL':>7}")
    print(hdr_line)
    for _, row in summary_df.iterrows():
        print(

```

```

        f" {row['Aspek']:<{col_w}} | {row['Ditemukan']:>10,} "
        f"| {row['Tidak Ditemukan']:>15,} | {row['Total']:>7,}"
    )
    print(sep)
    totals = summary_df[["Ditemukan", "Tidak Ditemukan", "Total"]].sum()
    print(
        f" {'TOTAL':<{col_w}} | {totals['Ditemukan']:>10,} "
        f"| {totals['Tidak Ditemukan']:>15,} | {totals['Total']:>7,}"
    )
    print(sep)

def count_found(sub: pd.DataFrame):
    """Hitung baris berdasarkan nilai literal kolom Sinonim/Padanan."""
    not_found = sub["Sinonim/Padanan"].astype(str).str.strip().str.lower() == "
↳ tidak ditemukan"
    return (~not_found).sum(), not_found.sum()

def build_distribution(df: pd.DataFrame, aspek_list=ASPEK_LIST) -> pd.DataFrame:
    """Bangun dataframe ringkasan Ditemukan / Tidak Ditemukan / Total per aspek.
    ↳ """
    rows = []
    for aspek in aspek_list:
        sub = df[df["Istilah Aspek"].str.strip().str.lower() == aspek.lower()]
        ditemukan, tidak = count_found(sub)
        rows.append({"Aspek": aspek, "Ditemukan": ditemukan,
                    "Tidak Ditemukan": tidak, "Total": len(sub)})
    return pd.DataFrame(rows)

def drop_invalid_rows(df: pd.DataFrame, valid_frasa=VALID_FRASA_RELASI,
                    aspek_list=ASPEK_LIST, label=""):
    """Data quality control - satu pass, dua cek:
    1. Frasa Kunci Penanda Relasi tidak ada di VALID_FRASA_RELASI
    2. Istilah Aspek tidak ada di ASPEK_LIST
    Baris yang gagal salah satu (atau keduanya) dihapus sekaligus.
    """
    col_frasa = df["Frasa Kunci Penanda Relasi"].astype(str).str.strip().str.
    ↳ lower()
    col_aspek = df["Istilah Aspek"].astype(str).str.strip().str.lower()
    aspek_lower = {a.lower() for a in aspek_list}

    mask_bad_frasa = ~col_frasa.isin(valid_frasa)
    mask_bad_aspek = ~col_aspek.isin(aspek_lower)
    mask_bad = mask_bad_frasa | mask_bad_aspek
    df_bad = df[mask_bad].copy()

```

```

print("=" * 70)
print(f" DATA QUALITY REPORT - {label}")
print("=" * 70)
print(f" Total baris diperiksa           : {len(df):,}")
print(f" Total baris invalid              : {len(df_bad):,}")
print(f" - Frasa Kunci Penanda Relasi inv : {mask_bad_frasa.sum():,}")
print(f" - Istilah Aspek di luar daftar    : {mask_bad_aspek.sum():,}")
print(f" - Keduanya invalid                : {(mask_bad_frasa &
↳mask_bad_aspek).sum():,}")
print()
print(f" {'ASPEK':<45} | {'FRASA INV':>10} | {'TOTAL INV':>10}")
print(" " + "-" * 70)
for aspek in aspek_list:
    m = col_aspek == aspek.lower()
    f = (m & mask_bad_frasa).sum()
    t = (m & mask_bad).sum()
    print(f" {aspek:<45} | {f:>10,} | {t:>10,}")
    # Rows with unknown aspek are already counted in the summary above;
    # no extra table row - the header line "Istilah Aspek di luar daftar"
    ↳covers them.
print("=" * 70)

if len(df_bad) > 0:
    print(f"\nSample (up to 10) invalid rows:")
    print(df_bad[["Istilah Aspek", "Frasa Kunci Penanda Relasi",
                  "Sinonim/Padanan"]].head(10).to_string(index=True))
else:
    print("\n[OK] Tidak ada baris invalid.")

df_ok = df[~mask_bad].copy().reset_index(drop=True)
print(f"\n[OK] Dihapus {len(df_bad):,} baris invalid. Tersisa: {len(df_ok):
↳,} baris.")
return df_ok

def report_duplicates(df: pd.DataFrame, aspek_list=ASPEK_LIST,
↳dup_keys=DUP_KEYS, label=""):
    """Hitung & cetak jumlah duplikat per aspek (tanpa menghapus)."""
    print("=" * 70)
    print(f" LAPORAN DUPLIKAT PER ASPEK - {label}")
    print("=" * 70)
    total_dup = 0
    for aspek in aspek_list:
        sub = df[df["Istilah Aspek"].str.strip().str.lower() == aspek.lower()]
        n_dup = sub.duplicated(subset=dup_keys, keep="first").sum()
        total_dup += n_dup

```

```

        print(f" {aspek:<45} → {n_dup:>4} duplikat")
    print("=" * 70)
    print(f" {'TOTAL':<45} → {total_dup:>4} duplikat")
    print("=" * 70)

def remove_duplicates(df: pd.DataFrame, aspek_list=ASPEK_LIST,
    ↳dup_keys=DUP_KEYS, label=""):
    """Hapus duplikat per aspek & cetak laporan sebelum/sesudah."""
    print("=" * 70)
    print(f" PENGHAPUSAN DUPLIKAT PER ASPEK - {label}")
    print("=" * 70)
    parts = []
    for aspek in aspek_list:
        mask = df["Istilah Aspek"].str.strip().str.lower() == aspek.lower()
        sub = df[mask].copy()
        before = len(sub)
        sub_clean = sub.drop_duplicates(subset=dup_keys, keep="first")
        removed = before - len(sub_clean)
        parts.append(sub_clean)
        print(f" {aspek:<45} → dihapus {removed:>4} baris "
            f"(sebelum: {before:>4}, sesudah: {len(sub_clean):>4})")
    df_clean = pd.concat(parts, ignore_index=True)
    total_removed = len(df) - len(df_clean)
    print("=" * 70)
    print(f" {'TOTAL DIHAPUS':<45} → {total_removed:>4} baris")
    print(f" {'TOTAL TERSISA':<45} → {len(df_clean):>4} baris")
    print("=" * 70)
    return df_clean

def save_per_aspek(df: pd.DataFrame, output_dir: str, aspek_list=ASPEK_LIST,
    ↳suffix=""):
    """Simpan dataframe per aspek menjadi file Excel terpisah."""
    print(f"Menyimpan file ke {output_dir} ...")
    for aspek in aspek_list:
        sub = df[df["Istilah Aspek"].str.strip().str.lower() == aspek.lower()]
        ↳copy()
        safe_name = aspek.replace(" ", "_").replace("/", "-")
        out_path = os.path.join(output_dir, f"{safe_name}{suffix}.xlsx")
        sub.to_excel(out_path, index=False)
        print(f" [OK] {aspek:<45} → {len(sub):>4} baris → {out_path}")

```

5.2 ACD_AK

```
[ ]: # [AK] Load data
df_ak = pd.read_excel(FILE_PATH_AK)
print(f"Total baris dimuat : {len(df_ak):,}")
print(f"Kolom                : {list(df_ak.columns)}")
df_ak.head(3)
```

Total baris dimuat : 4,752

Kolom : ['Istilah Aspek', 'Frasa Kunci Penanda Relasi', 'Sinonim/Padanan', 'Tipe Komentar', 'Nomor Usulan', 'Dataset']

```
[ ]:                               Istilah Aspek Frasa Kunci Penanda Relasi \
0   capaian pembelajaran lulusan                disingkat sebagai
1     indeks prestasi kumulatif                disingkat sebagai
2     kelulusan tepat waktu                    disebut sebagai

                               Sinonim/Padanan Tipe Komentar          Nomor Usulan \
0                                   cp                        AK
1                                   ipk                       AK
2   prosentase yang lulus tepat waktu                    AK

Dataset
0   ACD_AK
1   ACD_AK
2   ACD_AK
```

```
[ ]: # [AK] Lower-case seluruh kolom teks
df_ak = lowercase_text_columns(df_ak)
```

```
[ ]: # [AK] Data quality control: Frasa Kunci invalid + Istilah Aspek di luar daftar
df_ak = drop_invalid_rows(df_ak, label="ACD_AK")
```

DATA QUALITY REPORT - ACD_AK

```
=====
Total baris diperiksa      : 4,752
Total baris invalid       : 265
- Frasa Kunci Penanda Relasi inv : 252
- Istilah Aspek di luar daftar  : 14
- Keduanya invalid         : 1
```

ASPEK	FRASA INV	TOTAL INV
capaian pembelajaran lulusan	25	25
indeks prestasi kumulatif	13	13
kelulusan tepat waktu	31	31
masa tunggu lulusan	38	38
pelacakan dan perekaman data lulusan	40	40

prestasi mahasiswa		41		41
karya dengan hak kekayaan intelektual		22		22
kesesuaian bidang kerja		41		41

=====

Sample (up to 10) invalid rows:

	Istilah Aspek	Frasa Kunci Penanda
Relasi	Sinonim/Padanan	
58	kelulusan tepat waktu	muncul
sebagai	kelulusan tepat waktu	
59	masa tunggu lulusan	muncul
sebagai	masa tunggu	
61	prestasi mahasiswa	muncul
sebagai	akademik / non akademik	
63	kesesuaian bidang kerja	muncul
sebagai	kesesuaian bidang kerja	
108	pelacakan dan perekaman data lulusan (tidak ditemukan sinonim/variasi eksplisit)	-
161	capaian pembelajaran lulusan	dilakukan
melalui	audit mutu internal (ami)	
165	pelacakan dan perekaman data lulusan	diperoleh
melalui	tracer study	
168	prestasi mahasiswa	
meliputi	juara 2 turnamen futsal ypp cup 2017	
219	masa tunggu lulusan	(tidak ada sinonim
eksplisit)	rata-rata masa tunggu lulusan	
235	masa tunggu lulusan	
(n/a)	rata-rata masa tunggu	

[OK] Dihapus 265 baris invalid. Tersisa: 4,487 baris.

```
[ ]: # [AK] Distribusi setelah quality control
dist_ak_filtered = build_distribution(df_ak)
print_summary_table(dist_ak_filtered, title="ACD_AK - SEBELUM PENGHAPUSAN_
↳DUPLIKAT")
```

=====

=====

ACD_AK - SEBELUM PENGHAPUSAN DUPLIKAT

=====

=====

ASPEK		DITEMUKAN		TIDAK DITEMUKAN	
TOTAL					

-----+-----+-----+-----

capaian pembelajaran lulusan		440		147	
587					
indeks prestasi kumulatif		495		83	

578			
kelulusan tepat waktu		237	311
548			
masa tunggu lulusan		297	245
542			
pelacakan dan perekaman data lulusan		285	269
554			
prestasi mahasiswa		289	272
561			
karya dengan hak kekayaan intelektual		388	190
578			
kesesuaian bidang kerja		275	264
539			
=====			
=====			
TOTAL		2,706	1,781
4,487			
=====			
=====			

```
[ ]: # [AK] Cek duplikat per aspek
report_duplicates(df_ak, label="ACD_AK")
```

=====	
LAPORAN DUPLIKAT PER ASPEK - ACD_AK	
=====	
capaian pembelajaran lulusan	→ 503 duplikat
indeks prestasi kumulatif	→ 532 duplikat
kelulusan tepat waktu	→ 427 duplikat
masa tunggu lulusan	→ 433 duplikat
pelacakan dan perekaman data lulusan	→ 412 duplikat
prestasi mahasiswa	→ 372 duplikat
karya dengan hak kekayaan intelektual	→ 538 duplikat
kesesuaian bidang kerja	→ 371 duplikat
=====	
TOTAL	→ 3588 duplikat
=====	

```
[ ]: # [AK] Hapus duplikat per aspek
df_ak_clean = remove_duplicates(df_ak, label="ACD_AK")
```

=====	
PENGHAPUSAN DUPLIKAT PER ASPEK - ACD_AK	
=====	
capaian pembelajaran lulusan	→ dihapus 503 baris (sebelum: 587, sesudah: 84)
indeks prestasi kumulatif	→ dihapus 532 baris (sebelum: 578, sesudah: 46)
kelulusan tepat waktu	→ dihapus 427 baris (sebelum: 578, sesudah: 46)

```

548, sesudah: 121)
    masa tunggu lulusan                → dihapus 433 baris (sebelum:
542, sesudah: 109)
    pelacakan dan perekaman data lulusan → dihapus 412 baris (sebelum:
554, sesudah: 142)
    prestasi mahasiswa                 → dihapus 372 baris (sebelum:
561, sesudah: 189)
    karya dengan hak kekayaan intelektual → dihapus 538 baris (sebelum:
578, sesudah: 40)
    kesesuaian bidang kerja            → dihapus 371 baris (sebelum:
539, sesudah: 168)
=====
TOTAL DIHAPUS                → 3588 baris
TOTAL TERSISA                → 899 baris
=====

```

```

[ ]: # [AK] Distribusi setelah penghapusan duplikat
dist_ak_clean = build_distribution(df_ak_clean)
print_summary_table(dist_ak_clean, title="ACD_AK - SESUDAH PENGHAPUSAN_
↳DUPLIKAT")

```

```

=====
=====
ACD_AK - SESUDAH PENGHAPUSAN DUPLIKAT
=====
=====
ASPEK                                | DITEMUKAN | TIDAK DITEMUKAN |
TOTAL
-----+-----+-----+-----
----
capaian pembelajaran lulusan      |      83 |      1 |
84
indeks prestasi kumulatif         |      45 |      1 |
46
kelulusan tepat waktu             |     120 |      1 |
121
masa tunggu lulusan               |     108 |      1 |
109
pelacakan dan perekaman data lulusan |     141 |      1 |
142
prestasi mahasiswa                 |     188 |      1 |
189
karya dengan hak kekayaan intelektual |      39 |      1 |
40
kesesuaian bidang kerja           |     167 |      1 |
168
=====
=====

```

TOTAL		891		8	
899					

=====

=====

```
[ ]: # (cel ini sengaja dikosongkan setelah restrukturisasi blok AK)
```

5.3 ACD_AL

```
[ ]: # [AL] Load data
df_al = pd.read_excel(FILE_PATH_AL)
print(f"Total baris dimuat : {len(df_al):,}")
print(f"Kolom : {list(df_al.columns)}")
df_al.head(3)
```

```
Total baris dimuat : 2,319
Kolom : ['Istilah Aspek', 'Frasa Kunci Penanda Relasi',
'Sinonim/Padanan', 'Tipe Komentar', 'Nomor Usulan', 'Dataset']
```

```
[ ]:
Istilah Aspek Frasa Kunci Penanda Relasi \
0        capaian pembelajaran lulusan        disebut sebagai
1        indeks prestasi kumulatif        disingkat sebagai
2  pelacakan dan perekaman data lulusan        merujuk pada

Sinonim/Padanan Tipe Komentar        Nomor Usulan Dataset
0        cpl        AL        ACD_AL
1        ipk        AL        ACD_AL
2  rekap hasil tracer study        AL        ACD_AL
```

```
[ ]: # [AL] Lower-case seluruh kolom teks
df_al = lowercase_text_columns(df_al)
```

```
[ ]: # [AL] Data quality control: Frasa Kunci invalid + Istilah Aspek di luar daftar
df_al = drop_invalid_rows(df_al, label="ACD_AL")
```

=====

DATA QUALITY REPORT - ACD_AL

=====

```
Total baris diperiksa : 2,319
Total baris invalid : 157
- Frasa Kunci Penanda Relasi inv : 152
- Istilah Aspek di luar daftar : 5
- Keduanya invalid : 0
```

ASPEK		FRASA INV		TOTAL INV
capaian pembelajaran lulusan		13		13
indeks prestasi kumulatif		8		8
kelulusan tepat waktu		19		19

masa tunggu lulusan		16		16
pelacakan dan perekaman data lulusan		24		24
prestasi mahasiswa		27		27
karya dengan hak kekayaan intelektual		21		21
kesesuaian bidang kerja		24		24

=====

Sample (up to 10) invalid rows:

	Istilah Aspek	Frasa Kunci Penanda Relasi
Sinonim/Padanan		
11	kelulusan tepat waktu	-
	rata-rata kelulusan tepat waktu	
90	capaian pembelajaran lulusan	dijelaskan atas
	capaian prodi	
114	kelulusan tepat waktu	nan
	rata-rata kelulusan selama 4 tahun terakhir	
115	masa tunggu lulusan	nan
	rata-rata masa tunggu	
118	prestasi mahasiswa	nan
	mahasiswa berprestasi	
121	kesesuaian bidang kerja	nan
	kesesuaian bidang kerja	
179	prestasi mahasiswa	tidak ada sinonim/padanan eksplisit
-		
181	kesesuaian bidang kerja	tidak ada sinonim/padanan eksplisit
-		
186	pelacakan dan perekaman data lulusan	*(tidak ditemukan sinonim/variasi)*
-		
188	karya dengan hak kekayaan intelektual	*(tidak ditemukan dalam komentar)*
-		

[OK] Dihapus 157 baris invalid. Tersisa: 2,162 baris.

```
[ ]: # [AL] Distribusi setelah quality control
dist_al_filtered = build_distribution(df_al)
print_summary_table(dist_al_filtered, title="ACD_AL - SEBELUM PENGHAPUSAN_
↳DUPLIKAT")
```

=====

=====

ACD_AL - SEBELUM PENGHAPUSAN DUPLIKAT		
---------------------------------------	--	--

=====

=====

ASPEK		DITEMUKAN		TIDAK DITEMUKAN	
TOTAL					

-----+-----+-----+-----

capaian pembelajaran lulusan		212		75	
------------------------------	--	-----	--	----	--

287	indeks prestasi kumulatif		250		32	
282	kelulusan tepat waktu		129		136	
265	masa tunggu lulusan		154		112	
266	pelacakan dan perekaman data lulusan		160		104	
264	prestasi mahasiswa		159		112	
271	karya dengan hak kekayaan intelektual		182		84	
266	kesesuaian bidang kerja		145		116	
261						
=====						
=====						
	TOTAL		1,391		771	
2,162						
=====						
=====						

```
[ ]: # [AL] Cek duplikat per aspek
report_duplicates(df_al, label="ACD_AL")
```

```
=====
LAPORAN DUPLIKAT PER ASPEK - ACD_AL
=====
capaian pembelajaran lulusan          → 241 duplikat
indeks prestasi kumulatif              → 256 duplikat
kelulusan tepat waktu                  → 193 duplikat
masa tunggu lulusan                    → 197 duplikat
pelacakan dan perekaman data lulusan  → 173 duplikat
prestasi mahasiswa                     → 166 duplikat
karya dengan hak kekayaan intelektual  → 239 duplikat
kesesuaian bidang kerja                 → 152 duplikat
=====
TOTAL                                  → 1617 duplikat
=====
```

```
[ ]: # [AL] Hapus duplikat per aspek
df_al_clean = remove_duplicates(df_al, label="ACD_AL")
```

```
=====
PENGHAPUSAN DUPLIKAT PER ASPEK - ACD_AL
=====
capaian pembelajaran lulusan          → dihapus 241 baris (sebelum:
287, sesudah: 46)
indeks prestasi kumulatif              → dihapus 256 baris (sebelum:
```

```

282, sesudah: 26)
kelulusan tepat waktu → dihapus 193 baris (sebelum:
265, sesudah: 72)
masa tunggu lulusan → dihapus 197 baris (sebelum:
266, sesudah: 69)
pelacakan dan perekaman data lulusan → dihapus 173 baris (sebelum:
264, sesudah: 91)
prestasi mahasiswa → dihapus 166 baris (sebelum:
271, sesudah: 105)
karya dengan hak kekayaan intelektual → dihapus 239 baris (sebelum:
266, sesudah: 27)
kesesuaian bidang kerja → dihapus 152 baris (sebelum:
261, sesudah: 109)
=====
TOTAL DIHAPUS → 1617 baris
TOTAL TERSISA → 545 baris
=====

```

```

[ ]: # [AL] Distribusi setelah penghapusan duplikat
dist_al_clean = build_distribution(df_al_clean)
print_summary_table(dist_al_clean, title="ACD_AL - SESUDAH PENGHAPUSAN_
↳DUPLIKAT")

```

```

=====
=====
ACD_AL - SESUDAH PENGHAPUSAN DUPLIKAT
=====
=====
ASPEK | DITEMUKAN | TIDAK DITEMUKAN |
TOTAL
-----+-----+-----+-----
----
capaian pembelajaran lulusan | 45 | 1 |
46
indeks prestasi kumulatif | 25 | 1 |
26
kelulusan tepat waktu | 71 | 1 |
72
masa tunggu lulusan | 68 | 1 |
69
pelacakan dan perekaman data lulusan | 90 | 1 |
91
prestasi mahasiswa | 104 | 1 |
105
karya dengan hak kekayaan intelektual | 26 | 1 |
27
kesesuaian bidang kerja | 108 | 1 |
109

```

```
=====
```

TOTAL		537		8	
-------	--	-----	--	---	--

```
545
=====
=====
```

```
[ ]: # (cel ini sengaja dikosongkan setelah restrukturisasi blok AL)
```

5.4 Kombinasi ACD_AK + ACD_AL → ACD

```
[ ]: # Gabungkan thesaurus AK (bersih) + AL (bersih)
df_combined = pd.concat([df_ak_clean, df_al_clean], ignore_index=True)
print(f"Total baris gabungan: {len(df_combined):,} "
      f"(AK: {len(df_ak_clean):,} + AL: {len(df_al_clean):,})")

dist_combined_raw = build_distribution(df_combined)
print_summary_table(dist_combined_raw, title="ACD GABUNGAN - SETELAH KOMBINASI")
```

Total baris gabungan: 1,444 (AK: 899 + AL: 545)

```
=====
```

ACD GABUNGAN - SETELAH KOMBINASI					
=====					
ASPEK			DITEMUKAN		TIDAK DITEMUKAN
TOTAL					
-----+-----+-----+-----					

capaian pembelajaran lulusan			128		2
130					
indeks prestasi kumulatif			70		2
72					
kelulusan tepat waktu			191		2
193					
masa tunggu lulusan			176		2
178					
pelacakan dan perekaman data lulusan			231		2
233					
prestasi mahasiswa			292		2
294					
karya dengan hak kekayaan intelektual			65		2
67					
kesesuaian bidang kerja			275		2
277					
=====					
=====					
TOTAL			1,428		16

1,444

=====

```
[ ]: # Simpan hasil gabungan ke folder ACD/Distribution
save_per_aspek(df_combined, OUTPUT_DIR_COMBINED)
```

Menyimpan file ke /content/drive/MyDrive/Thesaurus/ACD/Distribution ...

[OK] capaian pembelajaran lulusan → 130 baris → /content/
↳drive/

MyDrive/Thesaurus/ACD/Distribution/capaian_pembelajaran_lulusan.xlsx

[OK] indeks prestasi kumulatif → 72 baris → /content/
↳drive/

MyDrive/Thesaurus/ACD/Distribution/indeks_prestasi_kumulatif.xlsx

[OK] kelulusan tepat waktu → 193 baris →
/content/drive/MyDrive/Thesaurus/ACD/Distribution/kelulusan_tepat_waktu.xlsx

[OK] masa tunggu lulusan → 178 baris →
/content/drive/MyDrive/Thesaurus/ACD/Distribution/masa_tunggu_lulusan.xlsx

[OK] pelacakan dan perekaman data lulusan → 233 baris → /content/
↳drive/

MyDrive/Thesaurus/ACD/Distribution/pelacakan_dan_perekaman_data_lulusan.xlsx

[OK] prestasi mahasiswa → 294 baris →
/content/drive/MyDrive/Thesaurus/ACD/Distribution/prestasi_mahasiswa.xlsx

[OK] karya dengan hak kekayaan intelektual → 67 baris → /content/
↳drive/

MyDrive/Thesaurus/ACD/Distribution/karya_dengan_hak_kekayaan_intelektual.xlsx

[OK] kesesuaian bidang kerja → 277 baris → /content/
↳drive/

MyDrive/Thesaurus/ACD/Distribution/kesesuaian_bidang_kerja.xlsx

```
[ ]: # Cek duplikat lintas-sumber pada data gabungan (AK & AL bisa beririsan)
report_duplicates(df_combined, label="ACD GABUNGAN")
```

=====

LAPORAN DUPLIKAT PER ASPEK - ACD GABUNGAN

=====

capaian pembelajaran lulusan	→ 17 duplikat
indeks prestasi kumulatif	→ 11 duplikat
kelulusan tepat waktu	→ 41 duplikat
masa tunggu lulusan	→ 42 duplikat
pelacakan dan perekaman data lulusan	→ 37 duplikat
prestasi mahasiswa	→ 43 duplikat
karya dengan hak kekayaan intelektual	→ 7 duplikat
kesesuaian bidang kerja	→ 52 duplikat

=====

TOTAL	→ 250 duplikat
-------	----------------

=====

```
[ ]: # Hapus duplikat pada data gabungan & simpan hasil bersih
df_combined_clean = remove_duplicates(df_combined, label="ACD GABUNGAN")
save_per_aspek(df_combined_clean, OUTPUT_DIR_COMBINED_CLEAN, suffix="_clean")

dist_combined_clean = build_distribution(df_combined_clean)
print_summary_table(dist_combined_clean, title="ACD GABUNGAN - SESUDAH_
↳PENGHAPUSAN DUPLIKAT")
```

```
=====
PENGHAPUSAN DUPLIKAT PER ASPEK - ACD GABUNGAN
=====
    capaian pembelajaran lulusan          → dihapus   17 baris (sebelum:
130, sesudah:  113)
    indeks prestasi kumulatif            → dihapus   11 baris (sebelum:
72, sesudah:   61)
    kelulusan tepat waktu                 → dihapus   41 baris (sebelum:
193, sesudah:  152)
    masa tunggu lulusan                   → dihapus   42 baris (sebelum:
178, sesudah:  136)
    pelacakan dan perekaman data lulusan → dihapus   37 baris (sebelum:
233, sesudah:  196)
    prestasi mahasiswa                    → dihapus   43 baris (sebelum:
294, sesudah:  251)
    karya dengan hak kekayaan intelektual → dihapus    7 baris (sebelum:
67, sesudah:   60)
    kesesuaian bidang kerja                → dihapus   52 baris (sebelum:
277, sesudah:  225)
=====
    TOTAL DIHAPUS                        → 250 baris
    TOTAL TERSISA                        → 1194 baris
=====
Menyimpan file ke /content/drive/MyDrive/Thesaurus/ACD/Distribution_clean
...
    [OK] capaian pembelajaran lulusan          → 113 baris → /content/
↳drive/
MyDrive/Thesaurus/ACD/Distribution_clean/capaian_pembelajaran_lulusan_clean.x
lsx
    [OK] indeks prestasi kumulatif            → 61 baris → /content/
↳drive/
MyDrive/Thesaurus/ACD/Distribution_clean/indeks_prestasi_kumulatif_clean.xlsx
    [OK] kelulusan tepat waktu                 → 152 baris → /content/
↳drive/
MyDrive/Thesaurus/ACD/Distribution_clean/kelulusan_tepat_waktu_clean.xlsx
    [OK] masa tunggu lulusan                   → 136 baris → /content/
↳drive/
MyDrive/Thesaurus/ACD/Distribution_clean/masa_tunggu_lulusan_clean.xlsx
```

```

[OK] pelacakan dan perekaman data lulusan          → 196 baris → /content/
↳drive/
MyDrive/Thesaurus/ACD/Distribution_clean/pelacakan_dan_perekaman_data_lulusan
_clean.xlsx
[OK] prestasi mahasiswa                             → 251 baris → /content/
↳drive/
MyDrive/Thesaurus/ACD/Distribution_clean/prestasi_mahasiswa_clean.xlsx
[OK] karya dengan hak kekayaan intelektual          → 60 baris → /content/
↳drive/
MyDrive/Thesaurus/ACD/Distribution_clean/karya_dengan_hak_kekayaan_intelektua
l_clean.xlsx
[OK] kesesuaian bidang kerja                       → 225 baris → /content/
↳drive/
MyDrive/Thesaurus/ACD/Distribution_clean/kesesuaian_bidang_kerja_clean.xlsx
=====
=====
ACD GABUNGAN - SESUDAH PENGHAPUSAN DUPLIKAT
=====
=====
ASPEK                                     | DITEMUKAN | TIDAK DITEMUKAN |
TOTAL
-----+-----+-----+-----
----
capaian pembelajaran lulusan          | 112 | 1 |
113
indeks prestasi kumulatif            | 60 | 1 |
61
kelulusan tepat waktu                | 151 | 1 |
152
masa tunggu lulusan                 | 135 | 1 |
136
pelacakan dan perekaman data lulusan | 195 | 1 |
196
prestasi mahasiswa                   | 250 | 1 |
251
karya dengan hak kekayaan intelektual | 59 | 1 |
60
kesesuaian bidang kerja              | 224 | 1 |
225
=====
=====
TOTAL                                | 1,186 | 8 |
1,194
=====
=====

```

5.5 Setelah Manual Validasi

```
[ ]: # [OPTIONAL] Muat ulang thesaurus ACD final dari folder & cek distribusi
def load_combined_from_folder(folder: str, aspek_list=ASPEK_LIST,
    suffix="_clean"):
    parts = []
    for aspek in aspek_list:
        safe_name = aspek.replace(" ", "_").replace("/", "-")
        path = os.path.join(folder, f"{safe_name}{suffix}.xlsx")
        parts.append(pd.read_excel(path))
    return pd.concat(parts, ignore_index=True)

df_verify = load_combined_from_folder(OUTPUT_DIR_COMBINED_CLEAN)
print(f"Total baris dimuat dari folder : {len(df_verify):,}")

dist_verify = build_distribution(df_verify)
print_summary_table(dist_verify, title="VERIFIKASI - ACD/Distribution_clean")
```

Total baris dimuat dari folder : 736

VERIFIKASI - ACD/Distribution_clean			
ASPEK	DITEMUKAN	TIDAK DITEMUKAN	
TOTAL			
capaian pembelajaran lulusan	82	0	
82			
indeks prestasi kumulatif	44	0	
44			
kelulusan tepat waktu	108	0	
108			
masa tunggu lulusan	73	0	
73			
pelacakan dan perekaman data lulusan	120	0	
120			
prestasi mahasiswa	156	0	
156			
karya dengan hak kekayaan intelektual	45	0	
45			
kesesuaian bidang kerja	108	0	
108			
TOTAL	736	0	
736			

=====

=====

6 Analisis & Kombinasi Sentimen - ASC (AK + AL)

Tahap ini memproses hasil ekstraksi sentimen ASC_AK dan ASC_AL secara terpisah (lower-casing, quality control terpadu, penghapusan duplikat), menggabungkan keduanya menjadi satu dataset ASC, memeriksa duplikat lintas-sumber, lalu menyimpan hasil akhir ke Excel.

Quality control terpadu (drop_invalid_rows): menghapus baris dengan (1) Kelas Sentimen tidak valid, (2) Frasa Kunci Penanda Sentimen tidak valid, dan/atau (3) Istilah Aspek di luar 8 aspek yang ditentukan, sehingga angka Total baris dimuat dan total tabel distribusi selalu konsisten.

```
[ ]: import pandas as pd
import os
from google.colab import drive
drive.mount('/content/drive')

# Konfigurasi
BASE_DIR = "/content/drive/MyDrive/Thesaurus"

FILE_PATH_AK = f"{BASE_DIR}/ASC_AK/gemma4-latest/HASIL_ASC_AK_sentiment.xlsx"
FILE_PATH_AL = f"{BASE_DIR}/ASC_AL/gemma4-latest/HASIL_ASC_AL_sentiment.xlsx"

OUTPUT_DIR_COMBINED = f"{BASE_DIR}/ASC/Distribution"
OUTPUT_DIR_COMBINED_CLEAN = f"{BASE_DIR}/ASC/Distribution_clean"

ASPEK_LIST = [
    "capaian pembelajaran lulusan",
    "indeks prestasi kumulatif",
    "kelulusan tepat waktu",
    "masa tunggu lulusan",
    "pelacakan dan perekaman data lulusan",
    "prestasi mahasiswa",
    "karya dengan hak kekayaan intelektual",
    "kesesuaian bidang kerja",
]

VALID_SENTIMEN = {"positif", "negatif", "netral", "tidak relevan"}
VALID_FRASA = {
    "dinilai positif melalui frasa",
    "dinilai negatif melalui frasa",
    "diidentifikasi sebagai penanda netral melalui frasa",
    "tidak ditemukan",
}

# Kolom teks yang di-lowercase di awal (sebelum analisis apa pun) agar
# perbandingan & deduplikasi antar-sumber (AK vs AL) konsisten.
TEXT_COLUMNS_TO_LOWER = [
    "Istilah Aspek",
```

```

    "Kelas Sentimen",
    "Frasa Kunci Penanda Sentimen",
    "Ekspresi Indikator Sentimen",
    "Ekspresi Generalisasi",
]

# Duplikat ditentukan berdasarkan Istilah Aspek + Ekspresi Generalisasi
DUP_KEYS = ["Istilah Aspek", "Ekspresi Generalisasi"]

os.makedirs(OUTPUT_DIR_COMBINED, exist_ok=True)
os.makedirs(OUTPUT_DIR_COMBINED_CLEAN, exist_ok=True)
print(f"[OK] File AK          : {FILE_PATH_AK}")
print(f"[OK] File AL          : {FILE_PATH_AL}")
print(f"[OK] Combined raw dir   : {OUTPUT_DIR_COMBINED}")
print(f"[OK] Combined clean dir : {OUTPUT_DIR_COMBINED_CLEAN}")

```

```

Mounted at /content/drive
[OK] File AK          : /content/drive/MyDrive/Thesaurus/ASC_AK/gemma4-
latest/HASIL_ASC_AK_sentiment.xlsx
[OK] File AL          : /content/drive/MyDrive/Thesaurus/ASC_AL/gemma4-
latest/HASIL_ASC_AL_sentiment.xlsx
[OK] Combined raw dir  : /content/drive/MyDrive/Thesaurus/ASC/Distribution
[OK] Combined clean dir :
/content/drive/MyDrive/Thesaurus/ASC/Distribution_clean

```

6.1 Fungsi Bantu

```

[ ]: def lowercase_text_columns(df: pd.DataFrame, columns=TEXT_COLUMNS_TO_LOWER) ->
    pd.DataFrame:
    """Lower-case (+ strip) kolom teks agar perbandingan & deduplikasi
    konsisten."""
    df = df.copy()
    for col in columns:
        if col in df.columns:
            df[col] = df[col].astype(str).str.strip().str.lower()
    return df

def count_sentimen(sub: pd.DataFrame):
    """Hitung distribusi Kelas Sentimen per aspek."""
    s = sub["Kelas Sentimen"].astype(str).str.strip().str.lower()
    return {
        "positif": (s == "positif").sum(),
        "negatif": (s == "negatif").sum(),
        "netral": (s == "netral").sum(),
        "tidak relevan": (s == "tidak relevan").sum(),
    }

```

```

def print_summary_table(summary_df: pd.DataFrame, title: str = "RINGKASAN"):
    """Cetak tabel ringkasan ASCII distribusi sentimen per aspek."""
    col_w = max(len(a) for a in summary_df["Aspek"]) + 4
    sep = "=" * (col_w + 14 + 10 + 10 + 10 + 18 + 10)
    hdr = "-" * col_w + "+" + "-" * 10 + "+" + "-" * 10 + "+" + "-" * 10 +
    ↪ "+" + "-" * 16 + "+" + "-" * 10

    print(sep)
    print(f" {title}")
    print(sep)
    print(f" {'ASPEK':<{col_w}} | {'POSITIF':>8} | {'NEGATIF':>8} | {'NETRAL':
    ↪>8} | {'TIDAK RELEVAN':>13} | {'TOTAL':>7}")
    print(hdr)
    for _, row in summary_df.iterrows():
        print(
            f" {row['Aspek']:<{col_w}} "
            f"| {row['positif']:>8,} "
            f"| {row['negatif']:>8,} "
            f"| {row['netral']:>8,} "
            f"| {row['tidak relevan']:>13,} "
            f"| {row['Total']:>7,}"
        )
    print(sep)
    totals = summary_df[["positif", "negatif", "netral", "tidak relevan",
    ↪"Total"]].sum()
    print(
        f" {'TOTAL':<{col_w}} "
        f"| {totals['positif']:>8,} "
        f"| {totals['negatif']:>8,} "
        f"| {totals['netral']:>8,} "
        f"| {totals['tidak relevan']:>13,} "
        f"| {totals['Total']:>7,}"
    )
    print(sep)

def build_distribution(df: pd.DataFrame, aspek_list=ASPEK_LIST) -> pd.DataFrame:
    """Bangun dataframe ringkasan distribusi sentimen per aspek."""
    rows = []
    for aspek in aspek_list:
        sub = df[df["Istilah Aspek"].str.strip().str.lower() == aspek.
    ↪lower()]
        counts = count_sentimen(sub)
        rows.append({"Aspek": aspek, **counts, "Total": len(sub)})
    return pd.DataFrame(rows)

```

```

def drop_invalid_rows(df: pd.DataFrame, valid_sentimen=VALID_SENTIMEN,
    ↪valid_frasa=VALID_FRASA,
        aspek_list=ASPEK_LIST, label=""):
    """Data quality control - satu pass, tiga cek:
        1. Kelas Sentimen tidak ada di VALID_SENTIMEN
        2. Frasa Kunci Penanda Sentimen tidak ada di VALID_FRASA
        3. Istilah Aspek tidak ada di ASPEK_LIST
        Baris yang gagal salah satu (atau lebih) dihapus sekaligus.
    """
    col_sentimen = df["Kelas Sentimen"].astype(str).str.strip().str.lower()
    col_frasa     = df["Frasa Kunci Penanda Sentimen"].astype(str).str.strip().
    ↪str.lower()
    col_aspek     = df["Istilah Aspek"].astype(str).str.strip().str.lower()
    aspek_lower  = {a.lower() for a in aspek_list}

    mask_bad_sentimen = ~col_sentimen.isin(valid_sentimen)
    mask_bad_frasa    = ~col_frasa.isin(valid_frasa)
    mask_bad_aspek    = ~col_aspek.isin(aspek_lower)
    mask_bad          = mask_bad_sentimen | mask_bad_frasa | mask_bad_aspek
    df_bad            = df[mask_bad].copy()

    print("=" * 70)
    print(f" DATA QUALITY REPORT - {label}")
    print("=" * 70)
    print(f" Total baris diperiksa           : {len(df):,}")
    print(f" Total baris invalid                 : {len(df_bad):,}")
    print(f" - Kelas Sentimen invalid             : {mask_bad_sentimen.sum():,}")
    print(f" - Frasa Kunci invalid                 : {mask_bad_frasa.sum():,}")
    print(f" - Istilah Aspek di luar daftar       : {mask_bad_aspek.sum():,}")
    print()
    print(f" {'ASPEK':<45} | {'SENT INV':>9} | {'FRASA INV':>10} | {'TOTAL_
    ↪INV':>10}")
    print(" " + "-" * 83)
    for aspek in aspek_list:
        m = col_aspek == aspek.lower()
        s = (m & mask_bad_sentimen).sum()
        f = (m & mask_bad_frasa).sum()
        t = (m & mask_bad).sum()
        print(f" {aspek:<45} | {s:>9,} | {f:>10,} | {t:>10,}")
    print("=" * 70)

    if len(df_bad) > 0:
        print(f"\nSample (up to 10) invalid rows:")
        print(df_bad[["Istilah Aspek", "Kelas Sentimen",
            "Frasa Kunci Penanda Sentimen",

```

```

        "Ekspresi Indikator Sentimen"]].head(10).
↳to_string(index=True))
    else:
        print("\n[OK] Tidak ada baris invalid.")

    df_ok = df[~mask_bad].copy().reset_index(drop=True)
    print(f"\n[OK] Dihapus {len(df_bad):,} baris invalid. Tersisa: {len(df_ok):
↳,} baris.")
    return df_ok

def report_duplicates(df: pd.DataFrame, aspek_list=ASPEK_LIST,
↳dup_keys=DUP_KEYS, label=""):
    """Hitung & cetak jumlah duplikat per aspek (tanpa menghapus)."""
    print("=" * 70)
    print(f" LAPORAN DUPLIKAT PER ASPEK - {label}")
    print("=" * 70)
    total_dup = 0
    for aspek in aspek_list:
        sub = df[df["Istilah Aspek"].str.strip().str.lower() == aspek.lower()]
        n_dup = sub.duplicated(subset=dup_keys, keep="first").sum()
        total_dup += n_dup
        print(f" {aspek:<45} → {n_dup:>4} duplikat")
    print("=" * 70)
    print(f" {'TOTAL':<45} → {total_dup:>4} duplikat")
    print("=" * 70)

def remove_duplicates(df: pd.DataFrame, aspek_list=ASPEK_LIST,
↳dup_keys=DUP_KEYS, label=""):
    """Hapus duplikat per aspek & cetak laporan sebelum/sesudah."""
    print("=" * 70)
    print(f" PENGHAPUSAN DUPLIKAT PER ASPEK - {label}")
    print("=" * 70)
    parts = []
    for aspek in aspek_list:
        mask = df["Istilah Aspek"].str.strip().str.lower() == aspek.lower()
        sub = df[mask].copy()
        before = len(sub)
        sub_clean = sub.drop_duplicates(subset=dup_keys, keep="first")
        removed = before - len(sub_clean)
        parts.append(sub_clean)
        print(f" {aspek:<45} → dihapus {removed:>4} baris "
              f"(sebelum: {before:>4}, sesudah: {len(sub_clean):>4})")
    df_clean = pd.concat(parts, ignore_index=True)
    total_removed = len(df) - len(df_clean)
    print("=" * 70)

```

```

print(f"  {'TOTAL DIHAPUS':<45} → {total_removed:>4} baris")
print(f"  {'TOTAL TERSISA':<45} → {len(df_clean)>4} baris")
print("=" * 70)
return df_clean

def save_per_aspek(df: pd.DataFrame, output_dir: str, aspek_list=ASPEK_LIST,
    ↪suffix=""):
    """Simpan dataframe per aspek menjadi file Excel terpisah."""
    print(f"Menyimpan file ke {output_dir} ...")
    for aspek in aspek_list:
        sub = df[df["Istilah Aspek"].str.strip().str.lower() == aspek.lower()].
    ↪copy()
        safe_name = aspek.replace(" ", "_").replace("/", "-")
        out_path = os.path.join(output_dir, f"{safe_name}{suffix}.xlsx")
        sub.to_excel(out_path, index=False)
        print(f"  [OK] {aspek:<45} → {len(sub)>4} baris → {out_path}")

```

6.2 ASC_AK

```

[ ]: # [AK] Load data
df_ak = pd.read_excel(FILE_PATH_AK)
print(f"Total baris dimuat : {len(df_ak):,}")
print(f"Kolom           : {list(df_ak.columns)}")
df_ak.head(3)

```

```

Total baris dimuat : 4,658
Kolom              : ['Istilah Aspek', 'Kelas Sentimen', 'Frasa Kunci Penanda
Sentimen', 'Ekspresi Indikator Sentimen', 'Ekspresi Generalisasi', 'Tipe
Komentar', 'Nomor Usulan', 'Dataset']

```

```

[ ]:
      Istilah Aspek Kelas Sentimen   Frasa Kunci Penanda Sentimen \
0  capaian pembelajaran lulusan      positif   dinilai positif melalui frasa
1    indeks prestasi kumulatif      positif   dinilai positif melalui frasa
2    kelulusan tepat waktu          positif   dinilai positif melalui frasa

      Ekspresi Indikator Sentimen   Ekspresi Generalisasi \
0    tersedia (di bagian suplemen)   ketersediaan dokumen
1                                ipk>3.35   nilai akademik tinggi
2  prosentase yang lulus tepat waktu 100% kelulusan sangat tinggi

Tipe Komentar      Nomor Usulan Dataset
0                AK      ASC_AK
1                AK      ASC_AK
2                AK      ASC_AK

```

```
[ ]: # [AK] Lower-case seluruh kolom teks
df_ak = lowercase_text_columns(df_ak)

[ ]: # [AK] Data quality control: Sentimen invalid, Frasa invalid, Aspek di luar
↳daftar
df_ak = drop_invalid_rows(df_ak, label="ASC_AK")
```

```
=====
DATA QUALITY REPORT - ASC_AK
=====

Total baris diperiksa      : 4,658
Total baris invalid        : 189
- Kelas Sentimen invalid   : 167
- Frasa Kunci invalid      : 161
- Istilah Aspek di luar daftar : 46

ASPEK                      | SENT INV | FRASA INV |
TOTAL INV
-----
-----
capaian pembelajaran lulusan      |      10 |      10 |
10
indeks prestasi kumulatif         |      18 |      19 |
19
kelulusan tepat waktu             |      17 |      18 |
18
masa tunggu lulusan              |      19 |      18 |
20
pelacakan dan perekaman data lulusan |      18 |      19 |
20
prestasi mahasiswa               |      18 |      17 |
19
karya dengan hak kekayaan intelektual |      14 |      14 |
17
kesesuaian bidang kerja           |      19 |      19 |
20
=====
```

Sample (up to 10) invalid rows:

	Istilah Aspek	Frasa Kunci Penanda
Kelas Sentimen		
Sentimen	Ekspresi Indikator Sentimen	
201	indeks prestasi kumulatif	
positif		ipk terendah 2.77 dan tertinggi
3.90	rentang nilai akademik yang baik	
202	kelulusan tepat waktu	
netral	data kelulusan tepat waktu, yang untuk tahun 2017/2018 sebanyak 10	
orang.	jumlah kelulusan tepat waktu	

204 pelacakan dan perekaman data lulusan
positif data hasil survei ke pengguna
lulusan hasil survei dari dunia kerja
205 prestasi mahasiswa
positif juara 3 kompetisi tingkat nasional smart
inotek prestasi akademik dan non-akademik tinggi
206 karya dengan hak kekayaan intelektual
positif mendapat hki dari hasil penelitian berjumlah 7
buah jumlah hak kekayaan intelektual yang banyak
207 kesesuaian bidang kerja
positif perusahaan yang menjadi tempat
kerja penyerapan lulusan di dunia industri
352 keterlaksanaan pemenuhan cpl
netral diidentifikasi sebagai penanda netral melalui
frasa keterlaksanaan pemenuhan cpl
361 tidak relevan
tidak ditemukan tidak
ditemukan tidak ditemukan
362 positif dinilai
positif melalui frasa telah terlaksana
dengan sangat baik pelaksanaan akademik sangat baik
363 netral diidentifikasi sebagai penanda
netral melalui frasa rata-rata kelulusan
mahasiswa tepat waktu tingkat kelulusan tepat waktu

[OK] Dihapus 189 baris invalid. Tersisa: 4,469 baris.

```
[ ]: # [AK] Distribusi setelah quality control
dist_ak_filtered = build_distribution(df_ak)
print_summary_table(dist_ak_filtered, title="ASC_AK - SEBELUM PENGHAPUSAN_
↳DUPLIKAT")
```

```
=====
=====
ASC_AK - SEBELUM PENGHAPUSAN DUPLIKAT
=====
=====
ASPEK | POSITIF | NEGATIF | NETRAL |
TIDAK RELEVAN | TOTAL
-----+-----+-----+-----+
-----+-----
capaian pembelajaran lulusan | 239 | 126 | 90 |
113 | 568
indeks prestasi kumulatif | 238 | 59 | 185 |
76 | 558
kelulusan tepat waktu | 138 | 128 | 206 |
86 | 558
masa tunggu lulusan | 162 | 84 | 193 |
```

117	556			
pelacakan dan perekaman data lulusan		166	135	145
111	557			
prestasi mahasiswa		279	98	71
109	557			
karya dengan hak kekayaan intelektual		159	121	80
200	560			
kesesuaian bidang kerja		225	81	126
123	555			
=====				
=====				
TOTAL		1,606	832	1,096
935	4,469			
=====				
=====				

```
[ ]: # [AK] Cek duplikat per aspek
report_duplicates(df_ak, label="ASC_AK")
```

=====	
LAPORAN DUPLIKAT PER ASPEK - ASC_AK	
=====	
capaian pembelajaran lulusan	→ 177 duplikat
indeks prestasi kumulatif	→ 317 duplikat
kelulusan tepat waktu	→ 232 duplikat
masa tunggu lulusan	→ 275 duplikat
pelacakan dan perekaman data lulusan	→ 184 duplikat
prestasi mahasiswa	→ 188 duplikat
karya dengan hak kekayaan intelektual	→ 260 duplikat
kesesuaian bidang kerja	→ 238 duplikat
=====	
TOTAL	→ 1871 duplikat
=====	

```
[ ]: # [AK] Hapus duplikat per aspek
df_ak_clean = remove_duplicates(df_ak, label="ASC_AK")
```

=====	
PENGHAPUSAN DUPLIKAT PER ASPEK - ASC_AK	
=====	
capaian pembelajaran lulusan	→ dihapus 177 baris (sebelum: 568, sesudah: 391)
indeks prestasi kumulatif	→ dihapus 317 baris (sebelum: 558, sesudah: 241)
kelulusan tepat waktu	→ dihapus 232 baris (sebelum: 558, sesudah: 326)
masa tunggu lulusan	→ dihapus 275 baris (sebelum: 556, sesudah: 281)
pelacakan dan perekaman data lulusan	→ dihapus 184 baris (sebelum: 556, sesudah: 281)

```

557, sesudah: 373)
    prestasi mahasiswa                    → dihapus 188 baris (sebelum:
557, sesudah: 369)
    karya dengan hak kekayaan intelektual → dihapus 260 baris (sebelum:
560, sesudah: 300)
    kesesuaian bidang kerja              → dihapus 238 baris (sebelum:
555, sesudah: 317)
=====
TOTAL DIHAPUS                    → 1871 baris
TOTAL TERSISA                    → 2598 baris
=====

```

```

[ ]: # [AK] Distribusi setelah penghapusan duplikat
dist_ak_clean = build_distribution(df_ak_clean)
print_summary_table(dist_ak_clean, title="ASC_AK - SESUDAH PENGHAPUSAN_
↳DUPLIKAT")

```

```

=====
ASC_AK - SESUDAH PENGHAPUSAN DUPLIKAT
=====
=====
ASPEK                                | POSITIF | NEGATIF | NETRAL |
TIDAK RELEVAN | TOTAL
-----+-----+-----+-----+
-----+-----
    capaian pembelajaran lulusan    |    210 |    114 |    66 |
1 |    391
    indeks prestasi kumulatif      |    113 |    50 |    77 |
1 |    241
    kelulusan tepat waktu          |    97 |   104 |   124 |
1 |    326
    masa tunggu lulusan            |    92 |    74 |   114 |
1 |    281
    pelacakan dan perekaman data lulusan |   142 |   124 |   106 |
1 |    373
    prestasi mahasiswa              |   229 |    95 |    44 |
1 |    369
    karya dengan hak kekayaan intelektual |   118 |   117 |    63 |
2 |    300
    kesesuaian bidang kerja        |   164 |    73 |    79 |
1 |    317
=====
TOTAL                                | 1,165 |    751 |    673 |
9 | 2,598
=====
=====

```

6.3 ASC_AL

```
[ ]: # [AL] Load data
df_al = pd.read_excel(FILE_PATH_AL)
print(f"Total baris dimuat : {len(df_al):,}")
print(f"Kolom                : {list(df_al.columns)}")
df_al.head(3)
```

```
Total baris dimuat : 2,277
Kolom                : ['Istilah Aspek', 'Kelas Sentimen', 'Frasa Kunci Penanda
Sentimen', 'Ekspresi Indikator Sentimen', 'Ekspresi Generalisasi', 'Tipe
Komentar', 'Nomor Usulan', 'Dataset']
```

```
[ ]:
      Istilah Aspek Kelas Sentimen \
0  capaian pembelajaran lulusan      positif
1    indeks prestasi kumulatif        netral
2    kelulusan tepat waktu           netral

      Frasa Kunci Penanda Sentimen \
0                               dinilai positif melalui frasa
1  diidentifikasi sebagai penanda netral melalui ...
2  diidentifikasi sebagai penanda netral melalui ...

      Ekspresi Indikator Sentimen \
0  "terlaksananya pemenuhan capaian pembelajaran ..."
1  "instrumen penilaian rata-rata ipk"
2  "laporan pbm - laporan yudisium"

      Ekspresi Generalisasi Tipe Komentar \
0  pemenuhan luaran dan capaian pembelajaran          AL
1  ketersediaan instrumen penilaian IPK                AL
2  dokumentasi status kelulusan tepat waktu           AL

      Nomor Usulan Dataset
0  ASC_AL
1  ASC_AL
2  ASC_AL
```

```
[ ]: # [AL] Lower-case seluruh kolom teks
df_al = lowercase_text_columns(df_al)
```

```
[ ]: # [AL] Data quality control: Sentimen invalid, Frasa invalid, Aspek di luar
      ↳daftar
df_al = drop_invalid_rows(df_al, label="ASC_AL")
```

```
=====
DATA QUALITY REPORT - ASC_AL
=====
Total baris diperiksa          : 2,277
```

Total baris invalid : 107
 - Kelas Sentimen invalid : 96
 - Frasa Kunci invalid : 96
 - Istilah Aspek di luar daftar : 26

ASPEK	SENT INV	FRASA INV
TOTAL INV		

capaian pembelajaran lulusan	6	8
8		
indeks prestasi kumulatif	11	10
12		
kelulusan tepat waktu	9	8
9		
masa tunggu lulusan	11	11
12		
pelacakan dan perekaman data lulusan	10	11
11		
prestasi mahasiswa	9	9
10		
karya dengan hak kekayaan intelektual	9	8
9		
kesesuaian bidang kerja	9	9
10		
=====		

Sample (up to 10) invalid rows:

	Istilah Aspek		Frasa Kunci Penanda Sentimen
Kelas Sentimen			
Ekspresi Indikator Sentimen			
224	pemenuhan capaian pembelajaran lulusan		
positif		dinilai positif melalui frasa	
	disertai bukti yang sah dan sangat lengkap		
458	indeks prestasi kumulatif		dinilai
positif melalui frasa		"sudah memenuhi ikt pnj	
2022"		sudah memenuhi ikt pnj 2022	
460	masa tunggu lulusan		dinilai
positif melalui frasa		"waktu tunggu masa bekerja cukup	
baik"		waktu tunggu masa bekerja cukup baik	
461	pelacakan dan perekaman data lulusan		dinilai
positif melalui frasa		"data keberhasilan lulusan program	
studi"		data keberhasilan lulusan program studi	
462	prestasi mahasiswa		dinilai
positif melalui frasa		"mendapatkan prestasi di bidang	
akademik"		mendapatkan prestasi di bidang akademik	
463	karya dengan hak kekayaan intelektual	diidentifikasi sebagai penanda	
netral melalui frasa		"hki dosen: hki	

mahasiswa:"		hki dosen: hki mahasiswa:
464	kesesuaian bidang kerja	dinilai
negatif melalui frasa		"makin
menurun"		makin menurun
521	capaian pembelajaran lulusan	dinilai
negatif melalui frasa	belum tersedia informasi mengenai instrumen hasil	
pemenuhan cpl	belum tersedia informasi mengenai instrumen hasil pemenuhan cpl	
522	indeks prestasi kumulatif	dinilai
positif melalui frasa		ipk sudah
melampaui iku		ipk sudah melampaui iku
523	kelulusan tepat waktu	diidentifikasi sebagai penanda
netral melalui frasa		kelulusan tepat waktu = 52
mahasiswa		kelulusan tepat waktu = 52 mahasiswa

[OK] Dihapus 107 baris invalid. Tersisa: 2,170 baris.

```
[ ]: # [AL] Distribusi setelah quality control
dist_al_filtered = build_distribution(df_al)
print_summary_table(dist_al_filtered, title="ASC_AL - SEBELUM PENGHAPUSAN_
↳DUPLIKAT")
```

=====								
ASC_AL - SEBELUM PENGHAPUSAN DUPLIKAT								
=====								
=====								
ASPEK			POSITIF		NEGATIF		NETRAL	
TIDAK RELEVAN		TOTAL						
-----+-----		+	-----+-----		+	-----+-----		+
-----+-----								
capaian pembelajaran lulusan			140		49		33	
52		274						
indeks prestasi kumulatif			129		21		93	
26		269						
kelulusan tepat waktu			80		59		104	
29		272						
masa tunggu lulusan			89		30		99	
51		269						
pelacakan dan perekaman data lulusan			103		41		82	
45		271						
prestasi mahasiswa			187		22		17	
45		271						
karya dengan hak kekayaan intelektual			115		28		38	
92		273						
kesesuaian bidang kerja			137		26		67	
41		271						
=====								
=====								

TOTAL		980		276		533	
381		2,170					

```
=====
```

```
=====
```

```
[ ]: # [AL] Cek duplikat per aspek
report_duplicates(df_al, label="ASC_AL")
```

```
=====
```

```
LAPORAN DUPLIKAT PER ASPEK - ASC_AL
```

```
=====
```

capaian pembelajaran lulusan	→	74 duplikat
indeks prestasi kumulatif	→	134 duplikat
kelulusan tepat waktu	→	94 duplikat
masa tunggu lulusan	→	114 duplikat
pelacakan dan perekaman data lulusan	→	68 duplikat
prestasi mahasiswa	→	74 duplikat
karya dengan hak kekayaan intelektual	→	121 duplikat
kesesuaian bidang kerja	→	85 duplikat

```
=====
```

TOTAL	→	764 duplikat
-------	---	--------------

```
=====
```

```
[ ]: # [AL] Hapus duplikat per aspek
df_al_clean = remove_duplicates(df_al, label="ASC_AL")
```

```
=====
```

```
PENGHAPUSAN DUPLIKAT PER ASPEK - ASC_AL
```

```
=====
```

capaian pembelajaran lulusan	→ dihapus	74 baris	(sebelum: 274, sesudah: 200)
indeks prestasi kumulatif	→ dihapus	134 baris	(sebelum: 269, sesudah: 135)
kelulusan tepat waktu	→ dihapus	94 baris	(sebelum: 272, sesudah: 178)
masa tunggu lulusan	→ dihapus	114 baris	(sebelum: 269, sesudah: 155)
pelacakan dan perekaman data lulusan	→ dihapus	68 baris	(sebelum: 271, sesudah: 203)
prestasi mahasiswa	→ dihapus	74 baris	(sebelum: 271, sesudah: 197)
karya dengan hak kekayaan intelektual	→ dihapus	121 baris	(sebelum: 273, sesudah: 152)
kesesuaian bidang kerja	→ dihapus	85 baris	(sebelum: 271, sesudah: 186)

```
=====
```

TOTAL DIHAPUS	→	764 baris
TOTAL TERSISA	→	1406 baris

```
=====
```

```
[ ]: # [AL] Distribusi setelah penghapusan duplikat
dist_al_clean = build_distribution(df_al_clean)
print_summary_table(dist_al_clean, title="ASC_AL - SESUDAH PENGHAPUSAN_
↳DUPLIKAT")
```

```
=====
=====
ASC_AL - SESUDAH PENGHAPUSAN DUPLIKAT
=====
=====
```

ASPEK		POSITIF	NEGATIF	NETRAL
TIDAK RELEVAN	TOTAL			
-----+-----+-----+-----+-----				
-----+-----				
capaian pembelajaran lulusan		124	48	27
1 200				
indeks prestasi kumulatif		62	20	52
1 135				
kelulusan tepat waktu		55	51	71
1 178				
masa tunggu lulusan		58	28	68
1 155				
pelacakan dan perekaman data lulusan		90	39	73
1 203				
prestasi mahasiswa		164	22	10
1 197				
karya dengan hak kekayaan intelektual		101	26	24
1 152				
kesesuaian bidang kerja		112	24	49
1 186				
=====				
=====				
TOTAL		766	258	374
8 1,406				
=====				
=====				

```
[ ]: # (cel ini sengaja dikosongkan setelah restrukturisasi blok AL)
```

6.4 Kombinasi ASC_AK + ASC_AL → ASC

```
[ ]: # Gabungkan data sentimen AK (bersih) + AL (bersih)
df_combined = pd.concat([df_ak_clean, df_al_clean], ignore_index=True)
print(f"Total baris gabungan: {len(df_combined):,} "
      f"(AK: {len(df_ak_clean):,} + AL: {len(df_al_clean):,})")

dist_combined_raw = build_distribution(df_combined)
print_summary_table(dist_combined_raw, title="ASC GABUNGAN - SETELAH KOMBINASI")
```

Total baris gabungan: 4,004 (AK: 2,598 + AL: 1,406)

```
=====
=====
ASC GABUNGAN - SETELAH KOMBINASI
=====
=====
```

ASPEK		POSITIF	NEGATIF	NETRAL
TIDAK RELEVAN	TOTAL			
-----+-----+-----+-----+-----				
-----+-----				
capaian pembelajaran lulusan		334	162	93
2 591				
indeks prestasi kumulatif		175	70	129
2 376				
kelulusan tepat waktu		152	155	195
2 504				
masa tunggu lulusan		150	102	182
2 436				
pelacakan dan perekaman data lulusan		232	163	179
2 576				
prestasi mahasiswa		393	117	54
2 566				
karya dengan hak kekayaan intelektual		219	143	87
3 452				
kesesuaian bidang kerja		276	97	128
2 503				
=====				
=====				
TOTAL		1,931	1,009	1,047
17 4,004				
=====				
=====				

```
[ ]: # Simpan hasil gabungan ke folder ASC/Distribution
save_per_aspek(df_combined, OUTPUT_DIR_COMBINED)
```

Menyimpan file ke /content/drive/MyDrive/Thesaurus/ASC/Distribution ...
 [OK] capaian pembelajaran lulusan → 591 baris → /content/
 drive/
 MyDrive/Thesaurus/ASC/Distribution/capaian_pembelajaran_lulusan.xlsx

```

[OK] indeks prestasi kumulatif → 376 baris → /content/
↳drive/
MyDrive/Thesaurus/ASC/Distribution/indeks_prestasi_kumulatif.xlsx
[OK] kelulusan tepat waktu → 504 baris →
/content/drive/MyDrive/Thesaurus/ASC/Distribution/kelulusan_tepat_waktu.xlsx
[OK] masa tunggu lulusan → 436 baris →
/content/drive/MyDrive/Thesaurus/ASC/Distribution/masa_tunggu_lulusan.xlsx
[OK] pelacakan dan perekaman data lulusan → 576 baris → /content/
↳drive/
MyDrive/Thesaurus/ASC/Distribution/pelacakan_dan_perekaman_data_lulusan.xlsx
[OK] prestasi mahasiswa → 566 baris →
/content/drive/MyDrive/Thesaurus/ASC/Distribution/prestasi_mahasiswa.xlsx
[OK] karya dengan hak kekayaan intelektual → 452 baris → /content/
↳drive/
MyDrive/Thesaurus/ASC/Distribution/karya_dengan_hak_kekayaan_intelektual.xlsx
[OK] kesesuaian bidang kerja → 503 baris → /content/
↳drive/
MyDrive/Thesaurus/ASC/Distribution/kesesuaian_bidang_kerja.xlsx

```

```

[ ]: # Cek duplikat lintas-sumber pada data gabungan (AK & AL bisa beririsan)
report_duplicates(df_combined, label="ASC GABUNGAN")

```

```

=====
LAPORAN DUPLIKAT PER ASPEK - ASC GABUNGAN
=====
capaian pembelajaran lulusan → 35 duplikat
indeks prestasi kumulatif → 45 duplikat
kelulusan tepat waktu → 56 duplikat
masa tunggu lulusan → 54 duplikat
pelacakan dan perekaman data lulusan → 40 duplikat
prestasi mahasiswa → 37 duplikat
karya dengan hak kekayaan intelektual → 27 duplikat
kesesuaian bidang kerja → 45 duplikat
=====
TOTAL → 339 duplikat
=====

```

```

[ ]: # Hapus duplikat pada data gabungan & simpan hasil bersih
df_combined_clean = remove_duplicates(df_combined, label="ASC GABUNGAN")
save_per_aspek(df_combined_clean, OUTPUT_DIR_COMBINED_CLEAN, suffix="_clean")

dist_combined_clean = build_distribution(df_combined_clean)
print_summary_table(dist_combined_clean, title="ASC GABUNGAN - SESUDAH_
↳PENGHAPUSAN DUPLIKAT")

```

```

=====
PENGHAPUSAN DUPLIKAT PER ASPEK - ASC GABUNGAN
=====

```

capaian pembelajaran lulusan	→ dihapus	35 baris	(sebelum: 591, sesudah: 556)
indeks prestasi kumulatif	→ dihapus	45 baris	(sebelum: 376, sesudah: 331)
kelulusan tepat waktu	→ dihapus	56 baris	(sebelum: 504, sesudah: 448)
masa tunggu lulusan	→ dihapus	54 baris	(sebelum: 436, sesudah: 382)
pelacakan dan perekaman data lulusan	→ dihapus	40 baris	(sebelum: 576, sesudah: 536)
prestasi mahasiswa	→ dihapus	37 baris	(sebelum: 566, sesudah: 529)
karya dengan hak kekayaan intelektual	→ dihapus	27 baris	(sebelum: 452, sesudah: 425)
kesesuaian bidang kerja	→ dihapus	45 baris	(sebelum: 503, sesudah: 458)

```
=====
TOTAL DIHAPUS                → 339 baris
TOTAL TERSISA                 → 3665 baris
=====
```

Menyimpan file ke /content/drive/MyDrive/Thesaurus/ASC/Distribution_clean

...

```
[OK] capaian pembelajaran lulusan          → 556 baris → /content/
└drive/
MyDrive/Thesaurus/ASC/Distribution_clean/capaian_pembelajaran_lulusan_clean.xlsx
[OK] indeks prestasi kumulatif             → 331 baris → /content/
└drive/
MyDrive/Thesaurus/ASC/Distribution_clean/indeks_prestasi_kumulatif_clean.xlsx
[OK] kelulusan tepat waktu                 → 448 baris → /content/
└drive/
MyDrive/Thesaurus/ASC/Distribution_clean/kelulusan_tepat_waktu_clean.xlsx
[OK] masa tunggu lulusan                   → 382 baris → /content/
└drive/
MyDrive/Thesaurus/ASC/Distribution_clean/masa_tunggu_lulusan_clean.xlsx
[OK] pelacakan dan perekaman data lulusan → 536 baris → /content/
└drive/
MyDrive/Thesaurus/ASC/Distribution_clean/pelacakan_dan_perekaman_data_lulusan_clean.xlsx
[OK] prestasi mahasiswa                   → 529 baris → /content/
└drive/
MyDrive/Thesaurus/ASC/Distribution_clean/prestasi_mahasiswa_clean.xlsx
[OK] karya dengan hak kekayaan intelektual → 425 baris → /content/
└drive/
MyDrive/Thesaurus/ASC/Distribution_clean/karya_dengan_hak_kekayaan_intelektua
l_clean.xlsx
```

```

[OK] kesesuaian bidang kerja → 458 baris → /content/
drive/
MyDrive/Thesaurus/ASC/Distribution_clean/kesesuaian_bidang_kerja_clean.xlsx
=====
ASC GABUNGAN - SESUDAH PENGHAPUSAN DUPLIKAT
=====
=====
ASPEK | POSITIF | NEGATIF | NETRAL |
TIDAK RELEVAN | TOTAL
-----+-----+-----+-----+-----
capaian pembelajaran lulusan | 310 | 158 | 87 |
1 | 556
indeks prestasi kumulatif | 152 | 66 | 112 |
1 | 331
kelulusan tepat waktu | 135 | 140 | 172 |
1 | 448
masa tunggu lulusan | 125 | 97 | 159 |
1 | 382
pelacakan dan perekaman data lulusan | 210 | 159 | 166 |
1 | 536
prestasi mahasiswa | 366 | 112 | 50 |
1 | 529
karya dengan hak kekayaan intelektual | 201 | 140 | 82 |
2 | 425
kesesuaian bidang kerja | 247 | 96 | 114 |
1 | 458
=====
TOTAL | 1,746 | 968 | 942 |
9 | 3,665
=====
=====

```

6.5 Sesi Manual

```

[ ]: # [OPTIONAL] Muat ulang data ASC final dari folder & cek distribusi
def load_combined_from_folder(folder: str, aspek_list=ASPEK_LIST,
    ↪suffix="_filtered"):
    parts = []
    for aspek in aspek_list:
        safe_name = aspek.replace(" ", "_").replace("/", "-")
        path = os.path.join(folder, f"{safe_name}{suffix}.xlsx")
        parts.append(pd.read_excel(path))
    return pd.concat(parts, ignore_index=True)

```

```
df_verify = load_combined_from_folder(OUTPUT_DIR_COMBINED_CLEAN)
print(f"Total baris dimuat dari folder : {len(df_verify):,}")

dist_verify = build_distribution(df_verify)
print_summary_table(dist_verify, title="VERIFIKASI - ASC/Distribution_clean")
```

Total baris dimuat dari folder : 1,996

VERIFIKASI - ASC/Distribution_clean				
ASPEK		POSITIF	NEGATIF	NETRAL
TIDAK RELEVAN	TOTAL			
capaian pembelajaran lulusan		208	119	52
0 379				
indeks prestasi kumulatif		56	36	47
0 139				
kelulusan tepat waktu		85	97	87
0 269				
masa tunggu lulusan		58	76	54
0 188				
pelacakan dan perekaman data lulusan		108	112	95
0 315				
prestasi mahasiswa		155	62	34
0 251				
karya dengan hak kekayaan intelektual		91	71	43
0 205				
kesesuaian bidang kerja		131	54	65
0 250				
TOTAL		892	627	477
0 1,996				

7 Tahap Few-Shot Fine-Tuning - Baseline (IndoBERT)

Tahap ini melatih model IndoBERT (indobenchmark/indobert-base-p2) untuk ACD dan ASC di bawah kondisi data terbatas (few-shot), tanpa augmentasi apa pun - hanya mengandalkan class-weighted cross-entropy loss untuk menangani ketidakseimbangan kelas. Data dibagi 80/20 secara multilabel-stratified dengan minimal dua contoh uji per kelas per aspek, lalu diambil hingga lima belas komentar berlabel per kelas aspek untuk pelatihan. Kedelapan aspek dilatih melalui head klasifikasi terpisah di atas encoder IndoBERT yang sama. Hasil (prediksi, metrik precision/recall/F1, dan confusion matrix) disimpan untuk dibandingkan dengan konfigurasi tesaurus pada tahap berikutnya.

7.1 Install & Import

```
[ ]: !pip install -q transformers datasets scikit-learn pandas openpyxl matplotlib_↵
↵seaborn tqdm iterative-stratification
```

```
[ ]: import os, re, random, warnings, gc
import numpy as np
import pandas as pd
from copy import deepcopy
from tqdm.auto import tqdm

import torch
import torch.nn as nn
from torch.optim import AdamW
from torch.utils.data import Dataset, DataLoader

from transformers import (
    BertTokenizer, BertModel,
    set_seed,
)
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import (
    precision_recall_fscore_support, f1_score,
    confusion_matrix, accuracy_score,
)
import matplotlib.pyplot as plt
import seaborn as sns

warnings.filterwarnings('ignore')
sns.set_style('whitegrid')
print('Libraries loaded.')
```

Libraries loaded.

7.2 Configuration

```
[ ]: MODEL_CHECKPOINT = 'indobenchmark/indobert-base-p2'

# Hyperparameters
LEARNING_RATE = 2e-5
BATCH_SIZE     = 8
DROPOUT        = 0.1
EPOCHS         = 5
WEIGHT_DECAY   = 0.3
MAX_SEQ_LEN    = 512
SEED           = 42

# Few-shot sampling & split
N_PER_CLASS = 15
TEST_FRAC   = 0.20
MIN_TEST_PER_CLASS = 2    # enforce >= this many test examples per class, per
                           ↳ aspect
STRATIFY     = True       # multilabel-stratified split (keeps per-aspect class
                           ↳ mix)

# Labels
ACD_LABEL_MAP = {'relevan': 1, 'tidak relevan': 0}
ACD_LABEL_NAMES = ['tidak_relevan', 'relevan']

ASC_LABEL_MAP = {'positif': 0, 'negatif': 1, 'netral': 2}
ASC_LABEL_NAMES = ['positif', 'negatif', 'netral']

# Aspect columns (8 fixed aspects, '(manual)' gold columns)
ASPECT_COLUMNS = [
    'capaian pembelajaran lulusan (manual)',
    'indeks prestasi kumulatif (manual)',
    'kelulusan tepat waktu (manual)',
    'masa tunggu lulusan (manual)',
    'pelacakan dan perekaman data lulusan (manual)',
    'prestasi mahasiswa (manual)',
    'karya dengan hak kekayaan intelektual (manual) ',
    'kesesuaian bidang kerja (manual)',
]
NUM_ASPECTS = len(ASPECT_COLUMNS)
ASPECT_SHORTS = [c.replace(' (manual)', '').strip() for c in ASPECT_COLUMNS]

# Paths (Colab / Google Drive)
BASE_PATH = '/content/drive/MyDrive'

ACD_EXPERIMENTS = [
```

```

        {'name': 'ACD_AK', 'file': 'Hasil Ground Truth_ACD AK_clean.xlsx', 'task': 'ACD', 'text_col': 'komentar_AK'},
        {'name': 'ACD_AL', 'file': 'Hasil Ground Truth_ACD AL_clean.xlsx', 'task': 'ACD', 'text_col': 'komentar_AL'},
    ]
    ASC_EXPERIMENTS = [
        {'name': 'ASC_AK', 'file': 'Hasil Ground Truth_ASC AK_clean.xlsx', 'task': 'ASC', 'text_col': 'komentar_AK'},
        {'name': 'ASC_AL', 'file': 'Hasil Ground Truth_ASC AL_clean.xlsx', 'task': 'ASC', 'text_col': 'komentar_AL'},
    ]

    print(f'Model      : {MODEL_CHECKPOINT}')
    print(f'LR {LEARNING_RATE} | batch {BATCH_SIZE} | dropout {DROPOUT} | epochs {EPOCHS} | max_len {MAX_SEQ_LEN}')
    print(f'Few-shot      : {N_PER_CLASS} per (aspect,class), NO augmentation')
    print(f'Split         : {int((1-TEST_FRAC)*100)}/{int(TEST_FRAC*100)}, NO validation | min {MIN_TEST_PER_CLASS} test/class/aspect')

```

```

Model      : indobenchmark/indobert-base-p2
LR 2e-05 | batch 8 | dropout 0.1 | epochs 5 | max_len 512
Few-shot   : 15 per (aspect,class), NO augmentation
Split      : 80/20, NO validation | min 2 test/class/aspect

```

7.3 Device & Reproducibility

```

[ ]: def set_all_seeds(seed):
    random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
    set_seed(seed)

set_all_seeds(SEED)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Device:', device)
if torch.cuda.is_available():
    print(' GPU:', torch.cuda.get_device_name(0))

```

```

Device: cuda
GPU: NVIDIA L4

```

7.4 Mount Google Drive

```

[ ]: from google.colab import drive
drive.mount('/content/drive')

```

```

Mounted at /content/drive

```

7.5 Tokenizer

```
[ ]: tokenizer = BertTokenizer.from_pretrained(MODEL_CHECKPOINT)
print('Tokenizer loaded:', MODEL_CHECKPOINT, '| vocab', tokenizer.vocab_size)
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

tokenizer_config.json: 0%| | 0.00/2.00 [00:00<?, ?B/s]

vocab.txt: 0%| | 0.00/229k [00:00<?, ?B/s]

special_tokens_map.json: 0%| | 0.00/112 [00:00<?, ?B/s]

Tokenizer loaded: indobenchmark/indobert-base-p2 | vocab 30521

7.6 Preprocessing (singkatan + typo normalization)

```
[ ]: import re
# Singkatan + typo normalization dictionaries (provided by user).
# NOTE: typo_dict2 arrived truncated at the end ("...",\"amik\": \"amik\", \"} );
# the corrupted trailing fragment was dropped and the dict closed cleanly.

replace_weird_words1 = {
    "bbrp": "beberapa", "bhw": "bahwa", "bk": "bidang_keahlian", "bl": "bulan", "blm":
        ↪ "belum", "bln": "bulan",
    "bnsp": "badan_nasional_sertifikasi_profesi", "bpc":
        ↪ "bidang_pengembang_kompetensi", "bpjs":
        ↪ "badan_penyelenggara_jaminan_sosial", "brp": "berapa",
    "cmpk": "capaian_pembelajaran_mata_kuliah", "cp": "capaian_pembelajaran", "cs":
        ↪ "computer_science", "csd": "center_of_student_development", "dcs":
        ↪ "diploma_computer_science", "dg": "dengan", "dgn": "dengan", "dll":
        ↪ "dan_lain_lain", "dlm": "dalam", "dr": "dari", "dsb": "dan_sebagainya",
    "dtp": "dosen_tetap_penghitung_rasio", "dtpmhs":
        ↪ "dosen_tetap_penghitung_rasio_mahasiswa", "fcns":
        ↪ "foresec_certified_network_security",
    "fgd": "forum_group_discussion", "ft": "fakultas_teknik", "gbr": "gambar",
    "hk": "hak_kekayaan_intelektual", "hkl": "hak_kekayaan_intelektual", "hlm":
        ↪ "halaman", "jl": "jumlah", "jml": "jumlah", "jwd": "junior_web_developer",
    "kbb": "kurikulum_berbasis_kompetensi", "kbbm": "kegiatan_belajar_mengajar", "kklp":
        ↪ "kuliah_kerja_lapangan_pengganti", "kkn": "kuliah_kerja_nyata", "kp":
        ↪ "kerja_praktek", "kpt": "konsultan_pendidikan_tinggi", "krn": "karena",
    "ktw": "kelulusan_tepat_waktu", "lck": "laporan_capaian_kinerja", "lkps":
        ↪ "laporan_kinerja_program_studi", "lm": "layanan_mahasiswa", "lpdp":
        ↪ "lembaga_pengelola_dana_pendidikan", "lpk": "indeks_prestasi_kumulatif",
```

```

"lppm": "lembaga_penelitian_dan_pengabdian_masyarakat", "lsp":
  ↳ "lembaga_sertifikasi_profesi", "lspk":
  ↳ "lembaga_sertifikasi_profesi_komputer", "lt": "information_technology", "lttq":
  ↳ "lembaga_tahfizh_dan_tilawah_alquran",
"mbkm": "merdeka_belajar_kampus_merdeka", "mhn": "mohon", "mhs": "mahasiswa", "mhs": "mahasiswa", "mk": "mata_kuliah", "mkbm": "merdeka_belajar_kampus_merdeka", "ms":
  ↳ "mahasiswa", "ntb": "nusa_tenggara_barat",
"pd": "pada", "pk": "prestasi_kumulatif", "pkm":
  ↳ "program_kreativitas_mahasiswa", "pkmk":
  ↳ "program_kreativitas_mahasiswa_kewirausahaan", "pkmkw":
  ↳ "program_kreativitas_mahasiswa_kewirausahaan", "pkmm":
  ↳ "program_kreativitas_mahasiswa_pengabdian_masyarakat",
"pkmpm": "program_kreativitas_mahasiswa_pengabdian_masyarakat", "pl":
  ↳ "pembelajaran_lanjutan", "pmbb": "program_magang_mahasiswa_bersertifikat",
"pp": "pimpinan_pusat", "ppm": "pusat_penjaminan_mutu", "ps": "program_studi", "psbb":
  ↳ "pembatasan_sosial_berskala_besar",
"pspsk": "pusat_studi_pendidikan_dan_kebijakan", "ptnpts":
  ↳ "perguruan_tinggi_negeri_perguruan_tinggi_swasta", "ptpn":
  ↳ "perkebunan_tanaman_nusantara", "rpl": "rekayasa_perangkat_lunak", "rpm":
  ↳ "rencana_pembelajaran_mingguan", "rpp": "rencana_pelaksanaan_pembelajaran",
"rps": "rencana_pembelajaran_semester", "sbb": "sebagai_berikut", "sbg":
  ↳ "sebagai", "sbgmn": "sebagaimana", "scr": "secara", "sd": "sampai_dengan",
"sdh": "sudah", "sdlc": "system_development_life_cycle", "sdm":
  ↳ "sumber_daya_manusia", "shg": "sehingga", "sk": "surat_keputusan",
"sks": "satuan_kredit_semester", "sn": "standar_nasional", "snpt":
  ↳ "standar_nasional_pendidikan_tinggi", "spp":
  ↳ "surat_perjanjian_pemanfaatan", "str": "sarjana_terapan", "stt":
  ↳ "sekolah_tinggi_teknologi", "tdk": "tidak", "tgl": "tanggal", "th": "tahun", "thn":
  ↳ "tahun",
"thd": "terhadap", "tkj": "teknik_komputer_dan_jaringan", "tkk":
  ↳ "teknik_komputer_kontrol", "tkk": "tingkat", "trk":
  ↳ "teknologi_rekayasa_komputer",
"trkj": "teknologi_rekayasa_komputer", "ts": "tracer_study", "tsb":
  ↳ "tersebut", "tsbt": "tersebut", "ttg": "tentang",
"vm": "visi_misi", "vmst": "visi_misi_tujuan_dan_strategi", "wt":
  ↳ "waktu_tunggu", "prodi": "program_studi", "ik": "ilmu_komputer", "al":
  ↳ "asesmen_lapangan", "ak": "asesmen_kecukupan", "rtm": "papat_tinjauan_manajemen",
"cdc": "career_development_center", "spm": "sistem_penjaminan_mutu",
"phb": "program_hibah_binaan", "pt": "perguruan_tinggi",
"dtps": "dosen_tetap_program_studi", "lrs": "laporan_rencana_studi", "lpks":
  ↳ "lembaga_pengembangan_keterampilan_dan_sertifikasi",
"smk": "sekolah_menengah_kejuruan", "dst": "dan_seterusnya", "smp": "sampai", "lpm":
  ↳ "lembaga_penjaminan_mutu",
"std": "standar", "bpm": "badan_penjaminan_mutu", "bhs": "bahasa",
"kkmk": "kriteria_kinerja_manajemen_kualitas", "pk1":
  ↳ "rpaktek_kerja_lapangan", "psms": "program_studi_manajemen_sistem",

```

```

"kk":"kode_klasifikasi","rtl":"rencana_tindak_lanjut","smt":"semester",
"pbl":"problem_based_learning","rpsm":"rencana_pembelajaran_semester",
"lp":"lembaga_penjaminan","lpd":"laporan_penelitian_dosen","pppm":
↳"pengembangan_pendidikan_mahasiswa",
"bppm":"badan_penelitian_pengabdian_masyarakat","lpppm":
↳"lembaga_penelitian_pengabdian_masyarakat","rpjp":
↳"rencana_pembangunan_jangka_panjang",
"bpp":"biaya_penyelenggaraan_pendidikan","st":"sarjana_teknik","sp":
↳"surat_peringatan",
"dpp":"dosen_pendamping_program","upps":"unit_pengelola_program_studi","pts":
↳"perguruan_tinggi_swasta","ptn":"perguruan_tinggi_negeri","pr":
↳"penelitian_riset","ttd":"tanda_tangan","pllmm":
↳"lembaga_penelitian_dan_pengabdian_masyarakat",
"rp":"rencana_pembelajaran","drpm":"direktorat_riset_pengabdian_masyarakat",
"bph":"badan_pengurus_harian","ttp":"tetap","llppm":
↳"lembaga_penelitian_pengabdian_masyarakat",
"cfp":"certified_financial_planner","lppmbppm":
↳"lembaga_penelitian_pengabdian_masyarakat_badan_penjaminan_mutu_pendidikan",
"spk":"surat_perjanjian_kerja","pm":"penjaminan_mutu","dptr":
↳"dosen_tetap_penghitung_rasio","kpd":"kepada","dtpt":
↳"dosen_tetap_perguruan_tinggi","bgm":"bagaimana",
"dtp":"dosen_tetap_program","kts":"kurikulum_tingkat_satuan",
"jg":"juga","dgx":"dengan","rpjmn":
↳"rencana_pembangunan_jangka_menengah_nasional","ppkm":
↳"program_pengembangan_keterampilan_mahasiswa",
"dprpm":"direktorat_penelitian_pengabdian_masyarakat","bg":"bagaimana","spmp":
↳"sistem_penjaminan_mutu_pendidikan_tinggi","pnbp":
↳"penerimaan_negara_bukan_pajak",
"wkt":"waktu","csr":"corporate_social_responsibility",
"kkp":"kualitas_kinerja_program","pkmdtpr":
↳"program_kreativitas_mahasiswa_dosen_tetap_penghitung_rasio",
"lppp":"lembaga_penelitian_pengabdian_masyarakat",
"dtr":"dosen_tetap_penghitung_rasio","ln":"luar_negeri","tmps":
↳"tim_penjaminan_mutu_program_studi","tpmj":"tim_penjaminan_mutu_jurusan",
"pmk":"penjaminan_mutu_kinerja","bkn":"bukan","pkms":
↳"program_kreativitas_mahasiswa_sosial","skripsita":"skripsi_tugas_akhir",
"pstp":"program_studi_teknik_pertambangan","ppkmdm":
↳"program_pengembangan_keterampilan_mahasiswa_dan_dosen",
"plp":"program_latihan_profesi","skl":"surat_keterangan_lulus",
"cplcpmk":
↳"capaian_pembelajaran_lulusan_dan_capaian_pembelajaran_mata_kuliah","cpmk":
↳"capaian_pembelajaran_mata_kuliah","rkps":
↳"rencana_kegiatan_pembelajaran_semester",
"sbgmn":"sebagaimana","jmlh":"jumlah","sms":"semester","lplppm":
↳"lembaga_penelitian_pengabdian_masyarakat","pstg":
↳"program_studi_teknik_geologi","pstk":"program_studi_teknik_kimia",

```

```

"skp": "surat_kerja_praktek", "pblpbm":
  ↳ "problem_based_learning_project_based_learning", "prn":
  ↳ "program_riset_nasional", "ppg": "program_profesi_guru", "psbm":
  ↳ "program_studi_bisnis_dan_manajemen",
"lpj": "laporan_pertanggungjawaban", "kkt": "kualitas_kinerja_tim",
"pjj": "pendidikan_jarak_jauh", "pmp": "pemetaan_mutu_pendidikan", "bpmp":
  ↳ "balai_penjaminan_mutu_pendidikan", "bppmp":
  ↳ "balai_penjaminan_mutu_pendidikan", "bdp": "balai_guru_penggerak",
"bkd": "beban_kerja_dosen", "bkk": "bursa_kerja_khusus", "bmw":
  ↳ "belajar_melanjut_wirausaha", "bnpb":
  ↳ "badan_nasional_penanggulangan_bencana", "bnpp":
  ↳ "badan_nasional_pengelola_perbatasan", "bnpt":
  ↳ "badan_nasional_penanggulangan_terorisme",
"bp": "bimbingan_penyuluhan", "bpk": "badan_pengurus_keuangan", "bpl":
  ↳ "badang_pengelola_latihan", "bssn": "badan_siber_dan_sandi_negara",
"cpns": "calon_pegawai_negeri_sipil", "crf":
  ↳ "center_for_research_and_facilitation", "crm":
  ↳ "sistem_manajemen_hubungan_pelanggan", "csrpkbl":
  ↳ "corporate_social_responsibility_program_kemitraan_bina_lingkungan", "pkbl":
  ↳ "program_kemitraan_bina_lingkungan",
"ct": "computational_thinking", "ctf": "career_and_technology_foundations", "cv":
  ↳ "curriculum_vitae",
"ddtpr": "dosen_tetap_penghitung_rasio", "dkk": "dan_kawan_kawan", "dn": "dan", "dng":
  ↳ "dengan", "dpkm": "direktorat_pengabdian_masyarakat", "dpl":
  ↳ "dosen_pembimbing_lapangan", "dpm": "dewan_perwakilan_mahasiswa", "dppm":
  ↳ "direktorat_penelitian_pengabdian_masyarakat",
"dpr": "laporan_kemajuan_gelar", "dpt": "daftar_pemilih_tetap",
"hki": "hak_kekayaan_intelektual", "hku": "hak_kekayaan_usaha",
"iku": "indikator_kinerja_utama", "ikt": "indikator_kinerja_tambahan", "ikuikt":
  ↳ "indikator_kinerja_utama_indikator_kinerja_tambahan", "cbr":
  ↳ "community_based_research", "cbr": "computer_based_test", "cl":
  ↳ "collaborative_learning",
}

replace_weird_words2 = {
"tpg": "tunjangan_profesi_guru", "dtprmhs":
  ↳ "dosen_tetap_penghitung_rasio_mahasiswa", "mhs": "mahasiswa",
"upps": "unit_pengelola_program_studi", "spm":
  ↳ "sistem_penjaminan_mutu_internal", "renstra": "rencana_strategis",
"menemu": "menemukan", "isbn": "international_standard_book_number", "prosentase":
  ↳ "presentase", "yg": "yang", "haki": "hak_atas_kekayaan_intelektual",
"led": "laporan_evaluasi_diri", "danatau": "dan_atau", "peneltian":
  ↳ "penelitian", "utk": "untuk", "ta": "tugas_akhir", "dharma": "tridharma", "amik":
  ↳ "akademi_manajemen_informatika_dan_komputer",

```

```

"pgri":"persatuan_guru_republik_indonesia","umkm":
  ↳"usaha_mikro_kecil_menengah","abdimas":"pengabdian_masyarakat","uppsps":
  ↳"unit_pengelola_program_studi",
"berdasar":"berdasarkan","ti":"teknologi_informasi","dtprmahasiswa":
  ↳"dosen_tetap_penghitung_rasio_mahasiswa","ppepp":
  ↳"penetapan_pelaksanaan_evaluasi_pengendalian_dan_peningkatan",
"si":"sitem_informasi","kaprodi":"kepala_program_studi","dipa":
  ↳"daftar_isian_pelaksanaan_anggaran","itb":"institut_teknologi_bandung",
"kebijakanstandar":"kebijakan_standar","renop":
  ↳"rencana_operasional","lemabagaunit":"lembaga_unit","akakom":
  ↳"akademi_aplikasi_komputer",
"permendikbud":"peraturan_menteri_pendidikan",
"standard":"standar","simlitabmas":
  ↳"sistem_informasi_penelitian_dan_pengabdian_masyarakat","pob":
  ↳"pedoman_operasional_baku",
"penelitan":"penelitian","fti":"fakultas_teknologi_industri","puslit":
  ↳"pusat_penelitian","ri":"republik_indonesia","dosenmahasiswa":
  ↳"dosen_mahasiswa",
"kkni":"kerangka_kualifikasi_nasional_indonesia",
"kkn":"kuliah_kerja_nyata","pkmgft":
  ↳"program_kreativitas_mahasiswa_gagasan_futuristik_tertulis","pkmgfk":
  ↳"program_kreativitas_mahasiswa_gagasan_futuristik_konstruktif",
}

replace_weird_words3 = {
"prodi":"program_studi","ik":"ilmu_komputer","ps":"program_studi","al":
  ↳"asesmen_lapangan","ak":"asesmen_kecukupan","pkm":
  ↳"program_kreativitas_mahasiswa","rtm":"papat_tinjauan_manajemen",
"ppm":"pengabdian_dan_pemberdayaan_masyarakat","cp":"capaian_pembelajaran",
"lkps":"laporan_kinerja_program_studi",
"lrs":"laporan_rencana_studi","kp":"kerja_praktek","lpks":
  ↳"lembaga_pengembangan_keterampilan_dan_sertifikasi",
"smk":"sekolah_menengah_kejuruan","dst":"dan_seterusnya","smp":"sampai","pk":
  ↳"penilaian_kinerja","lpm":"lembaga_penjaminan_mutu",
"kkmk":"kriteria_kinerja_manajemen_kualitas","pkl":
  ↳"rpaktek_kerja_lapangan","psms":"program_studi_manajemen_sistem",
"kk":"kode_klasifikasi","tkk":"tenaga_kependidikan_khusus",
"vmsts":"visi_misi_tujuan_dan_sasaran","kbb":
  ↳"kurikulum_berbasis_kompetensi","pbl":"problem_based_learning","rpsm":
  ↳"rencana_pembelajaran_semester","ft":"fakultas_teknik","ttg":"tentang",
"snpt":"standar_nasional_perguruan_tinggi","lp":"lembaga_penjaminan","lpd":
  ↳"laporan_penelitian_dosen","pppm":"pengembangan_pendidikan_mahasiswa",
"rpjp":"rencana_pembangunan_jangka_panjang",
"st":"sarjana_teknik","sp":"surat_peringatan",
"dpp":"dosen_pendamping_program","pr":"penelitian_riset","ttd":
  ↳"tanda_tangan","pllm":"lembaga_penelitian_dan_pengabdian_masyarakat",

```

```

"rp":"rencana_pembelajaran","drpm":"direktorat_riset_pengabdian_masyarakat",
"ttp":"tetap","llppm":"lembaga_penelitian_pengabdian_masyarakat",
"cfp":"certified_financial_planner","lppmbppm":
  ↳"lembaga_penelitian_pengabdian_masyarakat_badan_penjaminan_mutu_pendidikan",
"spk":"surat_perjanjian_kerja","pm":"penjaminan_mutu","dptr":
  ↳"dosen_tetap_penghitung_rasio","kpd":"kepada","dtpt":
  ↳"dosen_tetap_perguruan_tinggi","bgm":"bagaimana",
"dtp":"dosen_tetap_program","kts":"kurikulum_tingkat_satuan",
"jg":"juga","dgx":"dengan","rpjmn":
  ↳"rencana_pembangunan_jangka_menengah_nasional","ppkm":
  ↳"program_pengembangan_keterampilan_mahasiswa",
"dprpm":"direktorat_penelitian_pengabdian_masyarakat","bg":"bagaimana","spmpt":
  ↳"sistem_penjaminan_mutu_pendidikan_tinggi","pnbp":
  ↳"penerimaan_negara_bukan_pajak",
"wkt":"waktu","csr":"corporate_social_responsibility","fgd":
  ↳"focus_group_discussion",
"kkp":"kualitas_kinerja_program","pkmdtpr":
  ↳"program_kreativitas_mahasiswa_dosen_tetap_penghitung_rasio",
"lppp":"lembaga_penelitian_pengabdian_masyarakat","rpp":
  ↳"rencana_pembelajaran_program",
"dtr":"dosen_tetap_penghitung_rasio","ln":"luar_negeri","tmps":
  ↳"tim_penjaminan_mutu_program_studi","tpmj":"tim_penjaminan_mutu_jurusan",
"pmk":"penjaminan_mutu_kinerja","bkn":"bukan","pkms":
  ↳"program_kreativitas_mahasiswa_sosial",
"pstp":"program_studi_teknik_pertambangan","ppkmdm":
  ↳"program_pengembangan_keterampilan_mahasiswa_dan_dosen",
"plp":"program_latihan_profesi","bnsp":
  ↳"badan_nasional_sertifikasi_profesi","skl":"surat_keterangan_lulus",
"cplcpmk":
  ↳"capaian_pembelajaran_lulusan_dan_capaian_pembelajaran_mata_kuliah","cpmk":
  ↳"capaian_pembelajaran_mata_kuliah","rkps":
  ↳"rencana_kegiatan_pembelajaran_semester","mhsw":"mahasiswa",
"jl":"jalan","sbgmn":"sebagaimana","jmlh":"jumlah","sms":"semester","lplppm":
  ↳"lembaga_penelitian_pengabdian_masyarakat","pstg":
  ↳"program_studi_teknik_geologi","pstk":"program_studi_teknik_kimia",
"kpt":"kualitas_pendidikan_tinggi","skp":"surat_kerja_praktek","rata2":
  ↳"rata_rata","mk":"mata_kuliah","pblpbm":
  ↳"problem_based_learning_project_based_learning","prn":
  ↳"program_riset_nasional","ppg":"program_profesi_guru","psbm":
  ↳"program_studi_bisnis_dan_manajemen",
"lpj":"laporan_pertanggungjawaban","dpts":"dosen_tetap_program_studi","kkt":
  ↳"kualitas_kinerja_tim",
"pjj":"pendidikan_jarak_jauh","pmp":"pemetaan_mutu_pendidikan","ts1":
  ↳"tracer_study_1","ts2":"tracer_study_2","ts3":"tracer_study_3","ts4":
  ↳"tracer_study_4","ts5":"tracer_study_5","ts6":"tracer_study_6",
}

```

```

typo_dict1 = {
"undri": "undur_diri", "detil": "detail", "luarancapaian":
    ↳ "luaran_capaian", "sebesar": "sebesar", "komptetensi": "kompetensi",
"luluipk": "lulus_indeks_prestasi_kumulatif", "perkulihan": "perkuliahan", "dirjen":
    ↳ "direktorat_jenderal", "belmawa": "pembelajaran_dan_kemahasiswaan",
"systemaplikasi": "system_aplikasi", "deskripsidatanarasi":
    ↳ "deskripsi_data_narasi", "informasidata": "informasi_data",
"egovernment": "government", "ipkprestasi":
    ↳ "indeks_prestasi_kumulatif_prestasi", "perkaman": "perekaman", "ydapat": "dapat",
"mahasiswat": "mahasiswa", "lululusan": "lulusan", "senasif":
    ↳ "seminar_nasional_informasi",
"prestai": "prestasi", "standart": "standar", "kessusuaian": "kesesuaian", "kwerja":
    ↳ "kerja", "mendaoat": "mendapat",
"maksimun": "maksimum", "berbading": "berbanding", "kebuman":
    ↳ "kebumen", "diperolehlah": "diperoleh", "tigatahunan": "tiga_tahunan",
"namum": "namun", "detailriil": "detail_ruil", "dtprdosen":
    ↳ "dosen_tetap_penghitung_rasio_dosen", "klarfikiasi": "klarifikasi",
"kepuasanan": "kepuasan", "fluaktif": "fluktuatif", "ketidakkonsitenan":
    ↳ "ketidakkonsistenan", "simkatmawa": "sistem_pemeringkatan_kemahasiswaan",
"ekplisit": "eksplisit", "terlenuhi": "terpenuhi", "diihat":
    ↳ "dilihat", "pujiansebesar": "pujian_sebesar", "pddikti":
    ↳ "pangkalan_data_pendidikan_tinggi",
"atunggu": "tunggu", "masy": "masyarakat", "ditumkan": "ditemukan", "reseponden":
    ↳ "responden", "kesesuaiaan": "kesesuaian",
"denagn": "dengan", "berturutan": "berurutan", "mnempuh": "menempuh", "emenuhan":
    ↳ "pemenuhan", "mahaswasiswa": "mahasiswa",
"melampui": "melampaui", "dilaporakan": "dilaporkan", "laporaan": "laporan", "elama":
    ↳ "lama", "mskor": "skor", "erlacak": "terlacak",
"lulusanlulusan": "lulusan_lulusan", "daapat": "dapat", "reepositori":
    ↳ "repositori", "bidanginfokom": "bidang_infokom",
"jumlh": "jumlah", "menyarakan": "menyarankan", "polinas":
    ↳ "politeknik_informatika_nasional", "inggirs": "inggris",
"datajumlah": "data_jumlah", "aktif": "aktif", "kementrian":
    ↳ "kementerian", "eraticukup": "erat_cukup", "pprodi": "program_studi",
"sertiikat": "sertifikat", "terakahir": "terakhir", "berkerjaberwirausaha":
    ↳ "bekerja_berwirausaha",
"lokalwilayahberwirausaha": "lokal_wilayah_berusaha", "depkominfo":
    ↳ "departemen_komunikasi_dan_informatika",
"linktracer": "link_tracer", "dtidak": "tidak", "berhuubng": "berhubung", "gkerja":
    ↳ "kerja", "subcpmk": "sub_capaian_pembelajaran_mata_kuliah",
"saaat": "saat", "danstr": "dan_sarjana_terapan", "hkiteknologi":
    ↳ "hak_kekayaan_intelektual_teknologi",
"gunaprodukbook": "guna_produk_buku", "kurikulumlink":
    ↳ "kurikulum_link", "ketersedian": "ketersediaan",

```

"sikappengetahuanketerampilan":"sikap_pengetahuan_keterampilan","penilainnya":
 ↳"penilaiannya","lulusantabel":"lulusan_tabel",
 "kerampilan":"keterampilan","tugaskuis":"tugas_kuis","pda":"pada","ket":
 ↳"keterangan","healthkathon":"health_kathon",
 "defenisi":"definisi","perekeman":"perekaman","oprestasi":"prestasi","jkarya":
 ↳"karya","academyc":"academic",
 "dibandikan":"dibandingkan","telahbelum":"telah_belum","sem":"semester","kkni":
 ↳"kerangka_kualifikasi_nasional_indonesia",
 "diharapka":"diharapkan","penuruanan":"penurunan","renatan":"rentan","anatar":
 ↳"antar","dilaksanakantarcer":"dilaksanakan_tracer",
 "dashboard":"dashboard","dalammkurun":"dalam_kurun","kelulusah":
 ↳"kelulusan","dtingkat":"tingkat","sekprodi":"sekretaris_program_studi",
 "sertau":"serta","unipa":"universitas_papua","ppemenuhan":
 ↳"pemenuhan","ditelusur":"ditelusuri",
 "sebelumnyasetiap":"sebelumnya_setiap","diberikansedangkan":
 ↳"diberikan_sedangkan","bbrapa":"beberapa",
 "mahasiswabelum":"mahasiswa_belum","rekrutmen":"rekrutmen","hakidibawah":
 ↳"hak_kekayaan_intelektual_di_bawah",
 "menghasillkan":"menghasilkan","disosialisasiakn":
 ↳"disosialisasikan","nasionalpada":"nasional_pada","infromatika":
 ↳"informatika",
 "ratrata":"rata_rata","dtpsmahasiswa":
 ↳"dosen_tetap_program_studi_mahasiswa","tahunberdasarkan":"tahun_berdasarkan",
 "eventevent":"event_event","erdasarkan":"berdasarkan","nyapelaksanaan":
 ↳"nya_pelaksanaan","temanteman":"teman_teman",
 "studyunikama":"study_unikama","skore":"skor","menjacapi":
 ↳"mencapai","acehsumatera":"aceh_sumatera",
 "internasiona":"internasional","tercpai":"tercapai","renstra":
 ↳"rencana_strategis",
 "renstrarenoprkat":
 ↳"rencana_strategis_rencana_operasional_rencana_kerja_dan_anggaran_tahunan",
 "kebermanfaatanaspek":"kebermanfaatan_aspek","tunggunya":
 ↳"tunggunya","silahkan":"silakan","oresec":"foresec",
 "mutinasionalinternasional":"multinasional_internasional","tercapaikesesuain":
 ↳"tercapai_kesesuaian","pelacakanperekaman":
 ↳"pelacakan_perekaman","matakuliah":"mata_kuliah",
 "stimik":"sekolah_tinggi_ilmu_manajemen_informatika_dan_komputer","sepnop":
 ↳"sepuluh_nopember","ikuikt":
 ↳"indikator_kinerja_utama_indikator_kinerja_tambahan",
 "pelbelajaran":"pembelajaran","dijelaskantetapi":"dijelaskan_tetapi","tingat":
 ↳"tingkat","katagari":"kategori",
 "keberadaanya":"keberadaannya","adalahdiperoleh":"adalah_diperoleh","perta":
 ↳"pertama","dikatatagorikan":"dikatatagorikan",
 "utuk":"untuk","dsetkan":"disertakan","standeriku":
 ↳"standar_indikator_kinerja_utama","prosentase":"persentase",

```

"nasional_sedangkan": "nasional_sedangkan", "tekait": "terkait", "crosscek":
    ↳ "cross_check", "kiurang": "kurang",
"kompetisi_lomba": "kompetisi_lomba", "acc": "accept", "diukur":
    ↳ "diukur", "keterlaksanaan": "keterlaksanaan", "kemabali": "kembali",
"kesesuaiakn": "kesesuaian", "psnya": "program_studi", "utaraperingkat":
    ↳ "utara_peringkat", "gbkperingkat": "gbk_peringkat", "bahw": "bahwa", "rerarta":
    ↳ "rerata",
"mendeskripsi": "mendeskripsikan", "padalink": "pada_link", "tercapaian":
    ↳ "ketercapaian", "pa": "pembimbing_akademik",
"regional_lokal": "regional_lokal", "intektual": "intelektual", "pdki":
    ↳ "pangkalan_data_kekayaan_intelektual", "butirbutir": "butir_butir",
"penejelasan": "penjelasan", "ilkom": "ilmu_komputer", "padangsidimpuan":
    ↳ "padang_sidimpuan", "didata": "di_data", "kehliah": "keahlian", "lulusn":
    ↳ "lulusan",
"finalisdan": "finalis_dan", "usni":
    ↳ "universitas_satya_negara_indonesia", "ekternal": "eksternal", "khusus_karena":
    ↳ "khusus_karena", "runnerup": "runner_up",
"edupreuner": "edupreneur", "sebagainyatidak": "sebagainya_tidak", "led":
    ↳ "laporan_evaluasi_diri", "ledlkps":
    ↳ "laporan_evaluasi_diri_laporan_kinerja_program_studi",
"meghasilkan": "menghasilkan", "pemenuhana": "pemenuhan", "sehingg":
    ↳ "sehingga", "kekesahihan": "kesahihan", "skalan": "skala", "ditetapkansemua":
    ↳ "ditetapkan_semua",
"ckelulusan": "kelulusan", "sebanyak": "sebanyak", "drivecek":
    ↳ "drive_cek", "lolakwilayah": "lolak_wilayah", "adakelulusan":
    ↳ "ada_kelulusan", "proditeknologi": "program_studi_teknologi",
"terpehuni": "terpenuhi", "terpenhi": "terpenuhi", "mahassiwa": "mahasiswa", "tungg":
    ↳ "tunggu", "lokalnasionalinternasional":
    ↳ "lokal_nasional_internasional", "piagamsertifikat": "piagam_sertifikat",
"kelulusa": "kelulusan", "dokumem": "dokumen", "ratarataipk":
    ↳ "rata_rata_indeks_prestasi_kumulatif", "berhki":
    ↳ "yang_memiliki_hak_kekayaan_intelektual", "upi":
    ↳ "universitas_pendidikan_indonesia",
"univ": "universitas", "akhiir": "akhir", "capaiantabel": "capaian_tabel", "peilaian":
    ↳ "penilaian", "pekerjaanketerlaksanaan":
    ↳ "pekerjaan_keterlaksanaan", "juarapesertapanitia": "juara_peserta_panitia",
"aklademiknon": "akademik_non", "kaademik": "akademik", "keseuain":
    ↳ "kesesuaian", "kalrfikasi": "klarifikasi", "kkniobeskkni":
    ↳ "kerangka_kualifikasi_nasional_indonesia_outcome_based_education_standar_kompetensi_kerja_n
"samapai": "sampai", "kesesuaikan": "kesesuaian", "tautautan":
    ↳ "tautan_tautan", "engisi": "mengisi", "akreditas": "akreditasi", "pembelajran":
    ↳ "pembelajaran", "dikur": "diukur", "presntasi": "presentasi", "unuk":
    ↳ "untuk", "sepeti": "seperti",
"pelaksaaan": "pelaksanaan", "pendidika": "pendidikan", "thun": "tahun", "lylusan":
    ↳ "lulusan", "keterlaksanaanya": "keterlaksanaannya",

```

```

"pihakpihak":"pihak_pihak","unitama":
↳"universitas_teknologi_akba_makassar","memilik":"memiliki","unsika":
↳"universitas_singaperbangsa_karawang",
"darisama":"dari_sama","terkahir":"terakhir","studyratarata":
↳"study_rata_rata","almuni":"alumni","angakatan":
↳"angkatan","terlacakterdapat":"terlacak_terdapat",
"aspekapa":"aspek_apa","pertanyaaan":"pertanyaan","infokomps":
↳"infokom_program_studi","mengujur":"mengukur","miyaitu":"mi_yaitu",
"dilaksananakan":"dilaksanakan","tuingkat":"tingkat","nasonal":
↳"nasional","presteasi":"prestasi",
"tetang":"tentang","terjad":"terjadi","pemurunan":"penurunan","menadi":
↳"menjadi","peremkaman":"perekaman","rendahkurang":"rendah_kurang","perekama":
↳"perekaman","llusan":"lulusan","lulusanpada":"lulusan_pada","siginifikan":
↳"signifikan","karenkan":"karenakan","unesa":
↳"universitas_negeri_surabaya","perkerjaan":"pekerjaan","pemutahiran":
↳"pemutakhiran",
"dilakukankonfirmasi":"dilakukan_konfirmasi","mahsa":"masa","rataata":
↳"rata_rata",
"pelaksnaan":"pelaksanaan","tidaada":"tidak_ada","askpek":"aspek","tentan":
↳"tentang","kemanpuan":"kemampuan","bulanadapun":"bulan_adapun","ulusan":
↳"lulusan","infokomkepuasan":"infokom_kepuasan","wilayahnasional":
↳"wilayah_nasional",
"nformatika":"informatika","daengan":"dengan","dilaksakan":
↳"dilaksanakan","kuisoner":"kuesioner","adala":"adalah","masayang":
↳"masa_yang","pemerintahdalam":"pemerintah_dalam","amsa":"masa","tudi":
↳"studi","mendpatkan":"mendapatkan","lkti":
↳"lomba_karya_tulis_ilmiah","psychoferia":"psychofrenia","sigra":
↳"sistem_terintegrasi","kalrifikasin":"klarifikasi","terupsat":
↳"terpusat","diaksesbroken":"diakses_broken","keilmuankeahlian":
↳"keilmuan_keahlian",
"dilingkungan":"di_lingkungan","dimatakuliah":"di_mata_kuliah","tracertubacid":
↳"tracer_ub_ac_id","rataarat":"rata_rata",
"dilakukanperlu":"dilakukan_perlu","emndapat":"mendapat","mendapar":
↳"mendapat","tpi":"tapi","diidentifikasi":"diidentifikasi","pad":"pada","oba":
↳"outcome_based_assessment","acmis":
↳"academic_management_information_system","almni":"alumni","wirausaa":
↳"wirausaha","mekanime":"mekanisme",
"pemenuhancapaianpembelajaranlulusancplrarataaikprestasi":
↳"pemenuhan_capaian_pembelajaran_lulusan_rata_rata_indeks_prestasi_kumulatif_prestasi","reke
↳"rekayasa","komputerbekerja":"komputer_bekerja","akademiknonakademik":
↳"akademik_non_akademik","upp":"unit_pengelola_program_studi",
"keterlaksanaan":"keterlaksanaan","idikator":"indikator","penetatpan":
↳"penetapan","disajika":"disajikan","memperolah":"memperoleh","nadhlatul":
↳"nahdlatul","tachnopreneur":"technopreneur","shasil":"hasil","waktuvisual":
↳"waktu","bersma":"bersama","isuisu":"isu_isu","semester":
↳"semester","sedangkah":"sedangkan","peenuhan":"pemenuhan",

```

```

"careersitsacidtracer": "careersits_ac_id_tracer", "standariku":
    ↳ "standar_indikator_kinerja_utama", "pelaksanaan": "pelaksanaan", "taraget":
    ↳ "target", "melaksanakan": "melaksanakan", "salama": "selama", "kesesuaiannya":
    ↳ "kesesuaiannya", "kesesusian": "kesesuaian",
"icts":
    ↳ "international_conference_on_information_and_communication_technology_and_systems", "sharepe
    ↳ "sharepedia_id",
"kompetisikompetisi": "kompetisi_kompetisi", "wa": "whatsapp", "ebanyak":
    ↳ "sebanyak", "tsnya": "tracer_study_nya",
"lipihki":
    ↳ "lembaga_inovasi_penulisan_ilmiah_dan_hak_kekayaan_intelektual", "standarikuikt":
    ↳ "standar_indikator_kinerja_utama_indikator_kinerja_tambahan", "negaranegara":
    ↳ "negara_negara", "bidangbidang": "bidang_bidang", "teknologiteknologi":
    ↳ "teknologi_teknologi", "kegiatankegiatan": "kegiatan_kegiatan",
}

typo_dict2 = {
"di konfirmasi": "dikonfirmasi", "di validasi": "divalidasi", "ter indentifikasi":
    ↳ "teridentifikasi", "ter sematkan": "tersematkan", "pdik":
    ↳ "program_studi_pendidikan", "ilmu iahnya": "ilmiahnya", "psmtif":
    ↳ "program_studi_magister_teknik_informatika", "perdosentahun":
    ↳ "per_dosen_tahun",
"lacak": "lacakan", "di tandai": "ditandai", "ter pusat": "terpusat", "per baikan":
    ↳ "perbaikan", "ber kualitas": "berkualitas", "mkom": "magister_komputer", "fmipa":
    ↳ "fakultas_matematika_dan_ilmu_pengetahuan_alam", "intelligence":
    ↳ "kecerdasan", "puslit": "pusat_penelitian", "unmuh":
    ↳ "universitas_muhammadiyah", "pbi":
    ↳ "pendidikan_bahasa_inggris", "hilirisasipenerapan":
    ↳ "hilirisasi_penerapan", "matakuliah": "mata_kuliah",
"unsiq": "universitas_sains_al_quran", "uppsprodi":
    ↳ "unit_pengelola_program_studi", "keseuaian": "kesesuaian", "kebijakankebijakan":
    ↳ "kebijakan_kebijakan", "indicator": "indikator", "procedure":
    ↳ "prosedur", "programprogram": "program_program", "ketrampilan":
    ↳ "keterampilan", "pengmas": "pengabdian_masyarakat",
"ugn": "universitas_graha_nusantara", "mtelah": "telah", "kepahahaman":
    ↳ "kepahaman", "meningkatkanmengoptimalisasi":
    ↳ "meningkatkan_mengoptimalisasi", "noninfokom": "non_infokom", "bindonesia":
    ↳ "bahasa_indonesia",
"mi": "manajemen_informatika", "renop": "rencana_operasi", "sndikti":
    ↳ "standar_nasional_pendidikan_tinggi",
}

DICTS = [replace_weird_words1, replace_weird_words2, replace_weird_words3,
    ↳ typo_dict1, typo_dict2]

```

```

# Merge dicts, drop empty / self-mapping entries
DICTS = [replace_weird_words1, replace_weird_words2, replace_weird_words3,
↳ typo_dict1, typo_dict2]
NORM_MAP = {}
for _d in DICTS:
    for _k, _v in _d.items():
        _k = _k.strip().lower(); _v = _v.strip()
        if _k and _v and _k != _v:
            NORM_MAP[_k] = _v

# Longest keys first so multi-word phrases match before their parts.
_NORM_KEYS = sorted(NORM_MAP.keys(), key=len, reverse=True)
_NORM_RE = re.compile(r'\b(' + '|'.join(re.escape(k) for k in _NORM_KEYS) +
↳ r')\b', re.IGNORECASE)
print(f'Normalization map: {len(NORM_MAP)} entries ({sum(1 for k in NORM_MAP if
↳ " " in k)} multi-word).')

def normalize_singkatan_typo(text):
    return _NORM_RE.sub(lambda m: NORM_MAP[m.group(0).lower()], text)

def clean_text(text):
    if pd.isna(text) or text == '':
        return ''
    text = str(text)
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)
    text = re.sub(r'@\w+', '', text)
    text = _emoji.sub('', text)
    text = ' '.join(text.split())
    return text.strip()

def preprocess_pipeline(text):
    """clean -> lowercase -> singkatan/typo normalization (the provided
↳ dictionaries)."""
    if pd.isna(text) or text == '':
        return ''
    return normalize_singkatan_typo(clean_text(str(text)).lower()).strip()

# quick check
_ex = 'IPK mhs prodi TI sdh sesuai dgn standart, namum detil ts belum lengkap.'
print('Example:')
print('  in :', _ex)
print('  out:', preprocess_pipeline(_ex))

```

Normalization map: 701 entries (9 multi-word).

Example:

```

in : IPK mhs prodi TI sdh sesuai dgn standart, namum detil ts belum lengkap.
out: ipk mahasiswa program_studi teknologi_informasi sudah sesuai dengan

```

standar, namun detail tracer_study belum lengkap.

7.7 Load & Label

```
[ ]: def load_and_preprocess(file_name, task, text_col):
    df = pd.read_excel(f'{BASE_PATH}/{file_name}')          # read the
    ↪labelled Excel file
    print(f'Loaded {file_name} shape={df.shape}')          # show how many
    ↪rows/columns came in
    missing = [c for c in [text_col] + ASPECT_COLUMNS if c not in df.columns]
    if missing:                                             # stop early if a
    ↪needed column is absent
        raise ValueError(f'Missing columns: {missing}')
    df = df.dropna(subset=[text_col]).reset_index(drop=True) # drop rows with
    ↪no comment text
    df['comment_original'] = df[text_col].astype(str)       # keep a copy of
    ↪the raw text (used later for augmentation)
    df[text_col] = df['comment_original'].map(preprocess_pipeline) # clean/
    ↪normalise the text we actually train on
    df = df[df[text_col].str.strip() != ''].reset_index(drop=True) # drop rows
    ↪that became empty after cleaning
    lab_map = ACD_LABEL_MAP if task == 'ACD' else ASC_LABEL_MAP # pick the
    ↪text->number label map for this task
    for col in ASPECT_COLUMNS:                             # convert each
    ↪aspect's word label to a number
        df[col] = df[col].astype(str).str.strip().str.lower().map(lab_map)
    if task == 'ACD':                                     # ACD: every
    ↪aspect must have a 0/1 label
        df = df.dropna(subset=ASPECT_COLUMNS).reset_index(drop=True)
        df[ASPECT_COLUMNS] = df[ASPECT_COLUMNS].astype(int)
    return df.reset_index(drop=True)                       # hand back the
    ↪cleaned, numeric-labelled table
```

7.8 Split (80/20), min-2-per-class guard, and few-shot sampling

```
[ ]: # Turn the label columns into a plain matrix so the stratified splitter can
    ↪read them.
def _label_matrix(df, task):
    rows = []
    for _, r in df.iterrows():                             # go row by row (one comment
    ↪at a time)
        vec = []
        for col in ASPECT_COLUMNS:                         # walk all 8 aspects
            v = r[col]
            if task == 'ACD':
```

```

        vec.append(int(v))                # ACD: one 0/1 value per
↪aspect
        else:
            oh = [0, 0, 0]                # ASC: one-hot for positif/
↪negatif/netral
            if not (isinstance(v, float) and np.isnan(v)):
                oh[int(v)] = 1             # mark the class that is
↪present
            vec.extend(oh)
        rows.append(vec)
        return np.array(rows)             # matrix of shape [comments,
↪labels]

# Split into 80% train / 20% test while keeping each aspect's class mix roughly
↪the same in both halves.
def stratified_split(df, task, test_size, seed):
    from sklearn.model_selection import train_test_split
    if not STRATIFY:                       # if stratification is turned
↪off, do a plain random split
        return train_test_split(df, test_size=test_size, random_state=seed,
↪shuffle=True)
    try:
        from iterstrat.ml_stratifiers import MultilabelStratifiedShuffleSplit
        Y = _label_matrix(df, task)        # label matrix the splitter
↪needs
        msss = MultilabelStratifiedShuffleSplit(n_splits=1,
↪test_size=test_size, random_state=seed)
        big, small = next(msss.split(df.values, Y))    # big = train indices,
↪small = test indices
        return df.iloc[big].reset_index(drop=True), df.iloc[small].
↪reset_index(drop=True)
    except Exception as e:                 # if the library is missing,
↪fall back to a plain split
        print(f' [stratify] fallback to random split ({type(e).__name__})')
        return train_test_split(df, test_size=test_size, random_state=seed,
↪shuffle=True)

# Guarantee the test set has at least min_count examples of every class, moving
↪rows from train if needed.
def enforce_min_test_per_class(train_df, test_df, task, min_count, seed):
    '''Move rows train->test until each (aspect,class) has >= min_count in test
↪(where data allows).'''
    rng = random.Random(seed)              # local RNG so the moves are
↪reproducible
    label_vals = [0, 1] if task == 'ACD' else [0, 1, 2]    # the class values to
↪check

```

```

    train_df = train_df.reset_index(drop=True); test_df = test_df.
↳reset_index(drop=True)
    moved = 0
    for col in ASPECT_COLUMNS:                # for every aspect...
        for lv in label_vals:                # ...and every class of
↳that aspect
            have = int((test_df[col] == lv).sum()) # how many are already in
↳the test set
            if have >= min_count:                # enough already -> skip
                continue
            need = min_count - have                # how many more we must
↳move over
            cand = train_df.index[train_df[col] == lv].tolist() # training
↳rows that have this class
            rng.shuffle(cand)                    # shuffle so the choice
↳isn't order-dependent
            take = cand[:need]                    # take just enough
            if take:
                test_df = pd.concat([test_df, train_df.loc[take]],
↳ignore_index=True) # add to test
                train_df = train_df.drop(index=take).reset_index(drop=True)
↳# remove from train
                moved += len(take)

    if moved:
        print(f' [min-{min_count}] moved {moved} row(s) train->test to satisfy
↳per-class minimum')
        for col in ASPECT_COLUMNS:                # warn about any class
↳still short (no data left to move)
            for lv in label_vals:
                if int((test_df[col] == lv).sum()) < min_count:
                    nm = (ACD_LABEL_NAMES if task=='ACD' else ASC_LABEL_NAMES)[lv]
                    print(f' [warn] {col.strip()} / {nm}: only
↳{int((test_df[col]==lv).sum())} in test (data-limited)')
                return train_df.reset_index(drop=True), test_df.reset_index(drop=True)

# THE FEW-SHOT SAMPLER.
# For each (aspect, class) cell it draws up to n_per_class REAL comments, then
↳UNIONS all the draws.
# Because a comment carries a label for every aspect, one comment can satisfy
↳several cells at once,
# so the final fit set has each comment ONCE and the frequent classes end up
↳above n_per_class.
def few_shot_fit(train_df, task, n_per_class, seed):
    '''Union of up to n_per_class real comments per (aspect,class). No
↳augmentation.'''
    rng = random.Random(seed)                    # reproducible sampling

```

```

    pool = train_df.reset_index(drop=True) # the training rows we
↪sample from
    label_vals = [0, 1] if task == 'ACD' else [0, 1, 2]
    keep = set() # a SET -> automatically
↪de-duplicates comments
    report = []
    for col, short in zip(ASPECT_COLUMNS, ASPECT_SHORTS): # loop over the 8
↪aspects
        for lv in label_vals: # loop over each
↪class of that aspect
            if task == 'ACD':
                idxs = pool.index[pool[col] == lv].tolist() # rows
↪with this ACD class
            else:
                idxs = pool.index[pool[col] == float(lv)].tolist() # rows
↪with this ASC class
            if not idxs: # none exist -> nothing
↪to sample
                continue
            sel = rng.sample(idxs, n_per_class) if len(idxs) >= n_per_class
↪else idxs # 15, or all if fewer
            keep.update(sel) # add these comments to
↪the fit set (union)
            nm = (ACD_LABEL_NAMES if task=='ACD' else ASC_LABEL_NAMES)[lv]
            report.append(f' {short[:28]:<28} | {nm:<13}: {len(sel):>2}/
↪{n_per_class} real')
            fit_df = pool.loc[sorted(keep)].reset_index(drop=True) # build the final
↪training table from the kept rows
            print(f' Few-shot fit set: {len(fit_df)} unique comments (target
↪{n_per_class})/class)')
            for r in report: # print how many real
↪examples each cell contributed
                print(r)
    return fit_df

```

7.9 Dataset

```

[ ]: class ABSAMultiHeadDataset(Dataset):
    '''One comment -> input_ids/mask + label vector of length NUM_ASPECTS.
        ASC absent-aspect (NaN) -> -100 so it is ignored by that head's loss.'''
    def __init__(self, df, text_col, task, tokenizer, max_len):
        self.texts = df[text_col].astype(str).tolist() # the cleaned comment
↪texts
        self.task = task
        labels = []

```

```

        for _, r in df.iterrows():
            # build one label row
            # per comment
            row = []
            for col in ASPECT_COLUMNS:
                # one entry per
                # aspect (8 total)
                v = r[col]
                if isinstance(v, float) and np.isnan(v):
                    row.append(-100)
                    # -100 = "no label
                    # here" -> loss will skip this head
                else:
                    row.append(int(v))
                    # otherwise the class
            # index
            labels.append(row)
            self.labels = np.array(labels, dtype=np.int64)
            self.tokenizer = tokenizer; self.max_len = max_len
            def __len__(self):
                return len(self.texts)
                # how many comments
            # in this dataset
            def __getitem__(self, i):
                enc = self.tokenizer(self.texts[i], add_special_tokens=True,
                # max_length=self.max_len,
                padding='max_length', truncation=True,
                # return_tensors='pt') # tokenise one comment
                return {'input_ids': enc['input_ids'].flatten(),
                # token IDs
                # the model reads
                'attention_mask': enc['attention_mask'].flatten(), # 1 for real
                # tokens, 0 for padding
                'labels': torch.tensor(self.labels[i], dtype=torch.long)} #
                # the 8-aspect label vector

```

7.10 Model — shared IndoBERT + per-aspect heads

```

[ ]: class IndoBERT_MultiHead_ABSA(nn.Module):
    def __init__(self, num_aspects, num_labels_per_head, dropout_rate=0.3):
        super().__init__()
        self.bert = BertModel.from_pretrained(MODEL_CHECKPOINT)
        # the shared
        # IndoBERT encoder
        h = self.bert.config.hidden_size
        # size of
        # BERT's output vector (768)
        self.dropout = nn.Dropout(dropout_rate)
        #
        # regularisation before the heads
        # ONE small linear classifier per aspect (8 heads). Each head decides
        # for its own aspect only.
        self.heads = nn.ModuleList([nn.Linear(h, num_labels_per_head) for _ in
        # range(num_aspects)])
        def forward(self, input_ids, attention_mask):

```

```

        pooled = self.bert(input_ids=input_ids, attention_mask=attention_mask).
        ↪pooler_output # encode the comment ONCE
        pooled = self.dropout(pooled) # apply
        ↪dropout to that shared vector
        return [head(pooled) for head in self.heads] # each head
        ↪reads the SAME vector -> 8 sets of logits

```

7.11 Per-aspect class weights (from the few-shot fit set)

```

[ ]: def compute_head_weights(fit_df, task):
    '''Balanced class weights per aspect head, computed on available (non -100)
    ↪labels.'''
    n_lab = 2 if task == 'ACD' else 3 # 2 classes for ACD, 3
    ↪for ASC
    weights = []
    for col in ASPECT_COLUMNS: # compute a weight
    ↪vector for each aspect head
        y = fit_df[col].dropna().astype(int).values # the labels actually
    ↪present for this aspect
        present = np.unique(y) # which classes appear
    ↪at all
        w = np.ones(n_lab, dtype=np.float32) # default weight 1.0
    ↪for every class
        if len(present) >= 2: # only balance if at
    ↪least 2 classes are present
            cw = compute_class_weight('balanced', classes=present, y=y) #
    ↪rarer class -> bigger weight
            for c, val in zip(present, cw):
                w[c] = val # store the computed
    ↪weight in its class slot
            weights.append(torch.tensor(w, dtype=torch.float32).to(device)) # move
    ↪to GPU/CPU as a tensor
        return weights # one weight vector per
    ↪aspect, used in the loss

```

7.12 Training (weighted CE, no validation, no early stopping)

```

[ ]: def train_model(fit_df, task, text_col, seed):
    set_all_seeds(seed) # reset all RNGs so
    ↪training is reproducible
    n_lab = 2 if task == 'ACD' else 3 # classes per head: 2
    ↪(ACD) or 3 (ASC)
    ds = ABSAMultiHeadDataset(fit_df, text_col, task, tokenizer, MAX_SEQ_LEN)
    ↪# wrap the fit set
    loader = DataLoader(ds, batch_size=BATCH_SIZE, shuffle=True)
    ↪# feed it in shuffled batches

```

```

    model = IndoBERT_MultiHead_ABSA(NUM_ASPECTS, n_lab, DROPOUT).to(device)
    ↪ # build the 8-head model
    head_weights = compute_head_weights(fit_df, task)
    ↪ # per-aspect class weights
    # One weighted cross-entropy loss per aspect. ignore_index=-100 -> absent
    ↪ labels don't contribute.
    loss_fns = [nn.CrossEntropyLoss(weight=head_weights[a], ignore_index=-100)
                 for a in range(NUM_ASPECTS)]

    optimizer = AdamW(model.parameters(), lr=LEARNING_RATE,
    ↪ weight_decay=WEIGHT_DECAY) # flat LR, no scheduler

    history = []
    for epoch in range(EPOCHS):
        ↪ # train for a fixed number
        ↪ of epochs (5)
        model.train(); run = 0.0
        ↪ # training mode; running
        ↪ loss accumulator
        pbar = tqdm(loader, desc=f' epoch {epoch+1}/{EPOCHS}', leave=False)
        for batch in pbar:
            ↪ # one mini-batch of
            ↪ comments at a time
            ids = batch['input_ids'].to(device) # token IDs -> GPU
            mask = batch['attention_mask'].to(device) # attention mask -> GPU
            lab = batch['labels'].to(device) # true labels, shape
            ↪ [batch, 8]
            logits_list = model(ids, mask)
            ↪ # forward pass -> 8 heads'
            ↪ logits
            # total loss = sum of each head's loss (each head compares its
            ↪ column of labels)
            loss = sum(loss_fns[a](logits_list[a], lab[:, a]) for a in
            ↪ range(NUM_ASPECTS))
            optimizer.zero_grad(); loss.backward()
            ↪ # clear old gradients,
            ↪ back-propagate
            optimizer.step()
            ↪ # update the weights
            run += loss.item(); pbar.set_postfix(loss=loss.item())
            ep_loss = run / len(loader)
            ↪ # average loss this epoch
            history.append(ep_loss)
            print(f' epoch {epoch+1}: train_loss={ep_loss:.4f}')
    return model, history
    ↪ # trained model + loss
    ↪ curve

```

7.13 Prediction

```
[ ]: @torch.no_grad()                                # no gradients needed when
      ↪only predicting
def predict(model, df, text_col, task):
    model.eval()                                     # evaluation mode (turns
    ↪off dropout)
    ds = ABSAMultiHeadDataset(df, text_col, task, tokenizer, MAX_SEQ_LEN)
    loader = DataLoader(ds, batch_size=BATCH_SIZE, shuffle=False) # keep order
    ↪so preds line up with rows
    preds, labels = [], []
    for batch in loader:
        ids = batch['input_ids'].to(device)
        mask = batch['attention_mask'].to(device)
        logits_list = model(ids, mask)               # 8 heads' logits for this
    ↪batch
        # for each head take the highest-scoring class -> one prediction per
    ↪aspect -> [batch, 8]
        p = torch.stack([torch.argmax(l, dim=1) for l in logits_list], dim=1)
        preds.append(p.cpu().numpy())                 # collect predictions
        labels.append(batch['labels'].numpy())         # collect the true labels
    return np.vstack(preds), np.vstack(labels)         # stack all batches into
    ↪full arrays
```

7.14 Metrics — per-aspect P/R/F1, macro-F1 + weighted-F1, confusion matrices

```
[ ]: def _valid(true, pred, task):
    if task == 'ASC':
        m = true != -100
        return true[m], pred[m]
    return true, pred

def print_metrics(preds, labels, task, name):
    label_names = ACD_LABEL_NAMES if task == 'ACD' else ASC_LABEL_NAMES
    idxs = list(range(len(label_names)))
    print(f'\n{"="*64}\n {name}\n{"="*64}')
    macro_list, weighted_list = [], []
    for a, asp in enumerate(ASPECT_SHORTS):
        true, pred = _valid(labels[:, a], preds[:, a], task)
        if len(true) == 0:
            print(f'\nAspect: {asp} (no labels)'); continue
        p, r, f, _ = precision_recall_fscore_support(true, pred, labels=idxs,
    ↪average=None, zero_division=0)
        mf1 = f1_score(true, pred, average='macro', zero_division=0)
        wf1 = f1_score(true, pred, average='weighted', zero_division=0)
        macro_list.append(mf1); weighted_list.append(wf1)
```

```

        print(f'\nAspect: {asp}')
        for i, ln in enumerate(label_names):
            print(f' {ln:<22} P={p[i]:.3f} R={r[i]:.3f} F1={f[i]:.3f}')
            print(f' Macro F1 {mf1:.4f} | Weighted F1 {wf1:.4f}')
            mean_macro = float(np.mean(macro_list)) if macro_list else float("nan")
            mean_weight = float(np.mean(weighted_list)) if weighted_list else
↪float("nan")
            print(f'\n MEAN Macro F1: {mean_macro:.4f} | MEAN Weighted F1:
↪{mean_weight:.4f}')
            return {'mean_macro_f1': mean_macro, 'mean_weighted_f1': mean_weight,
                    'per_aspect_macro': dict(zip(ASPECT_SHORTS, macro_list)),
                    'per_aspect_weighted': dict(zip(ASPECT_SHORTS, weighted_list))}

def plot_cms(preds, labels, task, name):
    label_names = ACD_LABEL_NAMES if task == 'ACD' else ASC_LABEL_NAMES
    idxs = list(range(len(label_names)))
    fig, axes = plt.subplots(2, 4, figsize=(24, 12))
    for a, (asp, ax) in enumerate(zip(ASPECT_SHORTS, axes.flatten())):
        true, pred = _valid(labels[:, a], preds[:, a], task)
        if len(true) == 0:
            ax.set_title(f'{asp}\n(no data)', fontsize=9); ax.axis('off');
↪continue
        cm = confusion_matrix(true, pred, labels=idxs)
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                    xticklabels=label_names, yticklabels=label_names, ax=ax,
                    cbar=True, linewidths=0.5, linecolor='white')
        ax.set_title(asp, fontsize=9); ax.set_xlabel('Pred', fontsize=8); ax.
↪set_ylabel('True', fontsize=8)
        ax.tick_params(labelsize=7)
        plt.suptitle(f'Confusion Matrices - {name}', fontsize=13, y=1.01)
        plt.tight_layout()
        fname = f'{BASE_PATH}/results/baseline/cm_{name}.png'; os.makedirs(os.path.
↪dirname(fname), exist_ok=True); plt.savefig(fname, dpi=150,
↪bbox_inches='tight'); plt.show()
        print('Saved ->', fname)

def plot_training(history, name):
    plt.figure(figsize=(7, 4))
    plt.plot(range(1, len(history) + 1), history, marker='o', linewidth=2,
↪color='#2b6cb0')
    plt.title(f'Training Loss - {name}', fontsize=12)
    plt.xlabel('Epoch'); plt.ylabel('Summed weighted CE loss')
    plt.xticks(range(1, len(history) + 1)); plt.grid(True, alpha=0.3)
    fname = f'{BASE_PATH}/results/baseline/training_{name}.png'
    os.makedirs(os.path.dirname(fname), exist_ok=True)

```

```
plt.tight_layout(); plt.savefig(fname, dpi=150, bbox_inches='tight'); plt.
↪show()
print('Saved ->', fname)
```

7.15 Save predictions to Excel

```
[ ]: def save_predictions_excel(test_df, preds, labels, text_col, task, name):
    names = ACD_LABEL_NAMES if task == 'ACD' else ASC_LABEL_NAMES
    nm = lambda v: '' if int(v) == -100 else names[int(v)]
    rows = []
    for i in range(len(test_df)):
        r = {'comment': str(test_df.iloc[i][text_col])}
        for a, asp in enumerate(ASPECT_SHORTS):
            r[f'{asp} | true'] = nm(labels[i, a])
            r[f'{asp} | pred'] = nm(preds[i, a])
        rows.append(r)
    out = pd.DataFrame(rows)
    os.makedirs(f'{BASE_PATH}/results/baseline', exist_ok=True)
    path = f'{BASE_PATH}/results/baseline/predictions_{name}_weighted_baseline.
↪xlsx'
    out.to_excel(path, index=False)
    print(f' Saved {len(out)} test comments + predictions -> {path}')
    return out
```

7.16 Runner

```
[ ]: def run_experiment(exp):
    name, task, text_col = exp['name'], exp['task'], exp['text_col']
    print(f'\n{"#"*66}\n {name} [{task} / weighted-class baseline / 80-20 /
↪few-shot {N_PER_CLASS}]\n{"#"*66}')

    df = load_and_preprocess(exp['file'], task, text_col) # load +
↪clean + numeric labels

    train_df, test_df = stratified_split(df, task, TEST_FRAC, SEED) # 80/20
↪stratified split
    train_df, test_df = enforce_min_test_per_class(train_df, test_df, task,
↪MIN_TEST_PER_CLASS, SEED) # guarantee test coverage
    print(f' Split: total={len(df)} | train={len(train_df)} |
↪test={len(test_df)} (NO validation)')

    # Save the EXACT split so the tahap thesaurus trains/tests on the SAME data
    # This is what makes the two experiments a fair comparison: identical train
↪and test rows.
    os.makedirs(f'{BASE_PATH}/results/baseline/splits', exist_ok=True)
    keep_cols = ['comment_original', text_col] + ASPECT_COLUMNS
```

```

train_df[keep_cols].to_excel(f'{BASE_PATH}/results/baseline/splits/
↳{name}_train.xlsx', index=False)
test_df[keep_cols].to_excel(f'{BASE_PATH}/results/baseline/splits/
↳{name}_test.xlsx', index=False)
print(f' Saved splits -> {BASE_PATH}/results/baseline/splits/{name}_train.
↳xlsx & {name}_test.xlsx')

fit_df = few_shot_fit(train_df, task, N_PER_CLASS, SEED)           # pick the
↳few-shot training comments

model, history = train_model(fit_df, task, text_col, SEED)        # fine-tune
↳IndoBERT on them

print('\n Evaluating on held-out TEST set...')
preds, labels = predict(model, test_df, text_col, task)           # predict on
↳the untouched test set
metrics = print_metrics(preds, labels, task, name)                # P/R/F1,
↳macro-F1, weighted-F1
plot_cms(preds, labels, task, name)                               # confusion
↳matrices
plot_training(history, name)                                       # loss curve
save_predictions_excel(test_df, preds, labels, text_col, task, name) #
↳dump predictions to Excel

del model; gc.collect()                                           # free memory
↳before the next experiment
if torch.cuda.is_available(): torch.cuda.empty_cache()
return {'metrics': metrics, 'preds': preds, 'labels': labels,
        'n_train': len(train_df), 'n_fit': len(fit_df), 'n_test':
↳len(test_df)}

```

7.17 Run all four experiments (ACD AK/AL, ASC AK/AL)

```

[ ]: ALL_RESULTS = {}
for exp in ACD_EXPERIMENTS + ASC_EXPERIMENTS:
    ALL_RESULTS[exp['name']] = run_experiment(exp)

print('\nDone. Mean macro-F1 / weighted-F1:')
for k, v in ALL_RESULTS.items():
    m = v['metrics']
    print(f' {k:<8} macro={m["mean_macro_f1"]:.4f}
↳weighted={m["mean_weighted_f1"]:.4f}')

```

```

#####
ACD_AK [ACD / weighted-class baseline / 80-20 / few-shot 15]
#####

```

```

Loaded Hasil Ground Truth_ACD AK_clean.xlsx shape=(200, 12)
[min-2] moved 1 row(s) train->test to satisfy per-class minimum
Split: total=200 | train=159 | test=41 (NO validation)
Saved splits ->
/content/drive/MyDrive/results/baseline/splits/ACD_AK_train.xlsx &
ACD_AK_test.xlsx
Few-shot fit set: 119 unique comments (target 15/class)
capaian pembelajaran lulusan | tidak_relevan: 2/15 real
capaian pembelajaran lulusan | relevan : 15/15 real
indeks prestasi kumulatif | tidak_relevan: 15/15 real
indeks prestasi kumulatif | relevan : 15/15 real
kelulusan tepat waktu | tidak_relevan: 15/15 real
kelulusan tepat waktu | relevan : 15/15 real
masa tunggu lulusan | tidak_relevan: 15/15 real
masa tunggu lulusan | relevan : 15/15 real
pelacakan dan perekaman data | tidak_relevan: 15/15 real
pelacakan dan perekaman data | relevan : 15/15 real
prestasi mahasiswa | tidak_relevan: 15/15 real
prestasi mahasiswa | relevan : 15/15 real
karya dengan hak kekayaan in | tidak_relevan: 15/15 real
karya dengan hak kekayaan in | relevan : 15/15 real
kesesuaian bidang kerja | tidak_relevan: 15/15 real
kesesuaian bidang kerja | relevan : 15/15 real
config.json: 0%| | 0.00/1.53k [00:00<?, ?B/s]
pytorch_model.bin: 0%| | 0.00/498M [00:00<?, ?B/s]
Loading weights: 0%| | 0/199 [00:00<?, ?it/s]
epoch 1/5: 0%| | 0/15 [00:00<?, ?it/s]
model.safetensors: 0%| | 0.00/498M [00:00<?, ?B/s]
epoch 1: train_loss=4.9322
epoch 2/5: 0%| | 0/15 [00:00<?, ?it/s]
epoch 2: train_loss=3.8374
epoch 3/5: 0%| | 0/15 [00:00<?, ?it/s]
epoch 3: train_loss=3.2890
epoch 4/5: 0%| | 0/15 [00:00<?, ?it/s]
epoch 4: train_loss=2.9521
epoch 5/5: 0%| | 0/15 [00:00<?, ?it/s]
epoch 5: train_loss=2.4887

Evaluating on held-out TEST set...

```

```
=====
```

ACD_AK

Aspect: capaian pembelajaran lulusan

tidak_relevan	P=0.000	R=0.000	F1=0.000
relevan	P=0.951	R=1.000	F1=0.975
Macro F1 0.4875		Weighted F1 0.9274	

Aspect: indeks prestasi kumulatif

tidak_relevan	P=0.429	R=0.500	F1=0.462
relevan	P=0.912	R=0.886	F1=0.899
Macro F1 0.6800		Weighted F1 0.8346	

Aspect: kelulusan tepat waktu

tidak_relevan	P=0.700	R=0.538	F1=0.609
relevan	P=0.806	R=0.893	F1=0.847
Macro F1 0.7281		Weighted F1 0.7718	

Aspect: masa tunggu lulusan

tidak_relevan	P=0.800	R=0.800	F1=0.800
relevan	P=0.935	R=0.935	F1=0.935
Macro F1 0.8677		Weighted F1 0.9024	

Aspect: pelacakan dan perekaman data lulusan

tidak_relevan	P=0.727	R=0.500	F1=0.593
relevan	P=0.733	R=0.880	F1=0.800
Macro F1 0.6963		Weighted F1 0.7191	

Aspect: prestasi mahasiswa

tidak_relevan	P=0.538	R=0.778	F1=0.636
relevan	P=0.929	R=0.812	F1=0.867
Macro F1 0.7515		Weighted F1 0.8161	

Aspect: karya dengan hak kekayaan intelektual

tidak_relevan	P=0.636	R=0.467	F1=0.538
relevan	P=0.733	R=0.846	F1=0.786
Macro F1 0.6621		Weighted F1 0.6953	

Aspect: kesesuaian bidang kerja

tidak_relevan	P=0.800	R=0.615	F1=0.696
relevan	P=0.839	R=0.929	F1=0.881
Macro F1 0.7885		Weighted F1 0.8225	

MEAN Macro F1: 0.7077 | MEAN Weighted F1: 0.8111

Saved -> /content/drive/MyDrive/results/baseline/cm_ACD_AK.png

Saved -> /content/drive/MyDrive/results/baseline/training_ACD_AK.png

Saved 41 test comments + predictions -> /content/drive/MyDrive/results/base

```

line/predictions_ACD_AK_weighted_baseline.xlsx

#####
ACD_AL [ACD / weighted-class baseline / 80-20 / few-shot 15]
#####
Loaded Hasil Ground Truth_ACD AL_clean.xlsx shape=(200, 12)
[min-2] moved 2 row(s) train->test to satisfy per-class minimum
Split: total=200 | train=157 | test=43 (NO validation)
Saved splits ->
/content/drive/MyDrive/results/baseline/splits/ACD_AL_train.xlsx &
ACD_AL_test.xlsx
Few-shot fit set: 120 unique comments (target 15/class)
capaian pembelajaran lulusan | tidak_relevan: 1/15 real
capaian pembelajaran lulusan | relevan : 15/15 real
indeks prestasi kumulatif | tidak_relevan: 13/15 real
indeks prestasi kumulatif | relevan : 15/15 real
kelulusan tepat waktu | tidak_relevan: 15/15 real
kelulusan tepat waktu | relevan : 15/15 real
masa tunggu lulusan | tidak_relevan: 15/15 real
masa tunggu lulusan | relevan : 15/15 real
pelacakan dan perekaman data | tidak_relevan: 15/15 real
pelacakan dan perekaman data | relevan : 15/15 real
prestasi mahasiswa | tidak_relevan: 15/15 real
prestasi mahasiswa | relevan : 15/15 real
karya dengan hak kekayaan in | tidak_relevan: 15/15 real
karya dengan hak kekayaan in | relevan : 15/15 real
kesesuaian bidang kerja | tidak_relevan: 15/15 real
kesesuaian bidang kerja | relevan : 15/15 real
Loading weights: 0%| | 0/199 [00:00<?, ?it/s]
epoch 1/5: 0%| | 0/15 [00:00<?, ?it/s]
epoch 1: train_loss=4.7581
epoch 2/5: 0%| | 0/15 [00:00<?, ?it/s]
epoch 2: train_loss=3.9465
epoch 3/5: 0%| | 0/15 [00:00<?, ?it/s]
epoch 3: train_loss=3.2544
epoch 4/5: 0%| | 0/15 [00:00<?, ?it/s]
epoch 4: train_loss=2.6656
epoch 5/5: 0%| | 0/15 [00:00<?, ?it/s]
epoch 5: train_loss=2.2397

Evaluating on held-out TEST set...

```

=====

ACD_AL

=====

Aspect: capaian pembelajaran lulusan

tidak_relevan	P=1.000	R=0.500	F1=0.667
relevan	P=0.976	R=1.000	F1=0.988
Macro F1	0.8273		Weighted F1 0.9730

Aspect: indeks prestasi kumulatif

tidak_relevan	P=0.667	R=0.667	F1=0.667
relevan	P=0.946	R=0.946	F1=0.946
Macro F1	0.8063		Weighted F1 0.9070

Aspect: kelulusan tepat waktu

tidak_relevan	P=0.750	R=0.900	F1=0.818
relevan	P=0.968	R=0.909	F1=0.938
Macro F1	0.8778		Weighted F1 0.9098

Aspect: masa tunggu lulusan

tidak_relevan	P=0.769	R=0.769	F1=0.769
relevan	P=0.900	R=0.900	F1=0.900
Macro F1	0.8346		Weighted F1 0.8605

Aspect: pelacakan dan perekaman data lulusan

tidak_relevan	P=0.750	R=0.600	F1=0.667
relevan	P=0.806	R=0.893	F1=0.847
Macro F1	0.7571		Weighted F1 0.7844

Aspect: prestasi mahasiswa

tidak_relevan	P=0.733	R=0.846	F1=0.786
relevan	P=0.929	R=0.867	F1=0.897
Macro F1	0.8411		Weighted F1 0.8630

Aspect: karya dengan hak kekayaan intelektual

tidak_relevan	P=0.800	R=0.800	F1=0.800
relevan	P=0.893	R=0.893	F1=0.893
Macro F1	0.8464		Weighted F1 0.8605

Aspect: kesesuaian bidang kerja

tidak_relevan	P=0.769	R=0.909	F1=0.833
relevan	P=0.967	R=0.906	F1=0.935
Macro F1	0.8844		Weighted F1 0.9094

MEAN Macro F1: 0.8344 | MEAN Weighted F1: 0.8834

Saved -> /content/drive/MyDrive/results/baseline/cm_ACD_AL.png

Saved -> /content/drive/MyDrive/results/baseline/training_ACD_AL.png

Saved 43 test comments + predictions -> /content/drive/MyDrive/results/baseline/predictions_ACD_AL_weighted_baseline.xlsx

#####

ASC_AK [ASC / weighted-class baseline / 80-20 / few-shot 15]

#####

Loaded Hasil Ground Truth_ASC AK_clean.xlsx shape=(200, 12)

Split: total=200 | train=158 | test=42 (NO validation)

Saved splits ->

/content/drive/MyDrive/results/baseline/splits/ASC_AK_train.xlsx &

ASC_AK_test.xlsx

Few-shot fit set: 134 unique comments (target 15/class)

capaian pembelajaran lulusan | positif : 15/15 real

capaian pembelajaran lulusan | negatif : 15/15 real

capaian pembelajaran lulusan | netral : 15/15 real

indeks prestasi kumulatif | positif : 15/15 real

indeks prestasi kumulatif | negatif : 11/15 real

indeks prestasi kumulatif | netral : 15/15 real

kelulusan tepat waktu | positif : 15/15 real

kelulusan tepat waktu | negatif : 9/15 real

kelulusan tepat waktu | netral : 15/15 real

masa tunggu lulusan | positif : 15/15 real

masa tunggu lulusan | negatif : 10/15 real

masa tunggu lulusan | netral : 15/15 real

pelacakan dan perekaman data | positif : 15/15 real

pelacakan dan perekaman data | negatif : 8/15 real

pelacakan dan perekaman data | netral : 15/15 real

prestasi mahasiswa | positif : 15/15 real

prestasi mahasiswa | negatif : 14/15 real

prestasi mahasiswa | netral : 15/15 real

karya dengan hak kekayaan in | positif : 15/15 real

karya dengan hak kekayaan in | negatif : 12/15 real

karya dengan hak kekayaan in | netral : 15/15 real

kesesuaian bidang kerja | positif : 15/15 real

kesesuaian bidang kerja | negatif : 10/15 real

kesesuaian bidang kerja | netral : 15/15 real

Loading weights: 0%| | 0/199 [00:00<?, ?it/s]

epoch 1/5: 0%| | 0/17 [00:00<?, ?it/s]

epoch 1: train_loss=9.0780

epoch 2/5: 0%| | 0/17 [00:00<?, ?it/s]

epoch 2: train_loss=8.5834

epoch 3/5: 0%| | 0/17 [00:00<?, ?it/s]

epoch 3: train_loss=8.1590

epoch 4/5: 0%| | 0/17 [00:00<?, ?it/s]

epoch 4: train_loss=7.6994

epoch 5/5: 0%| | 0/17 [00:00<?, ?it/s]

epoch 5: train_loss=6.9389

Evaluating on held-out TEST set...

=====

ASC_AK

=====

Aspect: capaian pembelajaran lulusan

positif	P=0.769	R=0.526	F1=0.625
negatif	P=0.250	R=0.167	F1=0.200
netral	P=0.417	R=0.625	F1=0.500
Macro F1	0.4417	Weighted F1	0.5140

Aspect: indeks prestasi kumulatif

positif	P=0.800	R=0.632	F1=0.706
negatif	P=1.000	R=0.333	F1=0.500
netral	P=0.500	R=0.750	F1=0.600
Macro F1	0.6020	Weighted F1	0.6503

Aspect: kelulusan tepat waktu

positif	P=0.765	R=0.684	F1=0.722
negatif	P=0.111	R=0.500	F1=0.182
netral	P=0.500	R=0.143	F1=0.222
Macro F1	0.3754	Weighted F1	0.5586

Aspect: masa tunggu lulusan

positif	P=0.667	R=0.333	F1=0.444
negatif	P=0.000	R=0.000	F1=0.000
netral	P=0.318	R=0.700	F1=0.438
Macro F1	0.2940	Weighted F1	0.3992

Aspect: pelacakan dan perekaman data lulusan

positif	P=0.786	R=0.611	F1=0.688
negatif	P=0.000	R=0.000	F1=0.000
netral	P=0.333	R=0.400	F1=0.364
Macro F1	0.3504	Weighted F1	0.5677

Aspect: prestasi mahasiswa

positif	P=0.917	R=0.611	F1=0.733
negatif	P=0.000	R=0.000	F1=0.000
netral	P=0.462	R=0.750	F1=0.571
Macro F1	0.4349	Weighted F1	0.5924

Aspect: karya dengan hak kekayaan intelektual

positif	P=0.571	R=0.286	F1=0.381
negatif	P=0.000	R=0.000	F1=0.000
netral	P=0.353	R=0.667	F1=0.462
Macro F1 0.2808		Weighted F1 0.3649	

Aspect: kesesuaian bidang kerja

positif	P=0.615	R=0.533	F1=0.571
negatif	P=0.000	R=0.000	F1=0.000
netral	P=0.400	R=0.545	F1=0.462
Macro F1 0.3443		Weighted F1 0.4874	

MEAN Macro F1: 0.3904 | MEAN Weighted F1: 0.5168

Saved -> /content/drive/MyDrive/results/baseline/cm_ASC_AK.png

Saved -> /content/drive/MyDrive/results/baseline/training_ASC_AK.png

Saved 42 test comments + predictions -> /content/drive/MyDrive/results/baseline/predictions_ASC_AK_weighted_baseline.xlsx

#####

ASC_AL [ASC / weighted-class baseline / 80-20 / few-shot 15]

#####

Loaded Hasil Ground Truth_ASC AL_clean.xlsx shape=(200, 12)

[min-2] moved 3 row(s) train->test to satisfy per-class minimum

Split: total=200 | train=156 | test=44 (NO validation)

Saved splits ->

/content/drive/MyDrive/results/baseline/splits/ASC_AL_train.xlsx &

ASC_AL_test.xlsx

Few-shot fit set: 132 unique comments (target 15/class)

capaian pembelajaran lulusan	positif	: 15/15 real
capaian pembelajaran lulusan	negatif	: 12/15 real
capaian pembelajaran lulusan	netral	: 15/15 real
indeks prestasi kumulatif	positif	: 15/15 real
indeks prestasi kumulatif	negatif	: 5/15 real
indeks prestasi kumulatif	netral	: 15/15 real
kelulusan tepat waktu	positif	: 15/15 real
kelulusan tepat waktu	negatif	: 10/15 real
kelulusan tepat waktu	netral	: 15/15 real
masa tunggu lulusan	positif	: 15/15 real
masa tunggu lulusan	negatif	: 10/15 real
masa tunggu lulusan	netral	: 15/15 real
pelacakan dan perekaman data	positif	: 15/15 real
pelacakan dan perekaman data	negatif	: 4/15 real
pelacakan dan perekaman data	netral	: 15/15 real
prestasi mahasiswa	positif	: 15/15 real
prestasi mahasiswa	negatif	: 7/15 real
prestasi mahasiswa	netral	: 15/15 real
karya dengan hak kekayaan in	positif	: 15/15 real
karya dengan hak kekayaan in	negatif	: 3/15 real

```

karya dengan hak kekayaan in | netral      : 15/15 real
kesesuaian bidang kerja      | positif   : 15/15 real
kesesuaian bidang kerja      | negatif   :  2/15 real
kesesuaian bidang kerja      | netral     : 15/15 real

Loading weights:  0%|          | 0/199 [00:00<?, ?it/s]

epoch 1/5:  0%|          | 0/17 [00:00<?, ?it/s]
epoch 1: train_loss=8.6836

epoch 2/5:  0%|          | 0/17 [00:00<?, ?it/s]
epoch 2: train_loss=7.9509

epoch 3/5:  0%|          | 0/17 [00:00<?, ?it/s]
epoch 3: train_loss=7.2582

epoch 4/5:  0%|          | 0/17 [00:00<?, ?it/s]
epoch 4: train_loss=6.7324

epoch 5/5:  0%|          | 0/17 [00:00<?, ?it/s]
epoch 5: train_loss=6.3625

Evaluating on held-out TEST set...

=====
ASC_AL
=====

Aspect: capaian pembelajaran lulusan
positif          P=0.882  R=0.517  F1=0.652
negatif          P=0.500  R=0.571  F1=0.533
netral           P=0.316  R=0.750  F1=0.444
Macro F1 0.5433 | Weighted F1 0.5955

Aspect: indeks prestasi kumulatif
positif          P=0.500  R=0.286  F1=0.364
negatif          P=0.000  R=0.000  F1=0.000
netral           P=0.360  R=0.600  F1=0.450
Macro F1 0.2712 | Weighted F1 0.3786

Aspect: kelulusan tepat waktu
positif          P=1.000  R=0.077  F1=0.143
negatif          P=0.300  R=0.750  F1=0.429
netral           P=0.538  R=0.700  F1=0.609
Macro F1 0.3934 | Weighted F1 0.4255

Aspect: masa tunggu lulusan
positif          P=0.750  R=0.316  F1=0.444

```

negatif	P=0.250	R=0.667	F1=0.364
netral	P=0.389	R=0.583	F1=0.467
Macro F1 0.4249		Weighted F1 0.4452	

Aspect: pelacakan dan perekaman data lulusan

positif	P=0.733	R=0.524	F1=0.611
negatif	P=0.000	R=0.000	F1=0.000
netral	P=0.368	R=0.583	F1=0.452
Macro F1 0.3542		Weighted F1 0.5215	

Aspect: prestasi mahasiswa

positif	P=0.882	R=0.682	F1=0.769
negatif	P=0.200	R=0.500	F1=0.286
netral	P=0.455	R=0.556	F1=0.500
Macro F1 0.5183		Weighted F1 0.6665	

Aspect: karya dengan hak kekayaan intelektual

positif	P=0.875	R=0.700	F1=0.778
negatif	P=0.000	R=0.000	F1=0.000
netral	P=0.500	R=0.875	F1=0.636
Macro F1 0.4714		Weighted F1 0.6882	

Aspect: kesesuaian bidang kerja

positif	P=0.538	R=0.389	F1=0.452
negatif	P=0.500	R=0.500	F1=0.500
netral	P=0.450	R=0.600	F1=0.514
Macro F1 0.4886		Weighted F1 0.4812	

MEAN Macro F1: 0.4332 | MEAN Weighted F1: 0.5253

Saved -> /content/drive/MyDrive/results/baseline/cm_ASC_AL.png

Saved -> /content/drive/MyDrive/results/baseline/training_ASC_AL.png

Saved 44 test comments + predictions -> /content/drive/MyDrive/results/baseline/predictions_ASC_AL_weighted_baseline.xlsx

Done. Mean macro-F1 / weighted-F1:

ACD_AK	macro=0.7077	weighted=0.8111
ACD_AL	macro=0.8344	weighted=0.8834
ASC_AK	macro=0.3904	weighted=0.5168
ASC_AL	macro=0.4332	weighted=0.5253

7.18 Results summary

```
[ ]: summary = pd.DataFrame([
    {'experiment': k,
     'mean_macro_f1': round(v['metrics']['mean_macro_f1'], 4),
     'mean_weighted_f1': round(v['metrics']['mean_weighted_f1'], 4),
```

```

        'n_fit': v['n_fit'], 'n_test': v['n_test']}]
    for k, v in ALL_RESULTS.items()
]).set_index('experiment')
print(summary.to_string())

os.makedirs(f'{BASE_PATH}/results/baseline', exist_ok=True)
summary.to_excel(f'{BASE_PATH}/results/baseline/summary_weighted_baseline.xlsx')
print('\nSaved -> {}/results/baseline/summary_weighted_baseline.xlsx'.
      format(BASE_PATH))

```

	mean_macro_f1	mean_weighted_f1	n_fit	n_test
experiment				
ACD_AK	0.7077	0.8111	119	41
ACD_AL	0.8344	0.8834	120	43
ASC_AK	0.3904	0.5168	134	42
ASC_AL	0.4332	0.5253	132	44

Saved ->

/content/drive/MyDrive/results/baseline/summary_weighted_baseline.xlsx

8 Tahap Few-Shot Fine-Tuning — Tesaurus (IndoBERT)

Tahap ini melatih model IndoBERT (indobenchmark/indobert-base-p2) untuk ACD dan ASC dengan augmentasi data menggunakan penggantian sinonim dari tesaurus domain yang dibangun pada tahap sebelumnya. Tesaurus digunakan untuk mengisi kekurangan data pada kelas aspek yang kurang terwakili melalui variasi leksikal yang tetap mempertahankan label asli. Konfigurasi ini menggunakan split data uji yang persis sama dengan tahap baseline, sehingga perbedaan performa dapat sepenuhnya diatribusikan pada sumber augmentasi. Hasil (prediksi, metrik precision/recall/F1, dan confusion matrix) disimpan untuk dibandingkan dengan konfigurasi baseline.

8.1 Install & Import

```
[ ]: !pip install -q transformers datasets scikit-learn pandas openpyxl matplotlib_
↪seaborn tqdm iterative-stratification
```

```
[ ]: import os, re, random, warnings, gc
import numpy as np
import pandas as pd
from copy import deepcopy
from tqdm.auto import tqdm

import torch
import torch.nn as nn
from torch.optim import AdamW
from torch.utils.data import Dataset, DataLoader

from transformers import (
    BertTokenizer, BertModel,
    set_seed,
)
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import (
    precision_recall_fscore_support, f1_score,
    confusion_matrix, accuracy_score,
)
import matplotlib.pyplot as plt
import seaborn as sns

warnings.filterwarnings('ignore')
sns.set_style('whitegrid')
print('Libraries loaded.')
```

Libraries loaded.

8.2 Configuration

```
[ ]: MODEL_CHECKPOINT = 'indobenchmark/indobert-base-p2'

# Hyperparameters
LEARNING_RATE = 2e-5
BATCH_SIZE = 8
DROPOUT = 0.1
EPOCHS = 5
WEIGHT_DECAY = 0.3
MAX_SEQ_LEN = 512
SEED = 42

# Few-shot sampling & split
N_PER_CLASS = 15
TEST_FRAC = 0.20
MIN_TEST_PER_CLASS = 2 # enforce >= this many test examples per class, per
    ↳ aspect
STRATIFY = True # multilabel-stratified split (keeps per-aspect class
    ↳ mix)

# Labels
ACD_LABEL_MAP = {'relevan': 1, 'tidak relevan': 0}
ACD_LABEL_NAMES = ['tidak_relevan', 'relevan']

ASC_LABEL_MAP = {'positif': 0, 'negatif': 1, 'netral': 2}
ASC_LABEL_NAMES = ['positif', 'negatif', 'netral']

# Aspect columns (8 fixed aspects, '(manual)' gold columns)
ASPECT_COLUMNS = [
    'capaian pembelajaran lulusan (manual)',
    'indeks prestasi kumulatif (manual)',
    'kelulusan tepat waktu (manual)',
    'masa tunggu lulusan (manual)',
    'pelacakan dan perekaman data lulusan (manual)',
    'prestasi mahasiswa (manual)',
    'karya dengan hak kekayaan intelektual (manual) ',
    'kesesuaian bidang kerja (manual)',
]
NUM_ASPECTS = len(ASPECT_COLUMNS)
ASPECT_SHORTS = [c.replace(' (manual)', '').strip() for c in ASPECT_COLUMNS]

# Paths (Colab / Google Drive)
BASE_PATH = '/content/drive/MyDrive'

ACD_EXPERIMENTS = [
```

```

        {'name': 'ACD_AK', 'file': 'Hasil Ground Truth_ACD AK_clean.xlsx', 'task': 'ACD', 'text_col': 'komentar_AK'},
        {'name': 'ACD_AL', 'file': 'Hasil Ground Truth_ACD AL_clean.xlsx', 'task': 'ACD', 'text_col': 'komentar_AL'},
    ]
    ASC_EXPERIMENTS = [
        {'name': 'ASC_AK', 'file': 'Hasil Ground Truth_ASC AK_clean.xlsx', 'task': 'ASC', 'text_col': 'komentar_AK'},
        {'name': 'ASC_AL', 'file': 'Hasil Ground Truth_ASC AL_clean.xlsx', 'task': 'ASC', 'text_col': 'komentar_AL'},
    ]

    print(f'Model          : {MODEL_CHECKPOINT}')
    print(f'LR {LEARNING_RATE} | batch {BATCH_SIZE} | dropout {DROPOUT} | epochs {EPOCHS} | max_len {MAX_SEQ_LEN}')
    print(f'Few-shot          : {N_PER_CLASS} per (aspect,class), THESAURUS augmentation to fill')
    print(f'Split              : {int((1-TEST_FRAC)*100)}/{int(TEST_FRAC*100)}, NO validation | min {MIN_TEST_PER_CLASS} test/class/aspect')

    # Folder that holds the thesaurus union files (defaults to BASE_PATH).
    ACD_THESAURUS_DIR = BASE_PATH
    ASC_THESAURUS_DIR = f'{BASE_PATH}/Final Thesaurus/ASC/8 aspects_filtered'
    # ACD: ONE cleaned union file (covers both AK and AL). ASC: 8 per-aspect files, filtered by 'Dataset' column per stage.
    ACD_THESAURUS_FILES = {
        'ACD_AK': ['ACD_union_cleaned.xlsx'],
        'ACD_AL': ['ACD_union_cleaned.xlsx'],
    }
    ASC_ASPECT_FILES = [
        'capaian_pembelajaran_lulusan_filtered.xlsx',
        'indeks_prestasi_kumulatif_filtered.xlsx',
        'kelulusan_tepat_waktu_filtered.xlsx',
        'masa_tunggu_lulusan_filtered.xlsx',
        'pelacakan_dan_perekaman_data_lulusan_filtered.xlsx',
        'prestasi_mahasiswa_filtered.xlsx',
        'karya_dengan_hak_kekayaan_intelektual_filtered.xlsx',
        'kesesuaian_bidang_kerja_filtered.xlsx',
    ]
    ASC_THESAURUS_FILES = {
        'ASC_AK': ASC_ASPECT_FILES, # same 8 files; rows filtered by Dataset == 'ASC_AK'
        'ASC_AL': ASC_ASPECT_FILES, # same 8 files; rows filtered by Dataset == 'ASC_AL'
    }

```

```
# Thesaurus term filtering (keep concise, interchangeable phrases; drop noise/
↳meta notes)
SYN_MIN_CH, SYN_MAX_CH, SYN_MAX_WORDS = 3, 70, 8
_BAD_SUBSTR = ['tidak ditemukan', 'tidak ada', 'tidak relevan', 'tidak↳
↳terdapat',
               'akan diisi', 'sinonim', 'variasi', 'mengacu', 'nan']
SPLITS_DIR = f'{BASE_PATH}/results/baseline/splits' # written by the tahap↳
↳baseline
```

```
Model          : indobenchmark/indobert-base-p2
LR 2e-05 | batch 8 | dropout 0.1 | epochs 5 | max_len 512
Few-shot       : 15 per (aspect,class), THESAURUS augmentation to fill
Split          : 80/20, NO validation | min 2 test/class/aspect
```

8.3 Device & Reproducibility

```
[ ]: def set_all_seeds(seed):
    random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
    set_seed(seed)

set_all_seeds(SEED)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Device:', device)
if torch.cuda.is_available():
    print(' GPU:', torch.cuda.get_device_name(0))
```

```
Device: cuda
GPU: NVIDIA L4
```

8.4 Mount Google Drive

```
[ ]: from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

8.5 Tokenizer

```
[ ]: tokenizer = BertTokenizer.from_pretrained(MODEL_CHECKPOINT)
print('Tokenizer loaded:', MODEL_CHECKPOINT, '| vocab', tokenizer.vocab_size)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated
requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and
faster downloads.
```

```
tokenizer_config.json: 0%|          | 0.00/2.00 [00:00<?, ?B/s]
```



```

"pkmpm": "program_kreativitas_mahasiswi_pengabdian_masyarakat", "pl":
  ↳ "pembelajaran_lanjutan", "pmmb": "program_magang_mahasiswa_bersertifikat",
"pp": "pimpinan_pusat", "ppm": "pusat_penjaminan_mutu", "ps": "program_studi", "psbb":
  ↳ "pembatasan_sosial_berkala_besar",
"pspsk": "pusat_studi_pendidikan_dan_kebijakan", "ptnpts":
  ↳ "perguruan_tinggi_negeri_perguruan_tinggi_swasta", "ptpn":
  ↳ "perkebunan_tanaman_nusantara", "rpl": "rekayasa_perangkat_lunak", "rpm":
  ↳ "rencana_pembelajaran_mingguan", "rpp": "rencana_pelaksanaan_pembelajaran",
"rps": "rencana_pembelajaran_semester", "sbb": "sebagai_berikut", "sbg":
  ↳ "sebagai", "sbgmn": "sebagaimana", "scr": "secara", "sd": "sampai_dengan",
"sdh": "sudah", "sdlc": "system_development_life_cycle", "sdm":
  ↳ "sumber_daya_manusia", "shg": "sehingga", "sk": "surat_keputusan",
"sks": "satuan_kredit_semester", "sn": "standar_nasional", "snpt":
  ↳ "standar_nasional_pendidikan_tinggi", "spp":
  ↳ "surat_perjanjian_pemanfaatan", "str": "sarjana_terapan", "stt":
  ↳ "sekolah_tinggi_teknologi", "tdk": "tidak", "tgl": "tanggal", "th": "tahun", "thn":
  ↳ "tahun",
"thd": "terhadap", "tkj": "teknik_komputer_dan_jaringan", "tkk":
  ↳ "teknik_komputer_kontrol", "tkk": "tingkat", "trk":
  ↳ "teknologi_rekayasa_komputer",
"trkj": "teknologi_rekayasa_komputer", "ts": "tracer_study", "tsb":
  ↳ "tersebut", "tsbt": "tersebut", "ttg": "tentang",
"vm": "visi_misi", "vmts": "visi_misi_tujuan_dan_strategi", "wt":
  ↳ "waktu_tunggu", "prodi": "program_studi", "ik": "ilmu_komputer", "al":
  ↳ "asesmen_lapangan", "ak": "asesmen_kecukupan", "rtm": "papat_tinjauan_manajemen",
"cdc": "career_development_center", "spm": "sistem_penjaminan_mutu",
"phb": "program_hibah_binaan", "pt": "perguruan_tinggi",
"dtps": "dosen_tetap_program_studi", "lrs": "laporan_rencana_studi", "lpks":
  ↳ "lembaga_pengembangan_keterampilan_dan_sertifikasi",
"smk": "sekolah_menengah_kejuruan", "dst": "dan_seterusnya", "smp": "sampai", "lpm":
  ↳ "lembaga_penjaminan_mutu",
"std": "standar", "bpm": "badan_penjaminan_mutu", "bhs": "bahasa",
"kkmk": "kriteria_kinerja_manajemen_kualitas", "pkl":
  ↳ "rpaktek_kerja_lapangan", "psms": "program_studi_manajemen_sistem",
"kk": "kode_klasifikasi", "rtl": "rencana_tindak_lanjut", "smt": "semester",
"pbl": "problem_based_learning", "rpsm": "rencana_pembelajaran_semester",
"lp": "lembaga_penjaminan", "lpd": "laporan_penelitian_dosen", "pppm":
  ↳ "pengembangan_pendidikan_mahasiswa",
"bppm": "badan_penelitian_pengabdian_masyarakat", "lpppm":
  ↳ "lembaga_penelitian_pengabdian_masyarakat", "rpjp":
  ↳ "rencana_pembangunan_jangka_panjang",
"bpp": "biaya_penyelenggaraan_pendidikan", "st": "sarjana_teknik", "sp":
  ↳ "surat_peringatan",

```

```

"dpp":"dosen_pendamping_program","upps":"unit_pengelola_program_studi","pts":
  ↳"perguruan_tinggi_swasta","ptn":"perguruan_tinggi_negeri","pr":
  ↳"penelitian_riset","ttd":"tanda_tangan","pllm":
  ↳"lembaga_penelitian_dan_pengabdian_masyarakat",
"rp":"rencana_pembelajaran","drpm":"direktorat_riset_pengabdian_masyarakat",
"bph":"badan_pengurus_harian","ttp":"tetap","llppm":
  ↳"lembaga_penelitian_pengabdian_masyarakat",
"cfp":"certified_financial_planner","lppmbppm":
  ↳"lembaga_penelitian_pengabdian_masyarakat_badan_penjaminan_mutu_pendidikan",
"spk":"surat_perjanjian_kerja","pm":"penjaminan_mutu","dptri":
  ↳"dosen_tetap_penghitung_rasio","kpd":"kepada","dtpt":
  ↳"dosen_tetap_perguruan_tinggi","bgm":"bagaimana",
"dtp":"dosen_tetap_program","kts":"kurikulum_tingkat_satuan",
"jg":"juga","dgx":"dengan","rpjmn":
  ↳"rencana_pembangunan_jangka_menengah_nasional","ppkm":
  ↳"program_pengembangan_keterampilan_mahasiswa",
"dprpm":"direktorat_penelitian_pengabdian_masyarakat","bg":"bagaimana","spmpt":
  ↳"sistem_penjaminan_mutu_pendidikan_tinggi","pnbp":
  ↳"penerimaan_negara_bukan_pajak",
"wkt":"waktu","csr":"corporate_social_responsibility",
"kkp":"kualitas_kinerja_program","pkmdtpr":
  ↳"program_kreativitas_mahasiswa_dosen_tetap_penghitung_rasio",
"lppp":"lembaga_penelitian_pengabdian_masyarakat",
"dtr":"dosen_tetap_penghitung_rasio","ln":"luar_negeri","tmps":
  ↳"tim_penjaminan_mutu_program_studi","tpmj":"tim_penjaminan_mutu_jurusan",
"pmk":"penjaminan_mutu_kinerja","bkn":"bukan","pkms":
  ↳"program_kreativitas_mahasiswa_sosial","skripsita":"skripsi_tugas_akhir",
"pstp":"program_studi_teknik_pertambangan","ppkmdm":
  ↳"program_pengembangan_keterampilan_mahasiswa_dan_dosen",
"plp":"program_latihan_profesi","skl":"surat_keterangan_lulus",
"cplcpmk":
  ↳"capaian_pembelajaran_lulusan_dan_capaian_pembelajaran_mata_kuliah","cpmk":
  ↳"capaian_pembelajaran_mata_kuliah","rkps":
  ↳"rencana_kegiatan_pembelajaran_semester",
"sbgmn":"sebagaimana","jmlh":"jumlah","sms":"semester","lplppm":
  ↳"lembaga_penelitian_pengabdian_masyarakat","pstg":
  ↳"program_studi_teknik_geologi","pstk":"program_studi_teknik_kimia",
"skp":"surat_kerja_praktek","pblpbm":
  ↳"problem_based_learning_project_based_learning","prn":
  ↳"program_riset_nasional","ppg":"program_profesi_guru","psbm":
  ↳"program_studi_bisnis_dan_manajemen",
"lpj":"laporan_pertanggungjawaban","kkt":"kualitas_kinerja_tim",
"pjj":"pendidikan_jarak_jauh","pmp":"pemetaan_mutu_pendidikan","bpmp":
  ↳"balai_penjaminan_mutu_pendidikan","bppmp":
  ↳"balai_penjaminan_mutu_pendidikan","bdp":"balai_guru_penggerak",

```

```

"bkd":"beban_kerja_dosen","bkk":"bursa_kerja_khusus","bmw":
  ↳"belajar_melanjut_wirausaha","bnpb":
  ↳"badan_nasional_penanggulangan_bencana","bnpp":
  ↳"badan_nasional_pengelola_perbatasan","bnpt":
  ↳"badan_nasional_penanggulangan_terorisme",
"bp":"bimbingan_penyuluhan","bpk":"badan_pengurus_keuangan","bpl":
  ↳"badang_pengelola_latihan","bssn":"badan_siber_dan_sandi_negara",
"cpns":"calon_pegawai_negeri_sipil","crf":
  ↳"center_for_research_and_facilitation","crm":
  ↳"sistem_manajemen_hubungan_pelanggan","csrpkl":
  ↳"corporate_social_responsibility_program_kemitraan_bina_lingkungan","pkbl":
  ↳"program_kemitraan_bina_lingkungan",
"ct":"computational_thinking","ctf":"career_and_technology_foundations","cv":
  ↳"curriculum_vitae",
"ddtpr":"dosen_tetap_penghitung_rasio","dkk":"dan_kawan_kawan","dn":"dan","dng":
  ↳"dengan","dpkm":"direktorat_pengabdian_masyarakat","dpl":
  ↳"dosen_pembimbing_lapangan","dpm":"dewan_perwakilan_mahasiswa","dppm":
  ↳"direktorat_penelitian_pengabdian_masyarakat",
"dpr":"laporan_kemajuan_gelar","dpt":"daftar_pemilih_tetap",
"hki":"hak_kekayaan_intelektual","hku":"hak_kekayaan_usaha",
"iku":"indikator_kinerja_utama","ikt":"indikator_kinerja_tambahan","ikuikt":
  ↳"indikator_kinerja_utama_indikator_kinerja_tambahan","cbr":
  ↳"community_based_research","cbr":"computer_based_test","cl":
  ↳"collaborative_learning",
}

```

```

replace_weird_words2 = {
"tpg":"tunjangan_profesi_guru","dtpmhs":
  ↳"dosen_tetap_penghitung_rasio_mahasiswa","mhs":"mahasiswa",
"upps":"unit_pengelola_program_studi","spmi":
  ↳"sistem_penjaminan_mutu_internal","renstra":"rencana_strategis",
"menemu":"menemukan","isbn":"international_standard_book_number","prosentase":
  ↳"presentase","yg":"yang","haki":"hak_atas_kekayaan_intelektual",
"led":"laporan_evaluasi_diri","danatau":"dan_atau","peneltian":
  ↳"penelitian","utk":"untuk","ta":"tugas_akhir","dharma":"tridharma","amik":
  ↳"akademi_manajemen_informatika_dan_komputer",
"pgri":"persatuan_guru_republik_indonesia","umkm":
  ↳"usaha_mikro_kecil_menengah","abdimas":"pengabdian_masyarakat","uppsps":
  ↳"unit_pengelola_program_studi",
"berdasar":"berdasarkan","ti":"teknologi_informasi","dtpmahasiswa":
  ↳"dosen_tetap_penghitung_rasio_mahasiswa","ppepp":
  ↳"penetapan_pelaksanaan_evaluasi_pengendalian_dan_peningkatan",
"si":"sitem_informasi","kaprodi":"kepala_program_studi","dipa":
  ↳"daftar_isian_pelaksanaan_anggaran","itb":"institut_teknologi_bandung",
}

```

```

"kebijakanstandar": "kebijakan_standar", "renop":
  ↳ "rencana_operasional", "lemabagaunit": "lembaga_unit", "akakom":
  ↳ "akademi_aplikasi_komputer",
"permendikbud": "peraturan_menteri_pendidikan",
"standard": "standar", "simlitabmas":
  ↳ "sistem_informasi_penelitian_dan_pengabdian_masyarakat", "pob":
  ↳ "pedoman_operasional_baku",
"penelitan": "penelitian", "fti": "fakultas_teknologi_industri", "puslit":
  ↳ "pusat_penelitian", "ri": "republik_indonesia", "dosenmahasiswa":
  ↳ "dosen_mahasiswa",
"kkni": "kerangka_kualifikasi_nasional_indonesia",
"kkn": "kuliah_kerja_nyata", "pkmgft":
  ↳ "program_kreativitas_mahasiswa_gagasan_futuristik_tertulis", "pkmgfk":
  ↳ "program_kreativitas_mahasiswa_gagasan_futuristik_konstruktif",
}

replace_weird_words3 = {
"prodi": "program_studi", "ik": "ilmu_komputer", "ps": "program_studi", "al":
  ↳ "asesmen_lapangan", "ak": "asesmen_kecukupan", "pkm":
  ↳ "program_kreativitas_mahasiswa", "rtm": "papat_tinjauan_manajemen",
"ppm": "pengabdian_dan_pemberdayaan_masyarakat", "cp": "capaian_pembelajaran",
"lkps": "laporan_kinerja_program_studi",
"lrs": "laporan_rencana_studi", "kp": "kerja_praktek", "lpks":
  ↳ "lembaga_pengembangan_keterampilan_dan_sertifikasi",
"smk": "sekolah-menengah-kejuruan", "dst": "dan_seterusnya", "smp": "sampai", "pk":
  ↳ "penilaian_kinerja", "lpm": "lembaga_penjaminan_mutu",
"kkmk": "kriteria_kinerja_manajemen_kualitas", "pkl":
  ↳ "rpaktek_kerja_lapangan", "psms": "program_studi_manajemen_sistem",
"kk": "kode_klasifikasi", "tkk": "tenaga_kependidikan_khusus",
"vmts": "visi_misi_tujuan_dan_sasaran", "kbbk":
  ↳ "kurikulum_berbasis_kompetensi", "pbl": "problem_based_learning", "rpsm":
  ↳ "rencana_pembelajaran_semester", "ft": "fakultas_teknik", "ttg": "tentang",
"snpt": "standar_nasional_perguruan_tinggi", "lp": "lembaga_penjaminan", "lpd":
  ↳ "laporan_penelitian_dosen", "pppm": "pengembangan_pendidikan_mahasiswa",
"rpjp": "rencana_pembangunan_jangka_panjang",
"st": "sarjana_teknik", "sp": "surat_peringatan",
"dpp": "dosen_pendamping_program", "pr": "penelitian_riset", "ttd":
  ↳ "tanda_tangan", "pllm": "lembaga_penelitian_dan_pengabdian_masyarakat",
"rp": "rencana_pembelajaran", "drpm": "direktorat_riset_pengabdian_masyarakat",
"ttp": "tetap", "llppm": "lembaga_penelitian_pengabdian_masyarakat",
"cfp": "certified_financial_planner", "lppmbppm":
  ↳ "lembaga_penelitian_pengabdian_masyarakat_badan_penjaminan_mutu_pendidikan",
"spk": "surat_perjanjian_kerja", "pm": "penjaminan_mutu", "dptra":
  ↳ "dosen_tetap_penghitung_rasio", "kpd": "kepada", "dtpt":
  ↳ "dosen_tetap_perguruan_tinggi", "bgm": "bagaimana",
"dtp": "dosen_tetap_program", "kts": "kurikulum_tingkat_satuan",

```

```

"jg": "juga", "dgx": "dengan", "rpjmn":
    ↳ "rencana_pembangunan_jangka_menengah_nasional", "ppkm":
    ↳ "program_pengembangan_keterampilan_mahasiswa",
"dprpm": "direktorat_penelitian_pengabdian_masyarakat", "bg": "bagaimana", "spmpt":
    ↳ "sistem_penjaminan_mutu_pendidikan_tinggi", "pnbp":
    ↳ "penerimaan_negara_bukan_pajak",
"wkt": "waktu", "csr": "corporate_social_responsibility", "fgd":
    ↳ "focus_group_discussion",
"kkp": "kualitas_kinerja_program", "pkmdtpr":
    ↳ "program_kreativitas_mahasiswa_dosen_tetap_penghitung_rasio",
"lppp": "lembaga_penelitian_pengabdian_masyarakat", "rpp":
    ↳ "rencana_pembelajaran_program",
"dtr": "dosen_tetap_penghitung_rasio", "ln": "luar_negeri", "tmps":
    ↳ "tim_penjaminan_mutu_program_studi", "tpmj": "tim_penjaminan_mutu_jurusan",
"pmk": "penjaminan_mutu_kinerja", "bkn": "bukan", "pkms":
    ↳ "program_kreativitas_mahasiswa_sosial",
"pstp": "program_studi_teknik_pertambangan", "ppkmdm":
    ↳ "program_pengembangan_keterampilan_mahasiswa_dan_dosen",
"plp": "program_latihan_profesi", "bnsf":
    ↳ "badan_nasional_sertifikasi_profesi", "skl": "surat_keterangan_lulus",
"cplcpmk":
    ↳ "capaian_pembelajaran_lulusan_dan_capaian_pembelajaran_mata_kuliah", "cpmk":
    ↳ "capaian_pembelajaran_mata_kuliah", "rkps":
    ↳ "rencana_kegiatan_pembelajaran_semester", "mhsw": "mahasiswa",
"jl": "jalan", "sbgmn": "sebagaimana", "jmlh": "jumlah", "sms": "semester", "lplppm":
    ↳ "lembaga_penelitian_pengabdian_masyarakat", "pstg":
    ↳ "program_studi_teknik_geologi", "pstk": "program_studi_teknik_kimia",
"kpt": "kualitas_pendidikan_tinggi", "skp": "surat_kerja_praktek", "rata2":
    ↳ "rata_rata", "mk": "mata_kuliah", "pblpbm":
    ↳ "problem_based_learning_project_based_learning", "prn":
    ↳ "program_riset_nasional", "ppg": "program_profesi_guru", "psbm":
    ↳ "program_studi_bisnis_dan_manajemen",
"lpj": "laporan_pertanggungjawaban", "dpts": "dosen_tetap_program_studi", "kkt":
    ↳ "kualitas_kinerja_tim",
"pjj": "pendidikan_jarak_jauh", "pmp": "pemetaan_mutu_pendidikan", "ts1":
    ↳ "tracer_study_1", "ts2": "tracer_study_2", "ts3": "tracer_study_3", "ts4":
    ↳ "tracer_study_4", "ts5": "tracer_study_5", "ts6": "tracer_study_6",
}

typo_dict1 = {
    "undri": "undur_diri", "detil": "detail", "luarancapaian":
        ↳ "luaran_capaian", "sebasar": "sebesar", "komptetensi": "kompetensi",
    "luluipk": "lulus_indeks_prestasi_kumulatif", "perkulihan": "perkuliahahan", "dirjen":
        ↳ "direktorat_jenderal", "belmawa": "pembelajaran_dan_kemahasiswaan",
    "systemaplikasi": "system_aplikasi", "deskripsidatanarasi":
        ↳ "deskripsi_data_narasi", "informasidata": "informasi_data",

```

```

"egovernment":"government","ipkprestasi":
  ↳"indeks_prestasi_kumulatif_prestasi","perkaman":"perekaman","ydatat":"dapat",
"mahasiswat":"mahasiswa","lululusan":"lulusan","senasif":
  ↳"seminar_nasional_informasi",
"prestai":"prestasi","standart":"standar","kessusuaian":"kesesuaian","kkerja":
  ↳"kerja","mendaoat":"mendapat",
"maksimun":"maksimum","berbading":"berbanding","kebuman":
  ↳"kebumen","diperolehlah":"diperoleh","tigatahunan":"tiga_tahunan",
"namum":"namun","detailriil":"detail_riiil","dtprdosen":
  ↳"dosen_tetap_penghitung_rasio_dosen","klarfikiasi":"klarifikasi",
"kepuasanan":"kepuasan","fluaktif":"fluktuatif","ketidakkonsitenan":
  ↳"ketidakkonsistenan","simkatmawa":"sistem_pemeringkatan_kemahasiswaan",
"ekplisit":"eksplisit","terlenuhi":"terpenuhi","diihat":
  ↳"dilihat","pujiansebesar":"pujian_sebesar","pddikti":
  ↳"pangkalan_data_pendidikan_tinggi",
"atunggu":"tunggu","masy":"masyarakat","ditumkan":"ditemukan","reseponden":
  ↳"responden","kesesuaiaan":"kesesuaian",
"denagn":"dengan","berturutan":"berurutan","mnempuh":"menempuh","emenuhan":
  ↳"pemuhan","mahaswasiswa":"mahasiswa",
"melampui":"melampaui","dilaporakan":"dilaporkan","laporaan":"laporan","elama":
  ↳"lama","mskor":"skor","erlacak":"terlacak",
"lulusanlulusan":"lulusan_lulusan","daapat":"dapat","reepositori":
  ↳"repositori","bidanginfokom":"bidang_infokom",
"jumlh":"jumlah","menyarakan":"menyarankan","polinas":
  ↳"politeknik_informatika_nasional","inggirs":"inggris",
"datajumlah":"data_jumlah","aktf":"aktif","kementrian":
  ↳"kementerian","eratcukup":"erat_cukup","pprodi":"program_studi",
"sertiikat":"sertifikat","terakahir":"terakhir","berkerjaberwirausaha":
  ↳"bekerja_berwirausaha",
"lokalwilayahberwirausaha":"lokal_wilayah_berusaha","depkominfo":
  ↳"departemen_komunikasi_dan_informatika",
"linktracer":"link_tracer","dtidak":"tidak","berhuubng":"berhubung","gkerja":
  ↳"kerja","subcpmk":"sub_capaian_pembelajaran_mata_kuliah",
"saaat":"saat","danstr":"dan_sarjana_terapan","hkiteknologi":
  ↳"hak_kekayaan_intelektual_teknologi",
"gunaprodukbook":"guna_produk_buku","kurikulumlink":
  ↳"kurikulum_link","ketersedian":"ketersediaan",
"sikappengetahuanketerampilan":"sikap_pengetahuan_keterampilan","penilainnya":
  ↳"penilaiannya","lulusantabel":"lulusan_tabel",
"kerampilan":"keterampilan","tugaskuis":"tugas_kuis","pda":"pada","ket":
  ↳"keterangan","healthkathon":"health_kathon",
"defenisi":"definisi","perekaman":"perekaman","oprestasi":"prestasi","jkarya":
  ↳"karya","academyc":"academic",
"dibandakan":"dibandingkan","telahbelum":"telah_belum","sem":"semester","kkni":
  ↳"kerangka_kualifikasi_nasional_indonesia",

```

"diharapka":"diharapkan","penuruanan":"penurunan","renatan":"rentan","anatar":
 ↳"antar","dilaksanakantarcer":"dilaksanakan_tracer",
 "dashboard":"dashboard","dalammkurun":"dalam_kurun","kelulusah":
 ↳"kelulusan","dtingkat":"tingkat","sekprodi":"sekretaris_program_studi",
 "sertau":"serta","unipa":"universitas_papua","ppemenuhan":
 ↳"pemenuhan","ditelusur":"ditelusuri",
 "sebelumnyasetiap":"sebelumnya_setiap","diberikansedangkan":
 ↳"diberikan_sedangkan","bbrapa":"beberapa",
 "mahasiswabelum":"mahasiswa_belum","rekrutmen":"rekrutmen","hakidibawah":
 ↳"hak_kekayaan_intelektual_di_bawah",
 "menghasillkan":"menghasilkan","disosialisasiakn":
 ↳"disosialisasikan","nasionalpada":"nasional_pada","infromatika":
 ↳"informatika",
 "ratrata":"rata_rata","dtpsmahasiswa":
 ↳"dosen_tetap_program_studi_mahasiswa","tahunberdasarkan":"tahun_berdasarkan",
 "eventevent":"event_event","erdasarkan":"berdasarkan","nyapelaksanaan":
 ↳"nya_pelaksanaan","temanteman":"teman_teman",
 "studyunikama":"study_unikama","skore":"skor","menjacapi":
 ↳"mencapai","acehsumatera":"aceh_sumatera",
 "internasiona":"internasional","tercpai":"tercapai","renstra":
 ↳"rencana_strategis",
 "renstrarenoprkat":
 ↳"rencana_strategis_rencana_operasional_rencana_kerja_dan_anggaran_tahunan",
 "kebermanfaatanaspek":"kebermanfaatan_aspek","tunggunya":
 ↳"tunggunya","silahkan":"silakan","oresec":"foresec",
 "mutinasionalinternasional":"multinasional_internasional","tercapaikesesuain":
 ↳"tercapai_kesesuaian","pelacakanperekaman":
 ↳"pelacakan_perekaman","matakuliah":"mata_kuliah",
 "stimik":"sekolah_tinggi_ilmu_manajemen_informatika_dan_komputer","sepnop":
 ↳"sepuluh_nopember","ikuikt":
 ↳"indikator_kinerja_utama_indikator_kinerja_tambahan",
 "pelbelajaran":"pembelajaran","dijelaskantetapi":"dijelaskan_tetapi","tingat":
 ↳"tingkat","katagari":"kategori",
 "keberadaanya":"keberadaannya","adalahdiperoleh":"adalah_diperoleh","perta":
 ↳"pertama","dikategorikan":"dikategorikan",
 "utuk":"untuk","dsetkan":"disertakan","standeriku":
 ↳"standar_indikator_kinerja_utama","prosentase":"persentase",
 "nasionalsedangkan":"nasional_sedangkan","tekait":"terkait","crosscek":
 ↳"cross_check","kiurang":"kurang",
 "kompetisilomba":"kompetisi_lomba","acc":"accept","diuukur":
 ↳"diukur","keterlaksannan":"keterlaksanaan","kemabali":"kembali",
 "kesesuaiakn":"kesesuaian","psnya":"program_studi","utaraperingkat":
 ↳"utara_peringkat","gbkperingkat":"gbk_peringkat","bahw":"bahwa","rerarta":
 ↳"rerata",

"mendeskrupsi":"mendeskripsikan","padalink":"pada_link","tercapaian":
 ↳"ketercapaian","pa":"pembimbing_akademik",
 "regionallokal":"regional_lokal","intelektual":"intelektual","pdki":
 ↳"pangkalan_data_kekayaan_intelektual","butirbutir":"butir_butir",
 "penejelasan":"penjelasan","ilkom":"ilmu_komputer","padangsidimpuan":
 ↳"padang_sidimpuan","didata":"di_data","kehlilian":"keahlian","lulusn":
 ↳"lulusan",
 "finalisdan":"finalis_dan","usni":
 ↳"universitas_satya_negara_indonesia","ekternal":"eksternal","khususkarena":
 ↳"khusus_karena","runnerup":"runner_up",
 "edupreuner":"edupreneur","sebagainya_tidak":"sebagainya_tidak","led":
 ↳"laporan_evaluasi_diri","ledlkps":
 ↳"laporan_evaluasi_diri_laporan_kinerja_program_studi",
 "meghasilkan":"menghasilkan","pemenuhana":"pemenuhan","sehingg":
 ↳"sehingga","kekesahihan":"kesahihan","skalan":"skala","ditetapkansemua":
 ↳"ditetapkan_semua",
 "ckelulusan":"kelulusan","sebanyak":"sebanyak","drivecek":
 ↳"drive_cek","lolakwilayah":"lolak_wilayah","adakelulusan":
 ↳"ada_kelulusan","proditeknologi":"program_studi_teknologi",
 "terpehuni":"terpenuhi","terpenhi":"terpenuhi","mahassiwa":"mahasiswa","tungg":
 ↳"tunggu","lokalnasionalinternasional":
 ↳"lokal_nasional_internasional","piagamsertifikat":"piagam_sertifikat",
 "kelulusa":"kelulusan","dokumem":"dokumen","ratarataipk":
 ↳"rata_rata_indeks_prestasi_kumulatif","berhki":
 ↳"yang_memiliki_hak_kekayaan_intelektual","upi":
 ↳"universitas_pendidikan_indonesia",
 "univ":"universitas","akhiir":"akhir","capaiantabel":"capaian_tabel","peilaian":
 ↳"penilaian","pekerjaanketerlaksanaan":
 ↳"pekerjaan_keterlaksanaan","juarapesertapanitia":"juara_peserta_panitia",
 "aklademiknon":"akademik_non","kaademik":"akademik","keseuain":
 ↳"kesesuaian","kalrfikasi":"klarifikasi","kkniobeskni":
 ↳"kerangka_kualifikasi_nasional_indonesia_outcome_based_education_standar_kompetensi_kerja_n",
 "samapai":"sampai","kesesuaikan":"kesesuaian","tautantautan":
 ↳"tautan_tautan","engisi":"mengisi","akreditas":"akreditasi","pembelajaran":
 ↳"pembelajaran","dikur":"diukur","presntasi":"presentasi","unuk":
 ↳"untuk","sepeti":"seperti",
 "pelaksaaan":"pelaksanaan","pendidika":"pendidikan","thun":"tahun","lylusan":
 ↳"lulusan","keterlaksanaanya":"keterlaksanaannya",
 "pihakpihak":"pihak_pihak","unitama":
 ↳"universitas_teknologi_akba_makassar","memilik":"memiliki","unsika":
 ↳"universitas_singaperbangsa_karawang",
 "darisama":"dari_sama","terkahir":"terakhir","studyratarata":
 ↳"study_rata_rata","almuni":"alumni","angakatan":
 ↳"angkatan","terlacakterdapat":"terlacak_terdapat",
 "aspekapa":"aspek_apa","pertanyaaan":"pertanyaan","infokomps":
 ↳"infokom_program_studi","mengujur":"mengukur","miyaitu":"mi_yaitu",

"dilaksananakan":"dilaksanakan","tuingkat":"tingkat","nasonal":
↳"nasionaI","presteasi":"prestasi",
"tetang":"tentang","terjad":"terjadi","pemurunan":"penurunan","menadi":
↳"menjadi","peremkaman":"perekaman","rendahkurang":"rendah_kurang","perekama":
↳"perekaman","llusan":"lulusan","lulusanpada":"lulusan_pada","siginifikan":
↳"signifikan","karenkan":"karenakan","unesa":
↳"universitas_negeri_surabaya","perkerjaan":"pekerjaan","pemutahiran":
↳"pemutakhiran",
"dilakukankonfirmasi":"dilakukan_konfirmasi","mahsa":"masa","rataata":
↳"rata_rata",
"pelaksnaan":"pelaksanaan","tidaada":"tidak_ada","askpek":"aspek","tentan":
↳"tentang","kemanpuan":"kemampuan","bulanadapun":"bulan_adapun","ulusan":
↳"lulusan","infokomkepuasan":"infokom_kepuasan","wilayahnasionaI":
↳"wilayah_nasionaI",
"nformatika":"informatika","daengan":"dengan","dilaksakan":
↳"dilaksanakan","kuisoner":"kuesioner","adala":"adalah","masayang":
↳"masa_yang","pemerintahdalam":"pemerintah_dalam","amsa":"masa","tudi":
↳"studi","mendapatkan":"mendapatkan","lkti":
↳"lomba_karya_tulis_ilmiah","psychoferia":"psychofrenia","sigra":
↳"sistem_terintegrasi","kalrifikasin":"klarifikasi","terupsat":
↳"terpusat","diaksesbroken":"diakses_broken","keilmuankeahlian":
↳"keilmuan_keahlian",
"dilingkungan":"di_lingkungan","dimatakuliah":"di_mata_kuliah","tracertubacid":
↳"tracer_ub_ac_id","rataarat":"rata_rata",
"dilakukanperlu":"dilakukan_perlu","emndapat":"mendapat","mendapar":
↳"mendapat","tpi":"tapi","diidentifikasi":"diidentifikasi","pad":"pada","oba":
↳"outcome_based_assessment","acmis":
↳"academic_management_information_system","almni":"alumni","wirausaa":
↳"wirausaha","mekanime":"mekanisme",
"pemenuhancapaianpembelajaranlulusancplratarataipkprestasi":
↳"pemenuhan_capaian_pembelajaran_lulusan_rata_rata_indeks_prestasi_kumulatif_prestasi","reke
↳"rekayasa","komputerbekerja":"komputer_bekerja","akademiknonakademik":
↳"akademik_non_akademik","upp":"unit_pengelola_program_studi",
"keterlaksanaan":"keterlaksanaan","idikator":"indikator","penetatpan":
↳"penetapan","disajika":"disajikan","memperolah":"memperoleh","nadhlatul":
↳"nahdlatul","tachnopreneur":"technopreneur","shasil":"hasil","waktuvisual":
↳"waktu","bersma":"bersama","isuisu":"isu_isu","semester":
↳"semester","sedangkah":"sedangkan","peenuhan":"pemenuhan",
"careersitsacidtracer":"careersits_ac_id_tracer","standariku":
↳"standar_indikator_kinerja_utama","pelaksanaan":"pelaksanaan","taraget":
↳"target","melaksanakan":"melaksanakan","salama":"selama","kesesuaiannya":
↳"kesesuaiannya","kesesusian":"kesesuaian",
"icts":
↳"international_conference_on_information_and_communication_technology_and_systems","sharepe
↳"sharepedia_id",

```

"kompetisikompetisi":"kompetisi_kompetisi","wa":"whatsapp","ebanyak":
    ↳"sebanyak","tsnya":"tracer_study_nya",
"lipihki":
    ↳"lembaga_inovasi_penulisan_ilmiah_dan_hak_kekayaan_intelektual","standarikuikt":
    ↳"standar_indikator_kinerja_utama_indikator_kinerja_tambahan","negeranegara":
    ↳"negara_negara","bidangbidang":"bidang_bidang","teknologiteknologi":
    ↳"teknologi_teknologi","kegiatankegiatan":"kegiatan_kegiatan",
}

typo_dict2 = {
"di konfirmasi":"dikonfirmasi","di validasi":"divalidasi","ter indentifikasi":
    ↳"terindentifikasi","ter sematkan":"tersematkan","pdik":
    ↳"program_studi_pendidikan","ilmu iahnya":"ilmiahnya","psmtif":
    ↳"program_studi_magister_teknik_informatika","perdosentahun":
    ↳"per_dosen_tahun",
"lacak":"lacakan","di tandai":"ditandai","ter pusat":"terpusat","per baikan":
    ↳"perbaikan","ber kualitas":"berkualitas","mkom":"magister_komputer","fmipa":
    ↳"fakultas_matematika_dan_ilmu_pengetahuan_alam","intelligence":
    ↳"kecerdasan","puslit":"pusat_penelitian","unmuh":
    ↳"universitas_muhammadiyah","pbi":
    ↳"pendidikan_bahasa_inggris","hilirisasipenerapan":
    ↳"hilirisasi_penerapan","matakuliah":"mata_kuliah",
"unsiq":"universitas_sains_al_quran","uppsprodi":
    ↳"unit_pengelola_program_studi","keseuaian":"kesesuaian","kebijakankebijakan":
    ↳"kebijakan_kebijakan","indicator":"indikator","procedure":
    ↳"prosedur","programprogram":"program_program","ketrampilan":
    ↳"keterampilan","pengmas":"pengabdian_masyarakat",
"ugn":"universitas_graha_nusantara","mtelah":"telah","kepahahaman":
    ↳"kepahahaman","meningkatkanmengoptimalisasi":
    ↳"meningkatkan_mengoptimalisasi","noninfokom":"non_infokom","bindonesia":
    ↳"bahasa_indonesia",
"mi":"manajemen_informatika","renop":"rencana_operasi","sndikti":
    ↳"standar_nasional_pendidikan_tinggi",
}

DICTS = [replace_weird_words1, replace_weird_words2, replace_weird_words3,
    ↳typo_dict1, typo_dict2]

# Merge dicts, drop empty / self-mapping entries
DICTS = [replace_weird_words1, replace_weird_words2, replace_weird_words3,
    ↳typo_dict1, typo_dict2]
NORM_MAP = {}
for _d in DICTS:
    for _k, _v in _d.items():
        _k = _k.strip().lower(); _v = _v.strip()

```

```

        if _k and _v and _k != _v:
            NORM_MAP[_k] = _v

# Longest keys first so multi-word phrases match before their parts.
_NORM_KEYS = sorted(NORM_MAP.keys(), key=len, reverse=True)
_NORM_RE = re.compile(r'\b(' + '|'.join(re.escape(k) for k in _NORM_KEYS) +
    ↪r')\b', re.IGNORECASE)
print(f'Normalization map: {len(NORM_MAP)} entries ({sum(1 for k in NORM_MAP if
    ↪" " in k)} multi-word).')

def normalize_singkatan_typo(text):
    return _NORM_RE.sub(lambda m: NORM_MAP[m.group(0).lower()], text)

def clean_text(text):
    if pd.isna(text) or text == '':
        return ''
    text = str(text)
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)
    text = re.sub(r'@\w+', '', text)
    text = _emoji.sub('', text)
    text = ' '.join(text.split())
    return text.strip()

def preprocess_pipeline(text):
    """clean -> lowercase -> singkatan/typo normalization (the provided
    ↪dictionaries)."""
    if pd.isna(text) or text == '':
        return ''
    return normalize_singkatan_typo(clean_text(str(text)).lower()).strip()

# quick check
_ex = 'IPK mhs prodi TI sdh sesuai dgn standart, namum detil ts belum lengkap.'
print('Example:')
print('  in :', _ex)
print('  out:', preprocess_pipeline(_ex))

```

Normalization map: 701 entries (9 multi-word).

Example:

```

in : IPK mhs prodi TI sdh sesuai dgn standart, namum detil ts belum lengkap.
out: ipk mahasiswa program_studi teknologi_informasi sudah sesuai dengan
standar, namun detail tracer_study belum lengkap.

```

8.7 Load the baseline's saved splits

```
[ ]: # Load the EXACT train/test split the baseline saved to disk.
def load_split(name, task, which):
    path = f'{SPLITS_DIR}/{name}_{which}.xlsx'          # e.g. .../ACD_AK_train.
    ↪xlsx
    df = pd.read_excel(path)                            # read that saved split
    ↪back in
    for col in ASPECT_COLUMNS:
        df[col] = pd.to_numeric(df[col], errors='coerce') # ints for ACD,
    ↪NaN-able floats for ASC
        if task == 'ACD':
            df[col] = df[col].astype(int)                # ACD labels are strict 0/
    ↪1
    print(f' loaded {which}: {path} ({len(df)} rows)')
    return df.reset_index(drop=True)
```

8.8 Thesaurus + augmentation + few-shot fill

```
[ ]: import re as _re
from collections import defaultdict

def _tnorm(v): return ' '.join(str(v).strip().split()).lower() # normalise
    ↪text: trim + collapse spaces + lowercase

def _good_term(v):
    n = _tnorm(v)
    if not n or len(n) < SYN_MIN_CH or len(n) > SYN_MAX_CH: return False #
    ↪too short / too long
    if len(n.split()) > SYN_MAX_WORDS: return False #
    ↪too many words
    if any(ch in str(v) for ch in '[]*#'): return False #
    ↪contains markup junk
    if any(b in n for b in _BAD_SUBSTR): return False #
    ↪contains a banned substring
    return True

_CMAP = {'positif': 0, 'negatif': 1, 'netral': 2} # sentiment word -> class
    ↪number

# Build the ACD (synonym) thesaurus: for each aspect, the list of synonym
    ↪phrases from the Excel files.
def build_acd_thesaurus(files):
    """{aspect_short: [synonym phrases]} from Sinonim/Padanan grouped by
    ↪Istilah Aspek."""
```

```

    thes = {s: [] for s in ASPECT_SHORTS}; seen = {s: set() for s in
↳ASPECT_SHORTS}
    for fn in files:                                     # each validated
↳thesaurus file
        df = pd.read_excel(f'{ACD_THESAURUS_DIR}/{fn}')
        for _, r in df.iterrows():                       # each synonym row
            asp = _tnorm(r.get('Istilah Aspek'))           # which aspect this
↳row belongs to
            if asp not in thes: continue                 # skip aspects we
↳don't track
            for v in [r.get('Sinonim/Padanan'), r.get('Istilah Aspek')]: # the
↳synonym + the term itself
                if _good_term(v) and _tnorm(v) not in seen[asp]:
                    seen[asp].add(_tnorm(v)); thes[asp].append(str(v).strip())
↳# add if new + valid
            thes = {k: sorted(v, key=len, reverse=True) for k, v in thes.items()} #
↳longest first (match greedily later)
            print(' ACD thesaurus:', {k[:18]: len(v) for k, v in thes.items()})
            return thes

# Build the ASC (sentiment) thesaurus: per aspect + class, pairs of (specific
↳indicator -> general expression).
def build_asc_thesaurus(files, dataset_name):
    """{aspect_short: {class: [(indikator, generalisasi)]}} - specific ->
↳general expressions.
    files: the 8 per-aspect Excel files (same set for AK & AL).
    dataset_name: 'ASC_AK' or 'ASC_AL' - each file's rows are filtered to this
↳stage via its 'Dataset' column.
    """
    thes = defaultdict(lambda: {0: [], 1: [], 2: []}); seen = set()
    for fn in files:
        df = pd.read_excel(f'{ASC_THESAURUS_DIR}/{fn}')
        if 'Dataset' in df.columns:                       #
↳keep only this stage's rows
            df = df[df['Dataset'].astype(str).str.strip().str.upper() ==
↳dataset_name.upper()]
            for _, r in df.iterrows():
                asp = _tnorm(r.get('Istilah Aspek')); cls = _tnorm(r.get('Kelas
↳Sentimen'))
                if asp not in ASPECT_SHORTS or cls not in _CMAP: continue #
↳skip unknown aspect/class
                ind = str(r.get('Ekspresi Indikator Sentimen')).strip() # the
↳specific phrase in comments
                gen = str(r.get('Ekspresi Generalisasi')).strip() # its
↳generalised form

```

```

        if not _good_term(ind) or not _good_term(gen) or _tnorm(ind) == _tnorm(gen): continue
        key = (asp, _CMAP[cls], _tnorm(ind))
        if key in seen: continue
        skip duplicates
        seen.add(key); thes[asp][_CMAP[cls]].append((ind, gen))
        for a in ASPECT_SHORTS: thes[a]
        make sure every aspect key exists
        print('  ASC thesaurus:', {a[:18]: (len(thes[a][0]), len(thes[a][1]), len(thes[a][2])) for a in ASPECT_SHORTS})
        return thes

# Swap the longest matching term in a comment for a different synonym that isn't already there.
def _syn_replace(comment, terms, rng):
    """Replace the LONGEST present term with another synonym not already in the comment."""
    low = comment.lower()
    present = sorted([t for t in terms if t.lower() in low], key=len, reverse=True) # terms found, longest first
    for t in present:
        alts = [a for a in terms if a.lower() != t.lower() and a.lower() not in low] # candidate replacements
        rng.shuffle(alts)
        # random choice
        if alts:
            aug = _re.sub(_re.escape(t), alts[0], comment, count=1, flags=_re.IGNORECASE) # replace one occurrence
            if aug.lower() != comment.lower():
                # only if it changed
                return aug
    return None
    # nothing to swap -> None

# ACD augmentation: swap one keyword belonging to a relevant aspect of this comment.
def augment_acd(comment, src_row, target_aidx, acd_thes, rng):
    """Take the comment's relevant aspects (target first if relevant) and swap one keyword."""
    relevan = [j for j, col in enumerate(ASPECT_COLUMNS) if int(src_row[col]) == 1] # aspects marked relevant
    order = ([target_aidx] if target_aidx in relevan else []) + [j for j in relevan if j != target_aidx] # try target first
    for j in order:
        out = _syn_replace(comment, acd_thes.get(ASPECT_SHORTS[j], []), rng)
        # attempt a swap

```

```

        if out is not None:
            return out
        ↪ # success
    return None
    ↪ # no swap possible

# ASC augmentation: replace a specific sentiment phrase with its generalised
↪ version.
def augment_asc(comment, target_aidx, lv, asc_thes, rng):
    """Replace an Ekspresi Indikator Sentimen (this aspect + class) with its
    ↪ Ekspresi Generalisasi."""
    pairs = asc_thes.get(ASPECT_SHORTS[target_aidx], {}).get(lv, [])
    ↪ (indicator, general) pairs for this cell
    low = comment.lower()
    present = sorted([(i, g) for (i, g) in pairs if i.lower() in low],
    ↪ key=lambda x: -len(x[0])) # ones found, longest first
    for ind, gen in present:
        if gen.lower() in low:
            ↪ skip if the general form is already there
            continue
        aug = _re.sub(_re.escape(ind), gen, comment, count=1, flags=_re.
        ↪ IGNORECASE) # do the replacement
        if aug.lower() != comment.lower():
            return aug
    return None

# BUILD THE THESAURUS FIT SET.
# Pass 1 is IDENTICAL to the baseline (same seed -> same sampled comments).
# Pass 2 is the ONLY difference: it tops up any cell still under n with
↪ augmented sentences.
def build_fit_rows(train_df, task, text_col, n, seed, thes):
    rng = random.Random(seed)
    ↪ baseline -> pass 1 picks the same rows
    pool = train_df.reset_index(drop=True)
    label_vals = [0, 1] if task == 'ACD' else [0, 1, 2]
    names = ACD_LABEL_NAMES if task == 'ACD' else ASC_LABEL_NAMES

    # pass 1: real per-(aspect,class) union
    real_keep = set()
    for a, col in enumerate(ASPECT_COLUMNS):
        for lv in label_vals:
            idxs = pool.index[pool[col] == (lv if task == 'ACD' else
            ↪ float(lv))].tolist() # rows with this class
            if not idxs: continue
            real_keep.update(rng.sample(idxs, n) if len(idxs) >= n else idxs)
            ↪ # up to n, union

```

```

real_df = pool.loc[sorted(real_keep)].reset_index(drop=True)
↳ # the shared real fit set

# pass 2: fill deficits with augmentation (ONLY the starved cells get
↳extra rows)
aug_rows, aug_log = [], []
print(f'\n --- augmenting to fill few-shot cells ({task}) ---')
for a, (col, short) in enumerate(zip(ASPECT_COLUMNS, ASPECT_SHORTS)):
    for lv in label_vals:
        have = int((real_df[col] == (lv if task == 'ACD' else float(lv))).
↳sum()) # real examples already present
        deficit = n - have
↳ # how many short of n we are
        if deficit <= 0: continue
↳ # already at/above n -> skip
        src_idx = pool.index[pool[col] == (lv if task == 'ACD' else
↳float(lv))].tolist() # sources to augment from
        if not src_idx: continue
↳ # nothing to base augmentation on
        for k in range(deficit):
↳ # make one augmented row per missing slot
            src = pool.loc[src_idx[k % len(src_idx)]]
↳ # cycle through the few available sources
            ori = str(src['comment_original'])
↳ # the original comment
            aug = augment_acd(ori, src, a, thes, rng) if task == 'ACD' else
↳augment_asc(ori, a, lv, thes, rng) # try to reword
            tag = '' if aug is not None else ' (duplicate: no thesaurus
↳match)' # note if no swap was possible
            if aug is None:
                aug = ori
↳ # fallback: keep the original (a duplicate)
            row = {'comment_original': aug, 'text_col':
↳preprocess_pipeline(aug)} # build the new training row
            for c2 in ASPECT_COLUMNS:
                row[c2] = src[c2]
↳ # copy ALL 8 labels from the source
            aug_rows.append(row)
            aug_log.append({'aspect': short, 'class': names[lv], 'original':
↳ori, 'augmented': aug}) # log for the thesis
            print(f' [{short} / {names[lv]}]{tag}')
            print(f'     ori: {ori}')
            print(f'     aug: {aug}')
        fit = pd.concat([real_df, pd.DataFrame(aug_rows)], ignore_index=True) if
↳aug_rows else real_df # real + augmented

```

```

    print(f'\n Fit set: {len(real_df)} real + {len(aug_rows)} augmented = {len(fit)} rows')
    return fit, aug_log

```

8.9 Dataset

```

[ ]: class ABSAMultiHeadDataset(Dataset):
    '''One comment -> input_ids/mask + label vector of length NUM_ASPECTS.
        ASC absent-aspect (NaN) -> -100 so it is ignored by that head's loss.'''
    def __init__(self, df, text_col, task, tokenizer, max_len):
        self.texts = df[text_col].astype(str).tolist() # the cleaned comment
    texts
        self.task = task
        labels = []
        for _, r in df.iterrows(): # build one label row
    per comment
            row = []
            for col in ASPECT_COLUMNS: # one entry per
    aspect (8 total)
                v = r[col]
                if isinstance(v, float) and np.isnan(v):
                    row.append(-100) # -100 = "no label
    here" -> loss will skip this head
                else:
                    row.append(int(v)) # otherwise the class
    index
            labels.append(row)
            self.labels = np.array(labels, dtype=np.int64)
            self.tokenizer = tokenizer; self.max_len = max_len
        def __len__(self):
            return len(self.texts) # how many comments
    in this dataset
        def __getitem__(self, i):
            enc = self.tokenizer(self.texts[i], add_special_tokens=True,
    max_length=self.max_len,
                                padding='max_length', truncation=True,
    return_tensors='pt') # tokenise one comment
            return {'input_ids': enc['input_ids'].flatten(), # token IDs
    the model reads
                    'attention_mask': enc['attention_mask'].flatten(), # 1 for real
    tokens, 0 for padding
                    'labels': torch.tensor(self.labels[i], dtype=torch.long)} #
    the 8-aspect label vector

```

8.10 Model — shared IndoBERT + per-aspect heads

```
[ ]: class IndoBERT_MultiHead_ABSA(nn.Module):
    def __init__(self, num_aspects, num_labels_per_head, dropout_rate=0.3):
        super().__init__()
        self.bert = BertModel.from_pretrained(MODEL_CHECKPOINT)  # the shared
        ↪IndoBERT encoder
        h = self.bert.config.hidden_size  # size of
        ↪BERT's output vector (768)
        self.dropout = nn.Dropout(dropout_rate)  #
        ↪regularisation before the heads
        # ONE small linear classifier per aspect (8 heads). Each head decides
        ↪for its own aspect only.
        self.heads = nn.ModuleList([nn.Linear(h, num_labels_per_head) for _ in
        ↪range(num_aspects)])
        def forward(self, input_ids, attention_mask):
            pooled = self.bert(input_ids=input_ids, attention_mask=attention_mask).
            ↪pooler_output # encode the comment ONCE
            pooled = self.dropout(pooled)  # apply
            ↪dropout to that shared vector
            return [head(pooled) for head in self.heads]  # each head
            ↪reads the SAME vector -> 8 sets of logits
```

8.11 Per-aspect class weights

```
[ ]: def compute_head_weights(fit_df, task):
    '''Balanced class weights per aspect head, computed on available (non -100)
    ↪labels.'''
    n_lab = 2 if task == 'ACD' else 3  # 2 classes for ACD, 3
    ↪for ASC
    weights = []
    for col in ASPECT_COLUMNS:  # compute a weight
        ↪vector for each aspect head
        y = fit_df[col].dropna().astype(int).values  # the labels actually
        ↪present for this aspect
        present = np.unique(y)  # which classes appear
        ↪at all
        w = np.ones(n_lab, dtype=np.float32)  # default weight 1.0
        ↪for every class
        if len(present) >= 2:  # only balance if at
            ↪least 2 classes are present
            cw = compute_class_weight('balanced', classes=present, y=y)  #
            ↪rarer class -> bigger weight
            for c, val in zip(present, cw):
                w[c] = val  # store the computed
            ↪weight in its class slot
```

```

        weights.append(torch.tensor(w, dtype=torch.float32).to(device)) # move
↪to GPU/CPU as a tensor
        return weights # one weight vector per
↪aspect, used in the loss

```

8.12 Training (weighted CE, no validation)

```

[ ]: def train_model(fit_df, task, text_col, seed):
    set_all_seeds(seed) # reset all RNGs so
↪training is reproducible
    n_lab = 2 if task == 'ACD' else 3 # classes per head: 2
↪(ACD) or 3 (ASC)
    ds = ABSAMultiHeadDataset(fit_df, text_col, task, tokenizer, MAX_SEQ_LEN)
↪# wrap the fit set
    loader = DataLoader(ds, batch_size=BATCH_SIZE, shuffle=True)
↪# feed it in shuffled batches

    model = IndoBERT_MultiHead_ABSA(NUM_ASPECTS, n_lab, DROPOUT).to(device)
↪# build the 8-head model
    head_weights = compute_head_weights(fit_df, task)
↪# per-aspect class weights
    # One weighted cross-entropy loss per aspect. ignore_index=-100 -> absent
↪labels don't contribute.
    loss_fns = [nn.CrossEntropyLoss(weight=head_weights[a], ignore_index=-100)
                 for a in range(NUM_ASPECTS)]

    optimizer = AdamW(model.parameters(), lr=LEARNING_RATE,
↪weight_decay=WEIGHT_DECAY) # flat LR, no scheduler

    history = []
    for epoch in range(EPOCHS): # train for a fixed number
↪of epochs (5)
        model.train(); run = 0.0 # training mode; running
↪loss accumulator
        pbar = tqdm(loader, desc=f' epoch {epoch+1}/{EPOCHS}', leave=False)
        for batch in pbar: # one mini-batch of
↪comments at a time
            ids = batch['input_ids'].to(device) # token IDs -> GPU
            mask = batch['attention_mask'].to(device) # attention mask -> GPU
            lab = batch['labels'].to(device) # true labels, shape
↪[batch, 8]
            logits_list = model(ids, mask) # forward pass -> 8 heads'
↪logits
            # total loss = sum of each head's loss (each head compares its
↪column of labels)

```

```

        loss = sum(loss_fns[a](logits_list[a], lab[:, a]) for a in
↳range(NUM_ASPECTS))
        optimizer.zero_grad(); loss.backward()    # clear old gradients,
↳back-propagate
        optimizer.step()                        # update the weights
        run += loss.item(); pbar.set_postfix(loss=loss.item())
        ep_loss = run / len(loader)              # average loss this epoch
        history.append(ep_loss)
        print(f' epoch {epoch+1}: train_loss={ep_loss:.4f}')
        return model, history                    # trained model + loss
↳curve

```

8.13 Prediction

```

[ ]: @torch.no_grad()                          # no gradients needed when
↳only predicting
def predict(model, df, text_col, task):
    model.eval()                                # evaluation mode (turns
↳off dropout)
    ds = ABSAMultiHeadDataset(df, text_col, task, tokenizer, MAX_SEQ_LEN)
    loader = DataLoader(ds, batch_size=BATCH_SIZE, shuffle=False) # keep order
↳so preds line up with rows
    preds, labels = [], []
    for batch in loader:
        ids = batch['input_ids'].to(device)
        mask = batch['attention_mask'].to(device)
        logits_list = model(ids, mask)           # 8 heads' logits for this
↳batch
        # for each head take the highest-scoring class -> one prediction per
↳aspect -> [batch, 8]
        p = torch.stack([torch.argmax(l, dim=1) for l in logits_list], dim=1)
        preds.append(p.cpu().numpy())            # collect predictions
        labels.append(batch['labels'].numpy())    # collect the true labels
        return np.vstack(preds), np.vstack(labels) # stack all batches into
↳full arrays

```

8.14 Metrics — macro-F1 + weighted-F1

```

[ ]: def _valid(true, pred, task):
    if task == 'ASC':
        m = true != -100
        return true[m], pred[m]
    return true, pred

def print_metrics(preds, labels, task, name):
    label_names = ACD_LABEL_NAMES if task == 'ACD' else ASC_LABEL_NAMES

```

```

idxs = list(range(len(label_names)))
print(f'\n{"="*64}\n  {name}\n{"="*64}')
macro_list, weighted_list = [], []
for a, asp in enumerate(ASPECT_SHORTS):
    true, pred = _valid(labels[:, a], preds[:, a], task)
    if len(true) == 0:
        print(f'\nAspect: {asp} (no labels)'); continue
    p, r, f, _ = precision_recall_fscore_support(true, pred, labels=idxs,
↪average=None, zero_division=0)
    mf1 = f1_score(true, pred, average='macro', zero_division=0)
    wf1 = f1_score(true, pred, average='weighted', zero_division=0)
    macro_list.append(mf1); weighted_list.append(wf1)
    print(f'\nAspect: {asp}')
    for i, ln in enumerate(label_names):
        print(f'  {ln:<22} P={p[i]:.3f} R={r[i]:.3f} F1={f[i]:.3f}')
    print(f' Macro F1 {mf1:.4f} | Weighted F1 {wf1:.4f}')
    mean_macro = float(np.mean(macro_list)) if macro_list else float("nan")
    mean_weight = float(np.mean(weighted_list)) if weighted_list else
↪float("nan")
    print(f'\n MEAN Macro F1: {mean_macro:.4f} | MEAN Weighted F1:
↪{mean_weight:.4f}')
    return {'mean_macro_f1': mean_macro, 'mean_weighted_f1': mean_weight,
            'per_aspect_macro': dict(zip(ASPECT_SHORTS, macro_list)),
            'per_aspect_weighted': dict(zip(ASPECT_SHORTS, weighted_list))}

def plot_cms(preds, labels, task, name):
    label_names = ACD_LABEL_NAMES if task == 'ACD' else ASC_LABEL_NAMES
    idxs = list(range(len(label_names)))
    fig, axes = plt.subplots(2, 4, figsize=(24, 12))
    for a, (asp, ax) in enumerate(zip(ASPECT_SHORTS, axes.flatten())):
        true, pred = _valid(labels[:, a], preds[:, a], task)
        if len(true) == 0:
            ax.set_title(f'{asp}\n(no data)', fontsize=9); ax.axis('off');
↪continue
        cm = confusion_matrix(true, pred, labels=idxs)
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                    xticklabels=label_names, yticklabels=label_names, ax=ax,
                    cbar=True, linewidths=0.5, linecolor='white')
        ax.set_title(asp, fontsize=9); ax.set_xlabel('Pred', fontsize=8); ax.
↪set_ylabel('True', fontsize=8)
        ax.tick_params(labelsize=7)
        plt.suptitle(f'Confusion Matrices - {name}', fontsize=13, y=1.01)
        plt.tight_layout()
        fname = f'{BASE_PATH}/results/thesaurus/cm_{name}.png'; os.makedirs(os.path.
↪dirname(fname), exist_ok=True); plt.savefig(fname, dpi=150,
↪bbox_inches='tight'); plt.show()

```

```

print('Saved ->', fname)

def plot_training(history, name):
    plt.figure(figsize=(7, 4))
    plt.plot(range(1, len(history) + 1), history, marker='o', linewidth=2,
    ↪color='#2b6cb0')
    plt.title(f'Training Loss - {name}', fontsize=12)
    plt.xlabel('Epoch'); plt.ylabel('Summed weighted CE loss')
    plt.xticks(range(1, len(history) + 1)); plt.grid(True, alpha=0.3)
    fname = f'{BASE_PATH}/results/thesaurus/training_{name}.png'
    os.makedirs(os.path.dirname(fname), exist_ok=True)
    plt.tight_layout(); plt.savefig(fname, dpi=150, bbox_inches='tight'); plt.
    ↪show()
    print('Saved ->', fname)

```

8.15 Save predictions to Excel

```

[ ]: def save_predictions_excel(test_df, preds, labels, text_col, task, name):
    names = ACD_LABEL_NAMES if task == 'ACD' else ASC_LABEL_NAMES
    nm = lambda v: '' if int(v) == -100 else names[int(v)]
    rows = []
    for i in range(len(test_df)):
        r = {'comment': str(test_df.iloc[i][text_col])}
        for a, asp in enumerate(ASPECT_SHORTS):
            r[f'{asp} | true'] = nm(labels[i, a])
            r[f'{asp} | pred'] = nm(preds[i, a])
        rows.append(r)
    out = pd.DataFrame(rows)
    os.makedirs(f'{BASE_PATH}/results/thesaurus', exist_ok=True)
    path = f'{BASE_PATH}/results/thesaurus/predictions_{name}_weighted_baseline.
    ↪xlsx'
    out.to_excel(path, index=False)
    print(f' Saved {len(out)} test comments + predictions -> {path}')
    return out

```

8.16 Runner

```

[ ]: def run_experiment(exp):
    name, task, text_col = exp['name'], exp['task'], exp['text_col']
    print(f'\n{"#"*66}\n {name} [{task} / THESAURUS / few-shot {N_PER_CLASS} /
    ↪ same test set]\n{"#"*66}')

    # Load the SAME train/test split the baseline saved -> guarantees an
    ↪apples-to-apples comparison.
    train_df = load_split(name, task, 'train')
    test_df = load_split(name, task, 'test')

```

```

    # Build this stage's thesaurus, then fill the few-shot cells with
    ↪ augmentation.
    files = (ACD_THESAURUS_FILES if task == 'ACD' else
    ↪ ASC_THESAURUS_FILES)[name] # which thesaurus files for this run
    thes = build_acd_thesaurus(files) if task == 'ACD' else
    ↪ build_asc_thesaurus(files, name)
    fit_df, aug_log = build_fit_rows(train_df, task, text_col, N_PER_CLASS,
    ↪ SEED, thes) # real + augmented rows

    model, history = train_model(fit_df, task, text_col, SEED) #
    ↪ fine-tune on the augmented fit set

    print('\n Evaluating on the shared TEST set...')
    preds, labels = predict(model, test_df, text_col, task) # SAME
    ↪ test set as baseline
    metrics = print_metrics(preds, labels, task, name)
    plot_cms(preds, labels, task, name)
    plot_training(history, name)
    save_predictions_excel(test_df, preds, labels, text_col, task, name)

    # Save every augmented sentence so they can be inspected / reported in the
    ↪ thesis.
    if aug_log:
        os.makedirs(f'{BASE_PATH}/results/thesaurus', exist_ok=True)
        pd.DataFrame(aug_log).to_excel(f'{BASE_PATH}/results/thesaurus/
    ↪ augmented_{name}_thesaurus.xlsx', index=False)
        print(f' Saved {len(aug_log)} augmented sentences -> results/
    ↪ augmented_{name}_thesaurus.xlsx')

    del model; gc.collect() # free
    ↪ memory for the next run
    if torch.cuda.is_available(): torch.cuda.empty_cache()
    return {'metrics': metrics, 'preds': preds, 'labels': labels, 'n_fit':
    ↪ len(fit_df), 'n_test': len(test_df)}

```

8.17 Run all four experiments (ACD AK/AL, ASC AK/AL)

```

[ ]: ALL_RESULTS = {}
for exp in ACD_EXPERIMENTS + ASC_EXPERIMENTS:
    ALL_RESULTS[exp['name']] = run_experiment(exp)

print('\nDone. Mean macro-F1 / weighted-F1:')
for k, v in ALL_RESULTS.items():
    m = v['metrics']
    print(f' {k:<8} macro={m["mean_macro_f1"]:.4f}
    ↪ weighted={m["mean_weighted_f1"]:.4f} ({v["n_aug"]} aug)')

```

```
#####
ACD_AK [ACD / THESAURUS / few-shot 15 / same test set]
#####
loaded train:
/content/drive/MyDrive/results/baseline/splits/ACD_AK_train.xlsx (159 rows)
loaded test:
/content/drive/MyDrive/results/baseline/splits/ACD_AK_test.xlsx (41 rows)
ACD thesaurus: {'capaian pembelajar': 110, 'indeks prestasi ku': 53,
'kelulusan tepat wa': 125, 'masa tunggu lulusa': 96, 'pelacakan dan pere': 147,
'prestasi mahasiswa': 192, 'karya dengan hak k': 49, 'kesesuaian bidang ': 195}

--- augmenting to fill few-shot cells (ACD) ---
Fit set: 119 real + 13 augmented = 132 rows
config.json: 0%|          | 0.00/1.53k [00:00<?, ?B/s]
pytorch_model.bin: 0%|          | 0.00/498M [00:00<?, ?B/s]
Loading weights: 0%|          | 0/199 [00:00<?, ?it/s]
epoch 1/5: 0%|          | 0/17 [00:00<?, ?it/s]
model.safetensors: 0%|          | 0.00/498M [00:00<?, ?B/s]
epoch 1: train_loss=4.6045
epoch 2/5: 0%|          | 0/17 [00:00<?, ?it/s]
epoch 2: train_loss=3.5114
epoch 3/5: 0%|          | 0/17 [00:00<?, ?it/s]
epoch 3: train_loss=2.7536
epoch 4/5: 0%|          | 0/17 [00:00<?, ?it/s]
epoch 4: train_loss=2.6339
epoch 5/5: 0%|          | 0/17 [00:00<?, ?it/s]
epoch 5: train_loss=2.2488

Evaluating on the shared TEST set...

=====
ACD_AK
=====

Aspect: capaian pembelajaran lulusan
tidak_relevan      P=0.000  R=0.000  F1=0.000
relevan            P=0.951  R=1.000  F1=0.975
Macro F1 0.4875   | Weighted F1 0.9274

Aspect: indeks prestasi kumulatif
```

tidak_relevan	P=0.571	R=0.667	F1=0.615
relevan	P=0.941	R=0.914	F1=0.928
Macro F1 0.7715		Weighted F1 0.8819	

Aspect: kelulusan tepat waktu

tidak_relevan	P=0.778	R=0.538	F1=0.636
relevan	P=0.812	R=0.929	F1=0.867
Macro F1 0.7515		Weighted F1 0.7936	

Aspect: masa tunggu lulusan

tidak_relevan	P=0.875	R=0.700	F1=0.778
relevan	P=0.909	R=0.968	F1=0.938
Macro F1 0.8576		Weighted F1 0.8985	

Aspect: pelacakan dan perekaman data lulusan

tidak_relevan	P=0.667	R=0.750	F1=0.706
relevan	P=0.826	R=0.760	F1=0.792
Macro F1 0.7488		Weighted F1 0.7582	

Aspect: prestasi mahasiswa

tidak_relevan	P=0.429	R=1.000	F1=0.600
relevan	P=1.000	R=0.625	F1=0.769
Macro F1 0.6846		Weighted F1 0.7321	

Aspect: karya dengan hak kekayaan intelektual

tidak_relevan	P=0.667	R=0.533	F1=0.593
relevan	P=0.759	R=0.846	F1=0.800
Macro F1 0.6963		Weighted F1 0.7241	

Aspect: kesesuaian bidang kerja

tidak_relevan	P=0.643	R=0.692	F1=0.667
relevan	P=0.852	R=0.821	F1=0.836
Macro F1 0.7515		Weighted F1 0.7826	

MEAN Macro F1: 0.7187 | MEAN Weighted F1: 0.8123

Saved -> /content/drive/MyDrive/results/thesaurus/cm_ACD_AK.png

Saved -> /content/drive/MyDrive/results/thesaurus/training_ACD_AK.png

Saved 41 test comments + predictions -> /content/drive/MyDrive/results/thesaurus/predictions_ACD_AK_weighted_baseline.xlsx

Saved 13 augmented sentences -> results/augmented_ACD_AK_thesaurus.xlsx

#####

ACD_AL [ACD / THESAURUS / few-shot 15 / same test set]

#####

loaded train:

/content/drive/MyDrive/results/baseline/splits/ACD_AL_train.xlsx (157 rows)

loaded test:

```
/content/drive/MyDrive/results/baseline/splits/ACD_AL_test.xlsx (43 rows)
ACD thesaurus: {'capaian pembelajar': 110, 'indeks prestasi ku': 53,
'kelulusan tepat wa': 125, 'masa tunggu lulusa': 96, 'pelacakan dan pere': 147,
'prestasi mahasiswa': 192, 'karya dengan hak k': 49, 'kesesuaian bidang ': 195}
```

```
--- augmenting to fill few-shot cells (ACD) ---
```

```
Fit set: 120 real + 16 augmented = 136 rows
```

```
Loading weights: 0%|          | 0/199 [00:00<?, ?it/s]
```

```
epoch 1/5: 0%|          | 0/17 [00:00<?, ?it/s]
```

```
epoch 1: train_loss=4.2914
```

```
epoch 2/5: 0%|          | 0/17 [00:00<?, ?it/s]
```

```
epoch 2: train_loss=3.0152
```

```
epoch 3/5: 0%|          | 0/17 [00:00<?, ?it/s]
```

```
epoch 3: train_loss=2.5054
```

```
epoch 4/5: 0%|          | 0/17 [00:00<?, ?it/s]
```

```
epoch 4: train_loss=1.9329
```

```
epoch 5/5: 0%|          | 0/17 [00:00<?, ?it/s]
```

```
epoch 5: train_loss=1.6833
```

```
Evaluating on the shared TEST set...
```

```
=====
ACD_AL
=====
```

```
Aspect: capaian pembelajaran lulusan
```

```
tidak_relevan      P=1.000  R=0.500  F1=0.667
```

```
relevan            P=0.976  R=1.000  F1=0.988
```

```
Macro F1 0.8273 | Weighted F1 0.9730
```

```
Aspect: indeks prestasi kumulatif
```

```
tidak_relevan      P=0.667  R=0.667  F1=0.667
```

```
relevan            P=0.946  R=0.946  F1=0.946
```

```
Macro F1 0.8063 | Weighted F1 0.9070
```

```
Aspect: kelulusan tepat waktu
```

```
tidak_relevan      P=0.692  R=0.900  F1=0.783
```

```
relevan            P=0.967  R=0.879  F1=0.921
```

```
Macro F1 0.8516 | Weighted F1 0.8885
```

```
Aspect: masa tunggu lulusan
```

```
tidak_relevan      P=0.733  R=0.846  F1=0.786
```

```

relevan                P=0.929  R=0.867  F1=0.897
Macro F1 0.8411  |  Weighted F1 0.8630

Aspect: pelacakan dan perekaman data lulusan
tidak_relevan          P=0.750  R=0.600  F1=0.667
relevan                P=0.806  R=0.893  F1=0.847
Macro F1 0.7571  |  Weighted F1 0.7844

Aspect: prestasi mahasiswa
tidak_relevan          P=0.733  R=0.846  F1=0.786
relevan                P=0.929  R=0.867  F1=0.897
Macro F1 0.8411  |  Weighted F1 0.8630

Aspect: karya dengan hak kekayaan intelektual
tidak_relevan          P=0.857  R=0.800  F1=0.828
relevan                P=0.897  R=0.929  F1=0.912
Macro F1 0.8699  |  Weighted F1 0.8827

Aspect: kesesuaian bidang kerja
tidak_relevan          P=0.750  R=0.818  F1=0.783
relevan                P=0.935  R=0.906  F1=0.921
Macro F1 0.8516  |  Weighted F1 0.8853

MEAN Macro F1: 0.8308  |  MEAN Weighted F1: 0.8809

Saved -> /content/drive/MyDrive/results/thesaurus/cm_ACD_AL.png
Saved -> /content/drive/MyDrive/results/thesaurus/training_ACD_AL.png
Saved 43 test comments + predictions -> /content/drive/MyDrive/results/thesaurus/predictions_ACD_AL_weighted_baseline.xlsx
Saved 16 augmented sentences -> results/augmented_ACD_AL_thesaurus.xlsx

#####
ASC_AK [ASC / THESAURUS / few-shot 15 / same test set]
#####
loaded train:
/content/drive/MyDrive/results/baseline/splits/ASC_AK_train.xlsx (158 rows)
loaded test:
/content/drive/MyDrive/results/baseline/splits/ASC_AK_test.xlsx (42 rows)
ASC thesaurus: {'capaian pembelajar': (266, 134, 87), 'indeks prestasi ku':
(298, 66, 198), 'kelulusan tepat wa': (165, 139, 209), 'masa tunggu lulusa':
(195, 85, 188), 'pelacakan dan pere': (195, 147, 142), 'prestasi mahasiswa':
(340, 83, 58), 'karya dengan hak k': (211, 118, 75), 'kesesuaian bidang ': (273,
94, 129)}

--- augmenting to fill few-shot cells (ASC) ---
Fit set: 134 real + 31 augmented = 165 rows

Loading weights: 0%|          | 0/199 [00:00<?, ?it/s]

```

```

epoch 1/5:  0%|          | 0/21 [00:00<?, ?it/s]
epoch 1: train_loss=8.8574
epoch 2/5:  0%|          | 0/21 [00:00<?, ?it/s]
epoch 2: train_loss=7.9761
epoch 3/5:  0%|          | 0/21 [00:00<?, ?it/s]
epoch 3: train_loss=6.9743
epoch 4/5:  0%|          | 0/21 [00:00<?, ?it/s]
epoch 4: train_loss=5.8708
epoch 5/5:  0%|          | 0/21 [00:00<?, ?it/s]
epoch 5: train_loss=4.8105

```

Evaluating on the shared TEST set...

```

=====
ASC_AK
=====

```

Aspect: capaian pembelajaran lulusan

```

positif          P=0.607  R=0.895  F1=0.723
negatif          P=0.750  R=0.500  F1=0.600
netral           P=0.556  R=0.312  F1=0.400
Macro F1 0.5745  |  Weighted F1 0.5791

```

Aspect: indeks prestasi kumulatif

```

positif          P=0.667  R=0.737  F1=0.700
negatif          P=0.000  R=0.000  F1=0.000
netral           P=0.462  R=0.500  F1=0.480
Macro F1 0.3933  |  Weighted F1 0.5606

```

Aspect: kelulusan tepat waktu

```

positif          P=0.696  R=0.842  F1=0.762
negatif          P=1.000  R=0.500  F1=0.667
netral           P=0.250  R=0.143  F1=0.182
Macro F1 0.5368  |  Weighted F1 0.6101

```

Aspect: masa tunggu lulusan

```

positif          P=0.737  R=0.778  F1=0.757
negatif          P=0.000  R=0.000  F1=0.000
netral           P=0.300  R=0.300  F1=0.300
Macro F1 0.3523  |  Weighted F1 0.5362

```

Aspect: pelacakan dan perekaman data lulusan

```

positif          P=0.727  R=0.889  F1=0.800

```

negatif	P=0.000	R=0.000	F1=0.000
netral	P=0.000	R=0.000	F1=0.000
Macro F1 0.2667		Weighted F1 0.5760	

Aspect: prestasi mahasiswa

positif	P=0.750	R=0.833	F1=0.789
negatif	P=0.333	R=0.250	F1=0.286
netral	P=0.429	R=0.375	F1=0.400
Macro F1 0.4917		Weighted F1 0.6184	

Aspect: karya dengan hak kekayaan intelektual

positif	P=0.667	R=0.571	F1=0.615
negatif	P=0.000	R=0.000	F1=0.000
netral	P=0.462	R=0.667	F1=0.545
Macro F1 0.3869		Weighted F1 0.5202	

Aspect: kesesuaian bidang kerja

positif	P=0.588	R=0.667	F1=0.625
negatif	P=0.000	R=0.000	F1=0.000
netral	P=0.300	R=0.273	F1=0.286
Macro F1 0.3036		Weighted F1 0.4471	

MEAN Macro F1: 0.4132 | MEAN Weighted F1: 0.5560

Saved -> /content/drive/MyDrive/results/thesaurus/cm_ASC_AK.png

Saved -> /content/drive/MyDrive/results/thesaurus/training_ASC_AK.png

Saved 42 test comments + predictions -> /content/drive/MyDrive/results/thesaurus/predictions_ASC_AK_weighted_baseline.xlsx

Saved 31 augmented sentences -> results/augmented_ASC_AK_thesaurus.xlsx

#####

ASC_AL [ASC / THESAURUS / few-shot 15 / same test set]

#####

loaded train:

/content/drive/MyDrive/results/baseline/splits/ASC_AL_train.xlsx (156 rows)

loaded test:

/content/drive/MyDrive/results/baseline/splits/ASC_AL_test.xlsx (44 rows)

ASC thesaurus: {'capaian pembelajar': (166, 48, 33), 'indeks prestasi ku': (182, 24, 115), 'kelulusan tepat wa': (103, 71, 129), 'masa tunggu lulusa': (117, 31, 110), 'pelacakan dan pere': (133, 44, 86), 'prestasi mahasiswa': (243, 23, 17), 'karya dengan hak k': (152, 26, 52), 'kesesuaian bidang ': (185, 33, 71)}

--- augmenting to fill few-shot cells (ASC) ---

Fit set: 132 real + 67 augmented = 199 rows

Loading weights: 0% | 0/199 [00:00<?, ?it/s]

epoch 1/5: 0% | 0/25 [00:00<?, ?it/s]

```

epoch 1: train_loss=7.9725
epoch 2/5: 0%|          | 0/25 [00:00<?, ?it/s]
epoch 2: train_loss=5.8686
epoch 3/5: 0%|          | 0/25 [00:00<?, ?it/s]
epoch 3: train_loss=4.7693
epoch 4/5: 0%|          | 0/25 [00:00<?, ?it/s]
epoch 4: train_loss=3.9816
epoch 5/5: 0%|          | 0/25 [00:00<?, ?it/s]
epoch 5: train_loss=3.3354

```

Evaluating on the shared TEST set...

```

=====
ASC_AL
=====

```

Aspect: capaian pembelajaran lulusan

positif	P=0.905	R=0.655	F1=0.760
negatif	P=0.400	R=0.286	F1=0.333
netral	P=0.389	R=0.875	F1=0.538
Macro F1	0.5439		Weighted F1 0.6518

Aspect: indeks prestasi kumulatif

positif	P=0.556	R=0.476	F1=0.513
negatif	P=0.000	R=0.000	F1=0.000
netral	P=0.368	R=0.467	F1=0.412
Macro F1	0.3082		Weighted F1 0.4459

Aspect: kelulusan tepat waktu

positif	P=0.545	R=0.462	F1=0.500
negatif	P=0.250	R=0.250	F1=0.250
netral	P=0.591	R=0.650	F1=0.619
Macro F1	0.4563		Weighted F1 0.5373

Aspect: masa tunggu lulusan

positif	P=0.769	R=0.526	F1=0.625
negatif	P=0.000	R=0.000	F1=0.000
netral	P=0.429	R=0.750	F1=0.545
Macro F1	0.3902		Weighted F1 0.5418

Aspect: pelacakan dan perekaman data lulusan

positif	P=0.765	R=0.619	F1=0.684
negatif	P=0.000	R=0.000	F1=0.000
netral	P=0.375	R=0.500	F1=0.429

Macro F1 0.3709 | Weighted F1 0.5575

Aspect: prestasi mahasiswa

positif	P=0.900	R=0.409	F1=0.562
negatif	P=0.333	R=0.500	F1=0.400
netral	P=0.400	R=0.889	F1=0.552
Macro F1	0.5047	Weighted F1	0.5497

Aspect: karya dengan hak kekayaan intelektual

positif	P=0.889	R=0.800	F1=0.842
negatif	P=0.000	R=0.000	F1=0.000
netral	P=0.545	R=0.750	F1=0.632
Macro F1	0.4912	Weighted F1	0.7298

Aspect: kesesuaian bidang kerja

positif	P=0.667	R=0.778	F1=0.718
negatif	P=0.000	R=0.000	F1=0.000
netral	P=0.643	R=0.600	F1=0.621
Macro F1	0.4462	Weighted F1	0.6352

MEAN Macro F1: 0.4390 | MEAN Weighted F1: 0.5811

Saved -> /content/drive/MyDrive/results/thesaurus/cm_ASC_AL.png

Saved -> /content/drive/MyDrive/results/thesaurus/training_ASC_AL.png

Saved 44 test comments + predictions -> /content/drive/MyDrive/results/thesaurus/predictions_ASC_AL_weighted_baseline.xlsx

Saved 67 augmented sentences -> results/augmented_ASC_AL_thesaurus.xlsx

Done. Mean macro-F1 / weighted-F1:

ACD_AK	macro=0.7187	weighted=0.8123	(+13 aug)
ACD_AL	macro=0.8308	weighted=0.8809	(+16 aug)
ASC_AK	macro=0.4132	weighted=0.5560	(+31 aug)
ASC_AL	macro=0.4390	weighted=0.5811	(+67 aug)

8.18 Results summary

```
[ ]: summary = pd.DataFrame([
    {'experiment': k,
     'mean_macro_f1': round(v['metrics']['mean_macro_f1'], 4),
     'mean_weighted_f1': round(v['metrics']['mean_weighted_f1'], 4),
     'n_fit': v['n_fit'], 'n_aug': v['n_aug'], 'n_test': v['n_test']}
    for k, v in ALL_RESULTS.items()
]).set_index('experiment')
print(summary.to_string())

os.makedirs(f'{BASE_PATH}/results/thesaurus', exist_ok=True)
summary.to_excel(f'{BASE_PATH}/results/thesaurus/summary_thesaurus.xlsx')
```

```
print('\nSaved -> {}/results/thesaurus/summary_thesaurus.xlsx'.  
      ↪format(BASE_PATH))
```

	mean_macro_f1	mean_weighted_f1	n_fit	n_aug	n_test
experiment					
ACD_AK	0.7187	0.8123	132	13	41
ACD_AL	0.8308	0.8809	136	16	43
ASC_AK	0.4132	0.5560	165	31	42
ASC_AL	0.4390	0.5811	199	67	44

Saved -> /content/drive/MyDrive/results/thesaurus/summary_thesaurus.xlsx

9 Tahap Analisis Metrik Terpadu - Baseline vs Tesaurus

Tahap ini menggabungkan (pooling) hasil prediksi dari kedua tahap fine-tuning sebelumnya (baseline dan tesaurus) di seluruh delapan aspek dan kedua tahap asesmen (AK, AL), lalu menghitung ulang precision, recall, dan F1-score per kelas langsung dari confusion matrix gabungan — bukan dengan merata-ratakan skor per-run — sehingga hasil akhir mencerminkan performa aktual pada seluruh data uji sekaligus. Confusion matrix pooled yang lengkap (bukan hanya diagonal) turut disimpan untuk memeriksa pola kesalahan klasifikasi, misalnya berapa banyak komentar berlabel ‘positif’ yang justru diprediksi sebagai ‘netral’.

9.1 Mount Drive

```
[ ]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

9.2 Config (must match the training stages)

```
[ ]: BASE_PATH = '/content/drive/MyDrive'    # <-- same base path the training stages
      ↪used

# label order is fixed by the training config (cell 4 there)
ACD_LABEL_NAMES = ['tidak_relevan', 'relevan']
ASC_LABEL_NAMES = ['positif', 'negatif', 'netral']

# the 8 aspects, exactly as they appear in the prediction-file column names
ASPECT_SHORTS = [
    'capaian pembelajaran lulusan',
    'indeks prestasi kumulatif',
    'kelulusan tepat waktu',
    'masa tunggu lulusan',
    'pelacakan dan perekaman data lulusan',
    'prestasi mahasiswa',
    'karya dengan hak kekayaan intelektual',
    'kesesuaian bidang kerja',
]

EXPERIMENTS = ['ACD_AK', 'ACD_AL', 'ASC_AK', 'ASC_AL']

# where each model's predictions live on Drive
DIRS = {'baseline': f'{BASE_PATH}/results/baseline',
        'thesaurus': f'{BASE_PATH}/results/thesaurus'}

# prediction filenames carry the '_weighted_baseline' suffix in BOTH folders
FILE = 'predictions_{name}_weighted_baseline.xlsx'
```

9.3 Pooling functions

```
[ ]: import os
import numpy as np, pandas as pd
from sklearn.metrics import precision_recall_fscore_support, confusion_matrix

def _task(name): return name.split('_')[0] # 'ASC_AK'
    ↪-> 'ASC'
def _names(task): return ACD_LABEL_NAMES if task == 'ACD' else ASC_LABEL_NAMES

def pooled_from_excel(path, task):
    """Flatten the 8 aspect columns of a predictions file into one pooled
    (true, pred) vector. Empty cells ('') are masked-out ASC 'absent' rows and
    are dropped -- exactly as the training stage's _valid() drops -100."""
    names = _names(task); idx = {n: i for i, n in enumerate(names)}
    df = pd.read_excel(path); t_all, p_all, skipped = [], [], 0
    for asp in ASPECT_SHORTS:
        for tv, pv in zip(df[f'{asp} | true'].fillna(''), df[f'{asp} | pred'].
    ↪fillna('')):
            tv, pv = str(tv).strip(), str(pv).strip()
            if tv == '': # absent aspect -> excluded
    ↪from scoring
                continue
            if tv not in idx or pv not in idx: # guard; should never fire
                skipped += 1; continue
            t_all.append(idx[tv]); p_all.append(idx[pv])
            if skipped: print(f' [warn] {skipped} unrecognised label cell(s) skipped
    ↪in {os.path.basename(path)}')
    return np.array(t_all), np.array(p_all)

def report_pooled(true, pred, task, title, show=True):
    """Pooled per-class precision/recall/F1 + the raw TP/support counts."""
    names, ids = _names(task), list(range(len(_names(task))))
    cm = confusion_matrix(true, pred, labels=ids) # rows=true,
    ↪cols=pred
    P, R, F, S = precision_recall_fscore_support(true, pred, labels=ids,
                                                average=None, zero_division=0)

    if show:
        print(f'\n=== {title} (pooled over {len(ASPECT_SHORTS)} aspects,
    ↪n={len(true)}) ===')
        print(f'{"class":<15}{"TP/supp":>10}{"prec":>8}{"recall":>8}{"F1":>8}')
        for i, n in enumerate(names):
            print(f'{n:<15}{f"{cm[i,i]}/{int(S[i])}":>10}{P[i]:>8.3f}{R[i]:>8.
    ↪3f}{F[i]:>8.3f}')
        print(f'{"pooled macro-F1":<15}{F.mean():>34.3f}')
        print(f'{"pooled wtd-F1":<15}{np.average(F, weights=S):>34.3f}')
```

```

        print(' (headline macro-F1 on the slides = MEAN of the 8 per-aspect_
↳macro-F1,')
        print(' a different aggregation from this pooled macro-F1)')
        return {names[i]: dict(tp=int(cm[i, i]), support=int(S[i]),
                                precision=float(P[i]), recall=float(R[i]),
↳f1=float(F[i]))
                for i in range(len(names))}

```

9.4 Run it — baseline vs thesaurus, side by side

```

[ ]: rows = [] # collect a tidy table to save at the end

for name in EXPERIMENTS:
    task = _task(name)
    pb = f'{DIRS["baseline"]}/{FILE.format(name=name)}'
    pt = f'{DIRS["thesaurus"]}/{FILE.format(name=name)}'
    try:
        b = report_pooled(*pooled_from_excel(pb, task), task, f'{name} _
↳BASELINE')
        t = report_pooled(*pooled_from_excel(pt, task), task, f'{name} _
↳THESAURUS')
        except FileNotFoundError as e:
            print(f'\n[skip] {name}: missing file -> {e.filename}'); continue
            print(f'\n {name} baseline -> thesaurus:')
            for cls in _names(task):
                print(f' {cls:<15} recall {b[cls]["recall"]:.3f} ->
↳{t[cls]["recall"]:.3f}'
                      f' | F1 {b[cls]["f1"]:.3f} -> {t[cls]["f1"]:.3f}'
                      f' | TP {b[cls]["tp"]}/{b[cls]["support"]} -> {t[cls]["tp"]}/
↳{t[cls]["support"]}')
                for model, d in (('baseline', b), ('thesaurus', t)):
                    rows.append(dict(experiment=name, model=model, cls=cls,
                                    tp=d[cls]['tp'], support=d[cls]['support'],
                                    precision=round(d[cls]['precision'], 3),
                                    recall=round(d[cls]['recall'], 3),
                                    f1=round(d[cls]['f1'], 3)))

print('-' * 78)

```

=== ACD_AK BASELINE (pooled over 8 aspects, n=328) ===

class	TP/supp	prec	recall	F1
tidak_relevan	48/84	0.667	0.571	0.615
relevan	220/244	0.859	0.902	0.880
pooled macro-F1				0.748
pooled wtd-F1				0.812

(headline macro-F1 on the slides = MEAN of the 8 per-aspect macro-F1,
a different aggregation from this pooled macro-F1)

=== ACD_AK THESAURUS (pooled over 8 aspects, n=328) ===

class	TP/supp	prec	recall	F1
tidak_relevan	56/84	0.629	0.667	0.647
relevan	211/244	0.883	0.865	0.874
pooled macro-F1				0.761
pooled wtd-F1				0.816

(headline macro-F1 on the slides = MEAN of the 8 per-aspect macro-F1,
a different aggregation from this pooled macro-F1)

ACD_AK baseline -> thesaurus:

tidak_relevan	recall 0.571 -> 0.667		F1 0.615 -> 0.647		TP 48/84
-> 56/84					
relevan	recall 0.902 -> 0.865		F1 0.880 -> 0.874		TP 220/244
-> 211/244					

=== ACD_AL BASELINE (pooled over 8 aspects, n=344) ===

class	TP/supp	prec	recall	F1
tidak_relevan	66/85	0.759	0.776	0.767
relevan	238/259	0.926	0.919	0.922
pooled macro-F1				0.845
pooled wtd-F1				0.884

(headline macro-F1 on the slides = MEAN of the 8 per-aspect macro-F1,
a different aggregation from this pooled macro-F1)

=== ACD_AL THESAURUS (pooled over 8 aspects, n=344) ===

class	TP/supp	prec	recall	F1
tidak_relevan	66/85	0.750	0.776	0.763
relevan	237/259	0.926	0.915	0.920
pooled macro-F1				0.842
pooled wtd-F1				0.882

(headline macro-F1 on the slides = MEAN of the 8 per-aspect macro-F1,
a different aggregation from this pooled macro-F1)

ACD_AL baseline -> thesaurus:

tidak_relevan	recall 0.776 -> 0.776		F1 0.767 -> 0.763		TP 66/85
-> 66/85					
relevan	recall 0.919 -> 0.915		F1 0.922 -> 0.920		TP 238/259
-> 237/259					

=== ASC_AK BASELINE (pooled over 8 aspects, n=243) ===

class	TP/supp	prec	recall	F1
positif	75/140	0.750	0.536	0.625
negatif	3/25	0.115	0.120	0.118
netral	47/78	0.402	0.603	0.482
pooled macro-F1				0.408

```

pooled wtd-F1                                0.527
(headline macro-F1 on the slides = MEAN of the 8 per-aspect macro-F1,
 a different aggregation from this pooled macro-F1)

=== ASC_AK  THESAURUS  (pooled over 8 aspects, n=243) ===
class      TP/supp    prec  recall    F1
positif    110/140    0.679  0.786    0.728
negatif     5/25     0.417  0.200    0.270
netral     27/78     0.391  0.346    0.367
pooled macro-F1                                0.455
pooled wtd-F1                                0.565
(headline macro-F1 on the slides = MEAN of the 8 per-aspect macro-F1,
 a different aggregation from this pooled macro-F1)

ASC_AK  baseline -> thesaurus:
  positif      recall 0.536 -> 0.786   | F1 0.625 -> 0.728   | TP 75/140
-> 110/140
  negatif      recall 0.120 -> 0.200   | F1 0.118 -> 0.270   | TP 3/25 ->
5/25
  netral       recall 0.603 -> 0.346   | F1 0.482 -> 0.367   | TP 47/78
-> 27/78
-----

=== ASC_AL  BASELINE  (pooled over 8 aspects, n=286) ===
class      TP/supp    prec  recall    F1
positif    75/163    0.758  0.460    0.573
negatif    11/24     0.314  0.458    0.373
netral     64/99     0.421  0.646    0.510
pooled macro-F1                                0.485
pooled wtd-F1                                0.534
(headline macro-F1 on the slides = MEAN of the 8 per-aspect macro-F1,
 a different aggregation from this pooled macro-F1)

=== ASC_AL  THESAURUS  (pooled over 8 aspects, n=286) ===
class      TP/supp    prec  recall    F1
positif    97/163    0.752  0.595    0.664
negatif     4/24     0.250  0.167    0.200
netral     65/99     0.461  0.657    0.542
pooled macro-F1                                0.469
pooled wtd-F1                                0.583
(headline macro-F1 on the slides = MEAN of the 8 per-aspect macro-F1,
 a different aggregation from this pooled macro-F1)

ASC_AL  baseline -> thesaurus:
  positif      recall 0.460 -> 0.595   | F1 0.573 -> 0.664   | TP 75/163
-> 97/163
  negatif      recall 0.458 -> 0.167   | F1 0.373 -> 0.200   | TP 11/24
-> 4/24

```

```

    netral          recall 0.646 -> 0.657    |  F1 0.510 -> 0.542    |  TP 64/99
-> 65/99

```

9.5 (optional) Save the pooled table to Drive

```

[ ]: pooled = pd.DataFrame(rows)
    out = f'{BASE_PATH}/results/pooled_per_class_metrics.xlsx'
    pooled.to_excel(out, index=False)
    print('Saved ->', out)
    pooled

```

Saved -> /content/drive/MyDrive/TA/results/pooled_per_class_metrics.xlsx

```

[ ]:
  experiment      model      cls  tp  support  precision  recall  \
0    ACD_AK  baseline  tidak_relevan  48      84      0.667  0.571
1    ACD_AK  thesaurus  tidak_relevan  56      84      0.629  0.667
2    ACD_AK  baseline      relevan  220     244      0.859  0.902
3    ACD_AK  thesaurus      relevan  211     244      0.883  0.865
4    ACD_AL  baseline  tidak_relevan  66      85      0.759  0.776
5    ACD_AL  thesaurus  tidak_relevan  66      85      0.750  0.776
6    ACD_AL  baseline      relevan  238     259      0.926  0.919
7    ACD_AL  thesaurus      relevan  237     259      0.926  0.915
8    ASC_AK  baseline      positif  75     140      0.750  0.536
9    ASC_AK  thesaurus      positif  110     140      0.679  0.786
10   ASC_AK  baseline      negatif   3      25      0.115  0.120
11   ASC_AK  thesaurus      negatif   5      25      0.417  0.200
12   ASC_AK  baseline      netral   47      78      0.402  0.603
13   ASC_AK  thesaurus      netral   27      78      0.391  0.346
14   ASC_AL  baseline      positif  75     163      0.758  0.460
15   ASC_AL  thesaurus      positif  97     163      0.752  0.595
16   ASC_AL  baseline      negatif  11      24      0.314  0.458
17   ASC_AL  thesaurus      negatif   4      24      0.250  0.167
18   ASC_AL  baseline      netral   64     99      0.421  0.646
19   ASC_AL  thesaurus      netral   65     99      0.461  0.657

```

```

      f1
0  0.615
1  0.647
2  0.880
3  0.874
4  0.767
5  0.763
6  0.922
7  0.920
8  0.625
9  0.728
10 0.118

```

```

11 0.270
12 0.482
13 0.367
14 0.573
15 0.664
16 0.373
17 0.200
18 0.510
19 0.542

```

9.6 Full pooled confusion matrices (the REAL off-diagonal counts)

The per-class table above only shows the diagonal (TP) and the recall/precision margins. From those alone the *misrouting* cells (e.g. how many **positif** went to **netral**) are **not** uniquely determined. This section prints the complete pooled matrix straight from the prediction files, so every off-diagonal count is a real, readable number — nothing inferred.

```

[ ]: def pooled_confusion(true, pred, task, title, show=True):
    """Full pooled confusion matrix (rows = TRUE, cols = PREDICTED) with
    ↪margins.
    Off-diagonal cells are the REAL misrouting counts, read directly from the
    prediction files -- not inferred from precision/recall. recall = row
    ↪diagonal
    / row total; precision = col diagonal / col total."""
    names = _names(task); ids = list(range(len(names)))
    cm = confusion_matrix(true, pred, labels=ids)           # rows=true,
    ↪cols=pred
    row_tot = cm.sum(axis=1)                               # support per true
    ↪class
    col_tot = cm.sum(axis=0)                               # times each class
    ↪predicted
    diag = np.diag(cm)
    recall = np.divide(diag, row_tot, out=np.zeros(len(ids), float),
    ↪where=row_tot > 0)
    precision = np.divide(diag, col_tot, out=np.zeros(len(ids), float),
    ↪where=col_tot > 0)
    if show:
        print(f'\n=== {title} confusion matrix (pooled, n={int(cm.sum())})
        ↪===')
        print(' rows = TRUE, cols = PREDICTED\n')
        head = f'{"true \\" pred":<16}' + ''.join(f'{n:>14}' for n in names) +
        ↪f'{"| support":>12}{recall:>9}'
        print(head)
        for i, n in enumerate(names):
            body = ''.join(f'{cm[i, j]:>14}' for j in range(len(names)))
            print(f'{n:<16}{body} |{row_tot[i]:>8}{recall[i]:>9.3f}')
        print('-' * len(head))

```

```

        print(f'{"pred total":<16}' + ''.join(f'{col_tot[j]:>14}' for j in
↳range(len(names))))
        print(f'{"precision":<16}' + ''.join(f'{precision[j]:>14.3f}' for j in
↳range(len(names))))
    dfm = pd.DataFrame(cm, index=[f'true_{n}' for n in names],
                        columns=[f'pred_{n}' for n in names])
    dfm['support'] = row_tot
    dfm['recall'] = recall.round(3)
    return dfm

```

```

[ ]: CM_TABLES = {} # (experiment, model) -> labelled confusion-matrix DataFrame

for name in EXPERIMENTS:
    task = _task(name)
    paths = {'baseline': f'{DIRS["baseline"]}/{FILE.format(name=name)}',
             'thesaurus': f'{DIRS["thesaurus"]}/{FILE.format(name=name)}'}
    for model, path in paths.items():
        try:
            true, pred = pooled_from_excel(path, task)
        except FileNotFoundError as e:
            print(f'\n[skip] {name} {model}: missing file -> {e.filename}');
↳continue
        CM_TABLES[(name, model)] = pooled_confusion(true, pred, task, f'{name}
↳{model.upper()}')
        print('=' * 78)

```

=== ACD_AK BASELINE confusion matrix (pooled, n=328) ===

rows = TRUE, cols = PREDICTED

true \ pred	tidak_relevan	relevan		support	recall
tidak_relevan	48	36		84	0.571
relevan	24	220		244	0.902

pred total	72	256			
precision	0.667	0.859			

=== ACD_AK THESAURUS confusion matrix (pooled, n=328) ===

rows = TRUE, cols = PREDICTED

true \ pred	tidak_relevan	relevan		support	recall
tidak_relevan	56	28		84	0.667
relevan	33	211		244	0.865

pred total	89	239			
precision	0.629	0.883			

=====

```
=== ACD_AL BASELINE confusion matrix (pooled, n=344) ===
rows = TRUE, cols = PREDICTED
```

true \ pred	tidak_relevan	relevan	support	recall
tidak_relevan	66	19	85	0.776
relevan	21	238	259	0.919

pred total	87	257		
precision	0.759	0.926		

```
=== ACD_AL THESAURUS confusion matrix (pooled, n=344) ===
rows = TRUE, cols = PREDICTED
```

true \ pred	tidak_relevan	relevan	support	recall
tidak_relevan	66	19	85	0.776
relevan	22	237	259	0.915

pred total	88	256		
precision	0.750	0.926		

```
=== ASC_AK BASELINE confusion matrix (pooled, n=243) ===
rows = TRUE, cols = PREDICTED
```

true \ pred	positif	negatif	netral	support	recall
positif	75	13	52	140	0.536
negatif	4	3	18	25	0.120
netral	21	10	47	78	0.603

pred total	100	26	117		
precision	0.750	0.115	0.402		

```
=== ASC_AK THESAURUS confusion matrix (pooled, n=243) ===
rows = TRUE, cols = PREDICTED
```

true \ pred	positif	negatif	netral	support	recall
positif	110	2	28	140	0.786
negatif	6	5	14	25	0.200
netral	46	5	27	78	0.346

pred total	162	12	69		
precision	0.679	0.417	0.391		

```
=== ASC_AL BASELINE confusion matrix (pooled, n=286) ===
rows = TRUE, cols = PREDICTED
```

true \ pred	positif	negatif	netral		support	recall
positif	75	7	81		163	0.460
negatif	6	11	7		24	0.458
netral	18	17	64		99	0.646

pred total	99	35	152			
precision	0.758	0.314	0.421			

```
=== ASC_AL THESAURUS confusion matrix (pooled, n=286) ===
rows = TRUE, cols = PREDICTED
```

true \ pred	positif	negatif	netral		support	recall
positif	97	3	63		163	0.595
negatif	7	4	13		24	0.167
netral	25	9	65		99	0.657

pred total	129	16	141			
precision	0.752	0.250	0.461			

```
[ ]: # Save every pooled confusion matrix to one multi-sheet workbook.
cm_out = f'{BASE_PATH}/results/pooled_confusion_matrices.xlsx'
with pd.ExcelWriter(cm_out) as xl:
    for (name, model), dfm in CM_TABLES.items():
        dfm.to_excel(xl, sheet_name=f'{name}_{model}'[:31]) # 31 = Excel
        ↪ sheet-name limit
print('Saved ->', cm_out)
```

Saved -> /content/drive/MyDrive/TA/results/pooled_confusion_matrices.xlsx

9.7 (optional) Heatmaps — mirrors the appendix asc_cm / acd_cm figures

```
[ ]: import matplotlib.pyplot as plt

def plot_pooled_cm(dfm, task, title, save_path=None):
    names = _names(task)
    cm = dfm[[f'pred_{n}' for n in names]].values
    fig, ax = plt.subplots(figsize=(4.2, 3.6))
    im = ax.imshow(cm, cmap='Blues')
    ax.set_xticks(range(len(names))); ax.set_xticklabels(names, rotation=45,
    ↪ ha='right')
    ax.set_yticks(range(len(names))); ax.set_yticklabels(names)
    ax.set_xlabel('predicted'); ax.set_ylabel('true'); ax.set_title(title)
    thr = cm.max() / 2 if cm.max() else 0.5
    for i in range(len(names)):
        for j in range(len(names)):
            ax.text(j, i, int(cm[i, j]), ha='center', va='center',
            color='white' if cm[i, j] > thr else 'black')
```

```

fig.tight_layout()
if save_path:
    fig.savefig(save_path, dpi=150, bbox_inches='tight')
plt.show()

os.makedirs(f'{BASE_PATH}/results/cm_figs', exist_ok=True)
for (name, model), dfm in CM_TABLES.items():
    plot_pooled_cm(dfm, _task(name), f'{name} {model}',
                   save_path=f'{BASE_PATH}/results/cm_figs/cm_{name}_{model}.
↳png')

```